

True Single-Theme Pull Request Playbook — μOS++ Ille SMP Integration

Status 2026-10-06. This guide describes the earlier progressive-PR procedure; the project uses the commit-only procedure of `xpack-dev-smp.md` instead. The migration from `xpack-development` to `smp` was then executed with the commit-only procedure of `xpack-dev-smp.md` Part II: no `scripts/smp/`, no chunk recipes, one subject per commit, only added or modified files. The result is recorded in its Part III: 343 commits over `micro-os-plus-iii`, `-posix-arch`, `-cortexm`, `-aarch32` and `-aarch64`; `micro-os-plus-iii-devices` dissolved into the `ports/drivers/` and `soc/<chip>/` (no `UOS_DEVICES_DIR`); every commit built; the full `xpm` test run (native, QEMU Cortex-M, `test-smp-cmake`) green. Where this document disagrees with it (step or PR numbering, scripts, the devices repository, test counts), `xpack-dev-smp.md` applies.

Exclusively Single-Subject Pull Requests on GitHub (github.com/dan-maio → github.com/micro-os-plus)

1. Architectural Principle: Pure Single-Theme Pull Requests

In the initial migration scratchpad, early fixes were artificially bundled under generic buckets (e.g. "Step 1" lumped together POSIX `dirent.h` empty structs, C++ template list iterators in `lists.h`, `newlib` syscall macros, and exported thread suspend symbols).

These changes are completely unrelated. A maintainer reviewing C++ intrusive list iterators should not have to review POSIX directory structs or `newlib` syscall aliases.

This playbook enforces a **pure single-subject / single-theme architecture**: every pull request addresses **exactly one functional concern, one bug, or one subsystem**, ordered from uniprocessor bugfixes to full SMP multi-core integration.

```
+-----+
+
|           TRUE SINGLE-THEME PULL REQUEST PROGRESSION
| ARCHITECTURE           |
+-----+
+
--- PHASE 1: ATOMIC UNIPROCESSOR BUGFIXES & CONFORMANCE (ZERO SMP
MACROS) ---

[PR #1: POSIX dirent.h ISO C Empty Struct Conformance]
|
v
[PR #2: Intrusive List (lists.h) Iterator Member Access Semantics]
|
v
[PR #3: Newlib Syscall Return Type Macro (c-syscalls-aliases-
standard.h)]
|
```

```

v
[PR #4: Thread Suspend Symbol Export for CMSIS C Wrapper (os-
thread.cpp)]
|
v
[PR #5: Memory Usable Size & Calloc Multiplication Overflow Guard]
|
v
[PR #6: ISO C++17 Aligned Memory Allocation Operators (operator new/
delete)]
|
v
[PR #7: Static Error Category Storage for System Errors]
|
v
[PR #8: Steady Clock 64-Bit Tick Duration Overflow Protection]
|
v
[PR #9: CMSIS-RTOS v1 C API Wrapper Cleanups (os-c-wrapper.cpp)]
|
v
[PR #10: POSIX I/O File Descriptor Table Critical Section Mutexing]
|
v
[PR #11: POSIX I/O Deferred List Critical Section Protection]
|
v
[PR #12: Block Device Sector Size Validation Check]
|
v
[PR #13: ARMv8-M Architecture Semihosting Trap Instruction Guards]
|
v
[PR #14: ARMv8-M Mainline Architecture Exception Handlers Support]
|
v
[PR #15: Semihosting Syscall fstat() Fallback Mode & Board Console
Mirror]
|
v
[PR #16: Software Timer Callback Execution Decoupling & Drift
Prevention]
|
v
[PR #17: POSIX Mutex Priority Ceiling & Dynamic Inheritance Boost]
|
v
[PR #18: Thread Lifecycle State Machine (state::destroying) &
Reaper]
|
v
[PR #19: Condition Variable Lost-Signal Elimination & Clock Binding]
|
v
[PR #20: C++ std::thread Native Handle Synchronization & Functor
Lifetime]
|
v
[PR #21: Message Queue Preemptive Rescheduling on Wakeups]
|
v
[PR #22: High-Resolution Monotonic Clock Port Synchronization (K + C
+ P)]

--- PHASE 2: MULTI-CORE SMP INFRASTRUCTURE & GENUINE HOST VALIDATION
---
```

```

[PR #23: Multi-Core SMP Kernel Infrastructure, Ports & native-smp
Host]

--- PHASE 3: RELEASES, ARCHITECTURE DISSOLUTION & SILICON EXTENSIONS
---

[PR #24: Port Package Semantic Releases (posix-arch v1.1.0, cortexm
v1.2.0)]
    |
    v
[PR #25: History-Preserving Devices Repository Subtree Dissolution]
    |
    v
[PR #26: Modern Multi-Core Silicon Ports (ARM Cortex-M33 & RP2350)]
    |
    v
[PR #27: Modular Add-Only CMake Build Architecture & Cross
Toolchains]

--- PHASE 4: MULTI-CORE TEST SUITES, MULTI-ARCH PLATFORMS & CI
INTEGRATION ---

[PR #28: Multi-Core Hardware FPU Context Switch Stress Suite]
    |
    v
[PR #29: Multi-Architecture 4-Core QEMU Platforms & Composite CI
Action]

--- PHASE 5: DOCUMENTATION, FINAL MERGE & SILICON WIRING ---

[PR #30: Architectural Documentation & Specification Runbooks Suite]
    |
    v
[PR #31: Final Master Reconciliation, CI Metadata Restoration &
Cleanup]
    |
    v
[PR #32: Dedicated Hardware Silicon Platform Wiring (Raspberry Pi
Pico 2)]
+-----+
+

```

1.1 Mapping the 7 Progressive Macro PRs to the Granular PR Catalog

Depending on maintainer review preferences, the migration series can be submitted either as **7 Progressive Macro PRs** (as defined in [pull-request.md](#) and [files-modif-by-step.md](#)) or as **32 Pure Single-Theme PRs** (detailed below in §2):

Progressive Macro PR	Granular Single-Theme PRs	Underlying Steps	Repositories	Core Domain & Focus	Target Base ←\leftarrow Head
Macro PR #1: Single-Core Defect Corrections & Hardening	PR #1 – PR #22	Steps 1-13 (Part A)	K, +C+P at Step 13	ISO C/C++, Allocators, POSIX I/O, Condvars, Mutexes, Clocks	xpack-development ←\leftarrow step/13
Macro PR #2: Core SMP Architecture &	PR #23	Steps 14-23 (Part B collapsed)	K, C, P	SMP scheduler, _smp_klock,	step/13 ←\leftarrow step/14

Progressive Macro PR	Granular Single-Theme PRs	Underlying Steps	Repositories	Core Domain & Focus	Target Base ←\leftarrow Head
Native Host Platform				ready lists, stack switch, IPI, native-smp 2-core test	
Macro PR #3: Multi-Architecture Port Releases	PR #24	Step 24 (Part C1)	P, C	Semantic version bumps: posix-arch v1.1.0, cortexm v1.2.0	step/14 ←\leftarrow step/24
Macro PR #4: Dissolution & Subtree Ingestion of devices	PR #25	Part 0	P, C, A32, A64	Subtree ingestion of soc/*, drivers/*, CMake ALIAS shims	part0-devices (out-of-band)
Macro PR #5: ARMv8-M Silicon Ports & Modular CMake	PR #26 & PR #27	Steps 25-26 (Part C2)	K, C, P, Arch	Cortex-M33, RP2350, modular toolchains, AN521 platform	step/24 ←\leftarrow step/26
Macro PR #6: SMP Validation Testbed & Multi-Arch Harness	PR #28 & PR #29	Steps 27-28 (Part C3)	K, C, P, Arch	fp-switch, tests/smp-support/, 4-core RPi3B QEMU targets, test-smp-cmake	step/26 ←\leftarrow step/28
Macro PR #7: Documentation Suite & Master Reconciliation	PR #30 & PR #31	Steps 29-30 (Part D)	All Repos	PDF suite build, .github/** pristine restoration, scripts/smp/ purge	step/28 ←\leftarrow step/30
Appendix Step 31: Dedicated Pico 2 Hardware Platform	PR #32	Step 31	C, K	Physical RP2350 silicon SWD flashing & SIO boot mailbox	cortexm-pico2

2. Catalog of Pure Single-Theme Pull Requests

PR #1: POSIX dirent.h ISO C Empty Struct Conformance

- **Single Theme:** ISO C99/C11 syntax validity and -Wextra-semi compiler warning elimination.
- **Repository:** micro-os-plus-iii
- **File Touched:** include/cmsis-plus/posix/dirent.h
- **Problem & Solution:**
 - typedef struct { ; } DIR; violates ISO C99/C11 §6.7.2.1 (a struct declaration must specify at least one named member) and triggers -Wextra-semi under strict compilation flags.
 - Adding int reserved; with an explanatory comment complies with standard C requirements and eliminates compiler warnings on strict toolchains.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/posix/dirent.h b/include/cmsis-plus/
posix/dirent.h
index 1f0f0dbe..19a5165c 100644
--- a/include/cmsis-plus/posix/dirent.h
+++ b/include/cmsis-plus/posix/dirent.h
@@ -54,7 +54,8 @@ extern "C"
     // and casted to DIR.
     typedef struct
     {
-        ;
+        int reserved; /* C forbids an empty struct; a named member also
+            avoids
+
+            -Wextra-semi on the old `;` null declaration. */
     } DIR;

     // -----
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
--base xpack-development --head dan-maio:fix/posix-dirent-c-
conformance \
--title "fix(posix): dirent.h ISO C empty struct conformance" \
--body "Adds 'int reserved;' to DIR struct to comply with ISO C99/
C11 §6.7.2.1 and eliminate -Wextra-semi."
```

PR #2: Intrusive Double-List (lists.h) Iterator Member Access Semantics

- **Single Theme:** Modern C++ standard template iterator concept conformance.
- **Repository:** micro-os-plus-iii
- **File Touched:** include/cmsis-plus/utils/lists.h
- **Problem & Solution:**
 - In include/cmsis-plus/utils/lists.h, the double_list_iterator operators ++ and -- (prefix and postfix) accessed node_->next and node_->prev directly as member variables instead of invoking accessor member functions node_->next() / node_->prev().
 - In the intrusive list node design, next and prev pointers are exposed via member accessor methods. Accessing them without parentheses causes template compilation failures under strict C++17/C++20 flags where member function pointers cannot be implicitly accessed as data fields.
 - Calling node_->next () and node_->prev () restores standard iterator member access semantics and guarantees clean template instantiation across GCC and Clang.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/utils/lists.h b/include/cmsis-plus/
utils/lists.h
index b8c642d9..f80da4dd 100644
--- a/include/cmsis-plus/utils/lists.h
+++ b/include/cmsis-plus/utils/lists.h
@@ -917,7 +917,7 @@ namespace os
     double_list_iterator<T, N, MP, U>::operator++ (int)
     {
         const auto tmp = *this;
-        node_ = static_cast<iterator_pointer> (node_>next);
+        node_ = static_cast<iterator_pointer> (node_>next ());
         return tmp;
     }

@@ -925,7 +925,7 @@ namespace os
     inline double_list_iterator<T, N, MP, U>&
     double_list_iterator<T, N, MP, U>::operator-- ()
     {
-        node_ = static_cast<iterator_pointer> (node_>prev);
+        node_ = static_cast<iterator_pointer> (node_>prev ());
         return *this;
     }

@@ -934,7 +934,7 @@ namespace os
     double_list_iterator<T, N, MP, U>::operator-- (int)
     {
         const auto tmp = *this;
-        node_ = static_cast<iterator_pointer> (node_>prev);
+        node_ = static_cast<iterator_pointer> (node_>prev ());
         return tmp;
     }
}
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
--base xpack-development --head dan-maio:fix/list-iterator-
semantics \
--title "fix(utils): double_list_iterator member function access
semantics" \
--body "Corrects double_list_iterator operators to invoke node_-
>next() / node_>prev() member functions."
```

PR #3: Newlib C Syscall Weak Aliases Return Type Macro

- **Single Theme:** Newlib 64-bit libc syscall signature compatibility.
- **Repository:** micro-os-plus-iii
- **File Touched:** include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
- **Problem & Solution:**
 - Newlib on 64-bit architectures uses `_READ_WRITE_RETURN_TYPE` (int on aarch64-none-elf and `_ssize_t` on arm-none-eabi), whereas POSIX specifies `ssize_t`.
 - Maps `CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE` to newlib's `_READ_WRITE_RETURN_TYPE`, eliminating conflicting declaration compiler errors.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
      b/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
index e5703bb3..90acafe3 100644
--- a/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
+++ b/include/cmsis-plus/posix-io/c-syscalls-aliases-standard.h
@@ -17,6 +17,16 @@
 #ifdef __cplusplus
     extern "C"
     {
+endif
+
+ // newlib declares read()/write() with its own return type
+ // (_READ_WRITE_RETURN_TYPE, which is `int` on aarch64-none-elf and
+ // `_ssize_t` on arm-none-eabi). Match it, or the alias below
+ // conflicts
+ // with the system declaration.
+#if defined(_READ_WRITE_RETURN_TYPE)
+#define CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE _READ_WRITE_RETURN_TYPE
+#else
+#define CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE ssize_t
+#endif

     // -----
@@ -130,7 +140,7 @@ extern "C"
     int __attribute__((weak, alias ("__posix_raise")))
     raise (int sig);

-    ssize_t __attribute__((weak, alias ("__posix_read")))
+    CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE __attribute__((weak, alias
        ("__posix_read")))
     read (int fildes, void* buf, size_t nbyte);

     struct dirent* __attribute__((weak, alias ("__posix_readdir")))
@@ -247,7 +257,7 @@ extern "C"
     pid_t __attribute__((weak, alias ("__posix_wait")))
     wait (int* stat_loc);

-    ssize_t __attribute__((weak, alias ("__posix_write")))
+    CMSIS_PLUS_POSIX_IO_RW_RETURN_TYPE __attribute__((weak, alias
        ("__posix_write")))
     write (int fildes, const void* buf, size_t nbyte);

     ssize_t __attribute__((weak, alias ("__posix_writew")))

```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/newlib-syscall-return-
  type \
  --title "fix(posix-io): harmonize syscall return types with newlib
  _READ_WRITE_RETURN_TYPE"
```

PR #4: Thread Suspend External Symbol Export for CMSIS C Wrapper

- **Single Theme:** Linker external symbol visibility for thread suspension.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/rtos/os-thread.cpp
- **Problem & Solution:**
 - `this_thread::suspend()` is declared non-inline in `os-thread.h`, but was defined out-of-line with `inline` in `os-thread.cpp`.
 - Marking it `inline` in `.cpp` caused the compiler not to emit an external symbol, producing an undefined reference when called from the C API wrapper `os_this_thread_suspend`. Removing `inline` exports the symbol cleanly.
- **Exact Unified Diff Applied:**

```
diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
index 6476782f..3b8f293a 100644
--- a/src/rtos/os-thread.cpp
+++ b/src/rtos/os-thread.cpp
@@ -1837,7 +1837,7 @@ namespace os
     *
     * @warning Cannot be invoked from Interrupt Service Routines.
     */
-    inline void
+    void
     suspend (void)
     {
         os::instrumentation::thread::suspend (_thread ());
     }
 }
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/thread-suspend-symbol-
  export \
  --title "fix(rtos): export out-of-line this_thread::suspend symbol
  for C wrapper"
```

PR #5: Memory Allocator Usable Size & Integer Overflow Protection

- **Single Theme:** Memory resource introspection and heap integer multiplication overflow protection.
- **Repository:** micro-os-plus-iii
- **Files Touched:** include/cmsis-plus/rtos/os-memory.h, include/cmsis-plus/memory/first-fit-top.h, src/rtos/os-memory.cpp, src/memory/first-fit-top.cpp, src/memory/lifo.cpp, src/memory/block-pool.cpp, src/libc/stdlib/malloc.cpp, include/cmsis-plus/estd/memory_resource
- **Problem & Solution:**
 - Protects calloc() in malloc.cpp against integer multiplication overflow (nelem > SIZE_MAX / elbytes), preventing tiny buffer allocations for huge sizes.
 - Adds align_size() overflow check in os-memory.h.
 - Implements memory_resource::do_usable_size() base declaration and first_fit_top override, enabling realloc() to query the old block size and avoid copying out-of-bounds bytes when growing an allocation.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/estd/memory_resource b/include/cmsis-plus/estd/memory_resource
index befafb6c..5beb6427 100644
--- a/include/cmsis-plus/estd/memory_resource
+++ b/include/cmsis-plus/estd/memory_resource
@@ -243,7 +243,10 @@ namespace os
     polymorphic_allocator<T>::select_on_container_copy_construction (
         void) const noexcept
     {
-        return polymorphic_allocator ();
+        // C++17 requires a copy of the allocator, not a default-constructed
+        // one; returning a default allocator would silently rebind a copied
+        // container to the global resource instead of its own.
+        return *this;
     }

     template <typename T>
diff --git a/include/cmsis-plus/memory/first-fit-top.h b/include/cmsis-plus/memory/first-fit-top.h
index 39ac2791..c3942395 100644
--- a/include/cmsis-plus/memory/first-fit-top.h
+++ b/include/cmsis-plus/memory/first-fit-top.h
@@ -213,6 +213,14 @@ namespace os
     virtual std::size_t
     do_max_size (void) const noexcept override;

+    /**
+     * @brief Implementation of the function to get the usable
+     *        block size.
+     * @param [in] addr Address of a previously allocated block.
+     * @return Number of usable bytes from `addr` to the end of its
+     *        chunk.
+     */
+    virtual std::size_t
+    do_usable_size (void* addr) const noexcept override;
+
     /**
     * @brief Implementation of the function to reset the memory
     *        manager.
     * @par Parameters
diff --git a/include/cmsis-plus/rtos/os-memory.h b/include/cmsis-plus/rtos/os-memory.h
index 3d808899..3f567594 100644
--- a/include/cmsis-plus/rtos/os-memory.h
+++ b/include/cmsis-plus/rtos/os-memory.h
```

```

@@ -84,6 +84,10 @@ namespace os
    constexpr std::size_t
    align_size (std::size_t size, std::size_t align) noexcept
    {
+       if (size > static_cast<std::size_t> (-1) - (align - 1L))
+       {
+           return static_cast<std::size_t> (-1);
+       }
        return ((size) + (align)-1L) & ~((align)-1L);
    }

@@ -274,6 +278,21 @@ namespace os
    std::size_t
    max_size (void) const noexcept;

+    /**
+     * @brief Get the usable size of a previously allocated
+     * block.
+     * @param addr Address of a block returned by `allocate()`.
+     * @return Number of usable bytes, or 0 if the size is
+     * unknown.
+     *
+     * @details
+     * This is an extension used by `realloc()` to copy at most
+     * the
+     * old contents; the allocator is the only one who knows the
+     * real
+     * size of a block when the caller passed 0 to
+     * `deallocate()`.
+     *
+     * @see do_usable_size();
+     */
+    std::size_t
+    usable_size (void* addr) const noexcept;
+
+    /**
+     * @brief Set the out of memory handler.
+     * @param handler Pointer to new handler.
+
@@ -422,6 +441,18 @@ namespace os
    virtual std::size_t
    do_max_size (void) const noexcept;

+    /**
+     * @brief Implementation of the function to get the usable
+     * block size.
+     * @param addr Address of a previously allocated block.
+     * @return Number of usable bytes, or 0 if the size is
+     * unknown.
+     *
+     * @details
+     * The default implementation returns 0 (unknown); allocators
+     * that
+     * keep the block size, like `first_fit_top`, override it.
+     */
+    virtual std::size_t
+    do_usable_size (void* addr) const noexcept;
+
+    /**
+     * @brief Implementation of the function to reset the memory
+     * manager.
+     * @par Parameters
+
@@ -1342,6 +1373,15 @@ namespace os
    return do_max_size ();
    }

+    /**
+     * @see do_usable_size();
+     */
+    inline std::size_t
+    memory_resource::usable_size (void* addr) const noexcept

```

```

+     {
+         return do_usable_size (addr);
+     }
+
+     /**
+      * @see do_reset();
+      */
diff --git a/src/libc/stdlib/malloc.cpp b/src/libc/stdlib/malloc.cpp
index 9ff93d07..91ffcc4a 100644
--- a/src/libc/stdlib/malloc.cpp
+++ b/src/libc/stdlib/malloc.cpp
@@ -160,6 +160,17 @@ @@ calloc (size_t nelem, size_t elbytes)
     return nullptr;
 }

+ // Reject requests whose total size would overflow `size_t`.
+ // Without this
+ // guard the product wraps to a small value (e.g. on a 32-bit
+ // target
+ // `calloc(0x10001, 0x10000)` becomes 0) and a tiny block is
+ // returned while
+ // the caller believes the full, huge array was allocated.
+ // `elbytes` is
+ // known non-zero here, so the division is safe.
+ if (nelem > static_cast<std::size_t> (-1) / elbytes)
+ {
+     errno = ENOMEM;
+     return nullptr;
+ }
+
void* mem;
{
    // ----- Begin of critical section
    -----
@@ -294,7 +305,16 @@ @@ realloc (void* ptr, size_t bytes)
    mem = estd::pmr::get_default_resource ()->allocate (bytes);
    if (mem != nullptr)
    {
-        memcpy (mem, ptr, bytes);
+        // POSIX requires the contents to be preserved "up to the
+        // lesser of
+        // the new and old sizes". The old size is queried from the
+        // allocator
+        // (the bare-metal default resource is a `first_fit_top`,
+        // which keeps
+        // it in the chunk header). Copying `bytes` unconditionally
+        // would read
+        // past the end of the old block whenever the new size is
+        // larger,
+        // i.e. on every growing `realloc()`.
+        std::size_t old_bytes
+            = estd::pmr::get_default_resource ()->usable_size (ptr);
+        std::size_t copy_bytes = (bytes < old_bytes) ? bytes :
+        old_bytes;
+        memcpy (mem, ptr, copy_bytes);
+        estd::pmr::get_default_resource ()->deallocate (ptr, 0);
    }
    else
diff --git a/src/memory/block-pool.cpp b/src/memory/block-pool.cpp
index b3f583d1..a2432310 100644
--- a/src/memory/block-pool.cpp
+++ b/src/memory/block-pool.cpp
@@ -168,7 +168,7 @@ @@ namespace os
     pool_addr_, align_sz);

    // std::align() will fail if it cannot fit the adjusted block
    // size.
-    if (res != nullptr)
+    if (res == nullptr)
    {
        assert (res != nullptr);
    }

```

```

}
diff --git a/src/memory/first-fit-top.cpp b/src/memory/first-fit-
top.cpp
index 09478a00..cce94591 100644
--- a/src/memory/first-fit-top.cpp
+++ b/src/memory/first-fit-top.cpp
@@ -122,13 +122,32 @@ namespace os
{
    using namespace os;

+
+
+   if (bytes > total_bytes_)
+   {
+       return nullptr;
+   }
+
+   std::size_t block_padding = calc_block_padding (alignment);
+   std::size_t alloc_size = rtos::memory::align_size (bytes,
+       chunk_align);
+
+   if (alloc_size == static_cast<std::size_t> (-1)
+       || alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }
+
+   alloc_size += block_padding;
+   alloc_size += chunk_offset;
+
+   if (alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }
+
+   std::size_t block_minchunk = calc_block_minchunk
+       (block_padding);
+   alloc_size = os::rtos::memory::max (alloc_size,
+       block_minchunk);
+
+   if (alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }
+
+   chunk_t* chunk;

@@ -428,6 +447,37 @@ namespace os
    return total_bytes_;
}

+#pragma GCC diagnostic push
+#pragma GCC diagnostic ignored "-Wcast-align"
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#endif
+
+   std::size_t
+   first_fit_top::do_usable_size (void* addr) const noexcept
+   {
+       // Recover the chunk header placed immediately before the
+       // payload.
+       chunk_t* chunk = reinterpret_cast<chunk_t*> (static_cast<char*>
+           (addr)
+
+               - chunk_offset);
+
+       // For an aligned block, the adjusted header stores the
+       // (negative)
+       // alignment offset as its size; step back to the real chunk
+       // header
+       // exactly as do_deallocate() does.
+       if (static_cast<std::ptrdiff_t> (chunk->size) < 0)
+       {
+           chunk = reinterpret_cast<chunk_t*> (
+               reinterpret_cast<char*> (chunk)

```

```

+         + static_cast<std::ptrdiff_t> (chunk->size));
+     }
+
+     // Usable bytes are those from the given address to the end of
+     // the
+     // chunk; this naturally accounts for any alignment slack
+     // between the
+     // chunk payload and the returned pointer.
+     return static_cast<std::size_t> (
+         reinterpret_cast<char*> (chunk) + chunk->size
+         - static_cast<char*> (addr));
+ }
+ #pragma GCC diagnostic pop
+
+ void*
+ first_fit_top::internal_align_ (chunk_t* chunk, std::size_t
+     bytes,
+
+                                     std::size_t alignment)
diff --git a/src/memory/lifo.cpp b/src/memory/lifo.cpp
index 3519fb3e..9924fbf2 100644
--- a/src/memory/lifo.cpp
+++ b/src/memory/lifo.cpp
@@ -70,13 +70,31 @@ namespace os
 void*
 lifo::do_allocate (std::size_t bytes, std::size_t alignment)
 {
+     if (bytes > total_bytes_)
+     {
+         return nullptr;
+     }
+
+     std::size_t block_padding = calc_block_padding (alignment);
+     std::size_t alloc_size = rtos::memory::align_size (bytes,
+         chunk_align);
+
+     if (alloc_size == static_cast<std::size_t> (-1)
+         || alloc_size > total_bytes_)
+     {
+         return nullptr;
+     }
+
+     alloc_size += block_padding;
+     alloc_size += chunk_offset;
+
+     if (alloc_size > total_bytes_)
+     {
+         return nullptr;
+     }
+
+     std::size_t block_minchunk = calc_block_minchunk
+         (block_padding);
+     alloc_size = os::rtos::memory::max (alloc_size,
+         block_minchunk);
+
+     if (alloc_size > total_bytes_)
+     {
+         return nullptr;
+     }
+
+     chunk_t* chunk = nullptr;

@@ -126,6 +144,14 @@ namespace os
         // If this was the last chunk, the free list is
        empty.
    }
    }
    else
    {
        // The head chunk is smaller than the request, so
        // this arena
        // cannot satisfy it. Clear `chunk` so the loop
        // below falls
        // through to the out-of-memory handler (or returns
        nullptr)
    }

```

```

+         // instead of handing the too-small chunk to
+         internal_align_().
+         chunk = nullptr;
+     }
+
+     if (chunk != nullptr)
diff --git a/src/rtos/os-memory.cpp b/src/rtos/os-memory.cpp
index 7b3c9baa..d5f89855 100644
--- a/src/rtos/os-memory.cpp
+++ b/src/rtos/os-memory.cpp
@@ -438,6 +438,23 @@ namespace os
     return 0;
 }

+
+ /**
+  * @details
+  * The default implementation of this virtual function returns
+  * zero, meaning the usable size is not known.
+  *
+  * Override this function to return the actual size.
+  *
+  * @par Standard compliance
+  * Extension to standard.
+  */
+ std::size_t
+ memory_resource::do_usable_size (void* addr) const noexcept
+ {
+     static_cast<void*> (addr);
+     return 0;
+ }
+
+ /**
+  * @details
+  * The default implementation of this virtual function

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:feat/memory-usable-size-
  overflow-guard \
  --title "feat(memory): implement do_usable_size() and guard calloc
  against integer overflow"

```

PR #6: ISO C++17 Aligned Memory Allocation Operators

- **Single Theme:** C++17 over-aligned dynamic memory allocation overloads.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/libcpp/new.cpp
- **Problem & Solution:**
 - Implements C++17 over-aligned operator new(std::size_t, std::align_val_t) and operator delete(void*, std::align_val_t) variants (nothrow, array, and sized versions).
 - Routes requests through the RTOS default memory resource estd::pmr::get_default_resource()->allocate(bytes, alignment).
- **Exact Unified Diff Applied:**

```
diff --git a/src/libcpp/new.cpp b/src/libcpp/new.cpp
index 098df1d9..5203ee12 100644
--- a/src/libcpp/new.cpp
+++ b/src/libcpp/new.cpp
@@ -288,6 +288,135 @@ void* __attribute__((weak)) operator new[]
    (std::size_t bytes,

// -----

+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for a new object instance.
+ * @param bytes Number of bytes to allocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @return Pointer to aligned allocated object.
+ *
+ * @details
+ * C++17 calls this overload for types whose alignment exceeds
+ * `alignof(std::max_align_t)`. The requested alignment is passed
+ * through to
+ * the RTOS default memory resource; the bare-metal `first_fit_top`
+ * honours
+ * it, whereas the plain `size_t` overload only promises `max_align`.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__((weak))
+operator new (std::size_t bytes, std::align_val_t alignment)
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+    if (bytes == 0)
+    {
+        bytes = 1;
+    }
+    // ----- Begin of critical section
+    rtos::scheduler::critical_section scs;
+    while (true)
+    {
+        void* mem = estd::pmr::get_default_resource ()->allocate (
+            bytes, static_cast<std::size_t> (alignment));
+
+        if (mem != nullptr)
+        {
+            return mem;
+        }
+
+        if (new_handler_)
+        {
+

```

```

+         new_handler_ ();
+     }
+     else
+     {
+         estd::_throw_bad_alloc ();
+     }
+ }
+
+ // ----- End of critical section
+ -----
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for a new object instance
+ *        (nothrow).
+ * @param bytes Number of bytes to allocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @param nothrow (unused)
+ * @return Pointer to aligned allocated object or nullptr.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new (std::size_t bytes, std::align_val_t alignment,
+             const std::nothrow_t& nothrow __attribute__ ((unused)))
+    noexcept
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+    if (bytes == 0)
+    {
+        bytes = 1;
+    }
+
+    // ----- Begin of critical section
+    -----
+    rtos::scheduler::critical_section scs;
+
+    while (true)
+    {
+        void* mem = estd::pmr::get_default_resource ()->allocate (
+            bytes, static_cast<std::size_t> (alignment));
+
+        if (mem != nullptr)
+        {
+            return mem;
+        }
+
+        if (new_handler_)
+        {
+            new_handler_ ();
+        }
+        else
+        {
+            break; // return nullptr
+        }
+    }
+
+    // ----- End of critical section
+    -----
+
+    return nullptr;
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for an array of new object
+ *        instances.
+ * @param bytes Number of bytes to allocate.

```

```

+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @return Pointer to aligned allocated object.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new[] (std::size_t bytes, std::align_val_t alignment)
+{
+    return ::operator new (bytes, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for an array of new object
+ * instances (nothrow).
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new[] (std::size_t bytes, std::align_val_t alignment,
+               const std::nothrow_t& nothrow __attribute__
+               ((unused))) noexcept
+{
+    return ::operator new (bytes, alignment, std::nothrow);
+}
+
+// -----
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate the dynamically allocated object instance.
+ * @-516,6 +645,116 @@ void __attribute__ ((weak)) operator delete[]
+ * (void* ptr, const std::nothrow_t
+ * ::operator delete (ptr, nothrow);
+ */
+}
+
+// -----
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object.
+ * @param ptr Pointer to object.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @par Returns
+ * Nothing.
+ *
+ * @details
+ * Counterpart of `operator new(std::size_t, std::align_val_t)`.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__ ((weak))
+operator delete (void* ptr, std::align_val_t alignment) noexcept
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+
+    if (ptr)
+    {
+        // ----- Begin of critical section
+        -----
+
+        rtos::scheduler::critical_section scs;
+
+        estd::pmr::get_default_resource ()->deallocate (
+            ptr, 0, static_cast<std::size_t> (alignment));
+        // ----- End of critical section
+        -----
+    }
+}

```

```

+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object
+ *        (sized).
+ * @param ptr Pointer to object.
+ * @param bytes Number of bytes to deallocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @par Returns
+ *   Nothing.
+ *
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete (void* ptr, std::size_t bytes,
+                std::align_val_t alignment) noexcept
+{
+  assert (!rtos::interrupts::in_handler_mode ());
+
+  if (ptr)
+  {
+    // ----- Begin of critical section
+    -----
+    rtos::scheduler::critical_section scs;
+
+    estd::pmr::get_default_resource ()->deallocate (
+      ptr, bytes, static_cast<std::size_t> (alignment));
+    // ----- End of critical section
+    -----
+  }
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array.
+ *
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete[] (void* ptr, std::align_val_t alignment) noexcept
+{
+  ::operator delete (ptr, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array
+ *        (sized).
+ *
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete[] (void* ptr, std::size_t bytes,
+                std::align_val_t alignment) noexcept
+{
+  ::operator delete (ptr, bytes, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object
+ *        (nothrow).
+ *
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete (void* ptr, std::align_val_t alignment,
+                const std::nothrow_t& nothrow __attribute__((unused))) noexcept

```

```

+{
+  ::operator delete (ptr, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array
+ *        (nothrow).
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete[] (void* ptr, std::align_val_t alignment,
+                  const std::nothrow_t& nothrow __attribute__((unused))) noexcept
+{
+  ::operator delete[] (ptr, alignment);
+}
+
+// error: end of file with unbalanced grouping commands
+/*
+ * @}

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:feat/cpp17-aligned-new-
  delete \
  --title "feat(libcpp): implement ISO C++17 aligned operator new and
  delete"

```

PR #7: Static Error Category Storage for System Errors

- **Single Theme:** Elimination of dangling references to temporary error categories in `std::system_error`.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/libcpp/system-error.cpp`
- **Problem & Solution:**
 - `std::error_code` stores a pointer to its `error_category`. The previous code constructed temporaries (`system_error_category()`, `cmsis_error_category()`) in `__throw_system_error()` and `__throw_cmsis_error()`, leaving dangling pointers in thrown exceptions.
 - Adding function-local static accessors `get_system_error_category()` and `get_cmsis_error_category()` ensures category objects have static storage duration.
- **Exact Unified Diff Applied:**

```
diff --git a/src/libcpp/system-error.cpp b/src/libcpp/system-error.cpp
index c11ad9f0..d64fed39 100644
--- a/src/libcpp/system-error.cpp
+++ b/src/libcpp/system-error.cpp
@@ -21,6 +21,7 @@
 #if defined(__clang__)
 #pragma clang diagnostic ignored "-Wc++98-compat"
 #pragma clang diagnostic ignored "-Wc++98-compat-bind-to-temporary-copy"
+#pragma clang diagnostic ignored "-Wexit-time-destructors"
 #endif

 // -----
@@ -103,17 +104,33 @@ namespace os

 #pragma GCC diagnostic pop

+ // `std::error_code` stores a pointer to its `error_category`,
+ // and the
+ // `error_code` (via the thrown `std::system_error`) outlives the
+ // throw
+ // expression. The category must therefore have static storage
+ // duration;
+ // a temporary would dangle. Function-local statics are
+ // constructed on
+ // first use and live until exit.
+ static const std::error_category&
+ get_system_error_category (void) noexcept
+ {
+     static const system_error_category category;
+     return category;
+ }
+
+ static const std::error_category&
+ get_cmsis_error_category (void) noexcept
+ {
+     static const cmsis_error_category category;
+     return category;
+ }
+
 #endif

 void
 __throw_system_error (int ev, const char* what_arg)
 {
 #if defined(__EXCEPTIONS)
- // error: copying parameter of type
- 'os::estd::system_error_category'
```

```

-      // when binding a reference to a temporary would invoke a
-      deleted
-      // constructor in C++98 [-Werror,-Wc++98-compat-bind-to-
-      temporary-copy]
-      throw std::system_error (std::error_code (ev,
-      system_error_category ()),
-      what_arg);
+      throw std::system_error (
+      std::error_code (ev, get_system_error_category ()),
+      what_arg);
#else
    trace_printf ("system_error(%d, %s)\n", ev, what_arg);
    std::abort ();
@@ -124,10 +141,7 @@ namespace os
    __throw_cmsis_error (int ev, const char* what_arg)
    {
    #if defined(__EXCEPTIONS)
-      // error: copying parameter of type
-      'os::estd::cmsis_error_category' when
-      // binding a reference to a temporary would invoke a deleted
-      constructor
-      // in C++98 [-Werror,-Wc++98-compat-bind-to-temporary-copy]
-      throw std::system_error (std::error_code (ev,
-      cmsis_error_category ()),
+      throw std::system_error (std::error_code (ev,
+      get_cmsis_error_category ()),
+      what_arg);
    #else
        trace_printf ("system_error(%d, %s)\n", ev, what_arg);

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/system-error-category-
  storage \
  --title "fix(libcpp): static error category storage to prevent
  dangling pointers in system_error"

```

PR #8: Steady Clock 64-Bit Duration Multiplicative Overflow Protection

- **Single Theme:** Long-uptime duration integer overflow prevention.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/libcpp/chrono.cpp
- **Problem & Solution:**
 - cycles * 1000000000ULL / freq overflows 64-bit integer arithmetic after ~13 minutes of operation at typical clock frequencies.
 - Decomposes computation into seconds and remainder (sec = cycles / freq, rem = cycles % freq) before multiplying by 1000000000ULL, ensuring overflow-free duration calculations across multi-year uptimes.
- **Exact Unified Diff Applied:**

```
diff --git a/src/libcpp/chrono.cpp b/src/libcpp/chrono.cpp
index dfbe1829..6246956a 100644
--- a/src/libcpp/chrono.cpp
+++ b/src/libcpp/chrono.cpp
@@ -113,13 +113,15 @@ namespace os
 #endif
     // The duration is the number of sum of SysTick ticks plus
     // the current
     // count of CPU cycles (computed from the SysTick counter).
-    // Notice: a more exact solution would be to compute
-    // ticks * divisor + cycles, but this severely reduces the
-    // range of ticks.
+    // Decompose into seconds and remainder to avoid 64-bit
+    // overflow
+    // on cycles * 1e9 after ~13 minutes.
+    uint64_t freq = rtos::hrclock.input_clock_frequency_hz ();
+    uint64_t sec = cycles / freq;
+    uint64_t rem = cycles % freq;
+    uint64_t ns = sec * 1000000000ULL + (rem * 1000000000ULL) /
+    freq;
     return time_point{
         duration{
-            duration{ cycles * 1000000000ULL
-                / rtos::hrclock.input_clock_frequency_hz () }
+            duration{ ns }
+            + realtime_clock::startup_time_point.time_since_epoch
+        () } //
     };
 #pragma GCC diagnostic pop
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/chrono-duration-
  overflow \
  --title "fix(libcpp): prevent 64-bit integer duration overflow in
  steady clock conversions"
```

PR #9: CMSIS-RTOS v1 C API Wrapper Cleanups

- **Single Theme:** CMSIS-RTOS specification conformance in C wrapper.
- **Repository:** micro-os-plus-iii
- **File Touched:** `src/rtos/os-c-wrapper.cpp`
- **Problem & Solution:**
 - Fixes default timer mode to one-shot (`once_initializer`).
 - Ensures correct polymorphic destruction of `mutex_recursive` and `semaphore_binary`.
 - Casts milliseconds to `uint64_t` prior to `* 1000u` in `osDelay`, `osWait`, `osTimerStart`, `osSignalWait`, `osMutexWait`, `osSemaphoreWait`, `osMessagePut`, `osMessageGet`, `osMailAlloc`, `osMailGet`.
- **Exact Unified Diff Applied:**

```
diff --git a/src/rtos/os-c-wrapper.cpp b/src/rtos/os-c-wrapper.cpp
index fc898c72..4bd9e1bb 100644
--- a/src/rtos/os-c-wrapper.cpp
+++ b/src/rtos/os-c-wrapper.cpp
@@ -1504,7 +1504,9 @@ os_timer_construct (os_timer_t* timer, const
    char* name,
    assert (timer != nullptr);
    if (attr == nullptr)
    {
-       attr = (const os_timer_attr_t*)&timer::periodic_initializer;
+       // The C++ and CMSIS defaults are one-shot timers; only an
+       // explicit
+       // periodic attribute (os_timer_attr_get_periodic()) should
+       // repeat.
+       attr = (const os_timer_attr_t*)&timer::once_initializer;
    }
    new (timer)
        rtos::timer (name, (timer::func_t)function,
                      (timer::func_args_t)args,
@@ -1545,7 +1547,9 @@ os_timer_new (const char* name, os_timer_func_t
    function,
    {
        if (attr == nullptr)
        {
-           attr = (const os_timer_attr_t*)&timer::periodic_initializer;
+           // The C++ and CMSIS defaults are one-shot timers; only an
+           // explicit
+           // periodic attribute (os_timer_attr_get_periodic()) should
+           // repeat.
+           attr = (const os_timer_attr_t*)&timer::once_initializer;
        }
        return reinterpret_cast<os_timer_t*> (
            new rtos::timer (name, (timer::func_t)function,
                            (timer::func_args_t)args,
@@ -1773,7 +1777,21 @@ void
os_mutex_delete (os_mutex_t* mutex)
{
    assert (mutex != nullptr);
-    delete reinterpret_cast<rtos::mutex*> (mutex);
+
+    // `mutex` and `mutex_recursive` share the same C storage type and
+    // the base
+    // destructor is intentionally non-virtual (so that `os_mutex_t`
+    // keeps the
+    // same size as `rtos::mutex`). The concrete type is recorded in
+    // the object
+    // itself, so delete through the derived type and run the correct
+    // destructor instead of relying on undefined behaviour.
+    if (reinterpret_cast<rtos::mutex*> (mutex)->type ()
+        == rtos::mutex::type::recursive)
+    {
+        delete reinterpret_cast<rtos::mutex_recursive*> (mutex);
+    }
+    else
```

```

+     {
+         delete reinterpret_cast<rtos::mutex*> (mutex);
+     }
+ }

/**
@@ -2325,7 +2343,22 @@ void
os_semaphore_delete (os_semaphore_t* semaphore)
{
    assert (semaphore != nullptr);
-    delete reinterpret_cast<rtos::semaphore*> (semaphore);
+
+ // `semaphore_binary` and `semaphore_counting` share the same C
+ // storage
+ // type and the base destructor is intentionally non-virtual (so
+ // that
+ // `os_semaphore_t` keeps the same size as `rtos::semaphore`). The
+ // binary
+ // form is the one whose maximum count is 1, so delete through the
+ // derived
+ // type and run the correct destructor instead of relying on
+ // undefined
+ // behaviour. Both derived destructors are empty, so the routing is
+ // safe
+ // even if a counting semaphore is configured with a maximum of 1.
+ if (reinterpret_cast<rtos::semaphore*> (semaphore)->max_value () ==
+     1)
+     {
+         delete reinterpret_cast<rtos::semaphore_binary*> (semaphore);
+     }
+ else
+     {
+         delete reinterpret_cast<rtos::semaphore_counting*> (semaphore);
+     }
+ }

/**
@@ -3505,7 +3538,6 @@ osThreadCreate (const osThreadDef_t* thread_def,
void* args)
    thread::attributes attr;
    attr.th_priority = thread_def->tpriority;
    attr.th_stack_size_bytes = thread_def->stacksize;
-
- // Creating thread with invalid priority should fail (validator
- // requirement).
    if (thread_def->tpriority >= osPriorityError)
    {
@@ -3739,7 +3771,7 @@ osDelay (uint32_t millisec)
    }

    result_t res = sysclock.sleep_for (
-        clock_systick::ticks_cast ((uint64_t)(millisec * 1000u));
+        clock_systick::ticks_cast ((uint64_t) millisec * 1000u));

    if (res == ETIMEDOUT)
    {
@@ -3788,7 +3820,7 @@ osWait (uint32_t millisec)
    }

    result_t res = sysclock.wait_for (
-        clock_systick::ticks_cast ((uint64_t)(millisec * 1000u));
+        clock_systick::ticks_cast ((uint64_t) millisec * 1000u));

    // TODO: return events
    if (res == ETIMEDOUT)
@@ -3867,7 +3899,7 @@ osTimerStart (osTimerId timer_id, uint32_t
millisec)

    result_t res
        = (reinterpret_cast<rtos::timer&> (*timer_id))

```

```

-         .start (clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)));
+         .start (clock_systick::ticks_cast (((uint64_t) millisec *
1000u)));

    if (res == result::ok)
    {
@@ -4051,7 +4083,7 @@ osSignalWait (int32_t signals, uint32_t
millisec)
    {
        res = this_thread::flags_timed_wait (
            (flags::mask_t)signals,
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)),
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)),
            (flags::mask_t*)&event.value.signals);
    }

@@ -4162,7 +4194,7 @@ osMutexWait (osMutexId mutex_id, uint32_t
millisec)
    {
        ret = (reinterpret_cast<rtos::mutex&> (*mutex_id))
            .timed_lock (
-            clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec *
1000u)));
        // osErrorTimeoutResource:
    }

@@ -4355,7 +4387,7 @@ osSemaphoreWait (osSemaphoreId semaphore_id,
uint32_t millisec)
    {
        res = (reinterpret_cast<rtos::semaphore&> (*semaphore_id))
            .timed_wait (
-            clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec *
1000u)));
        if (res == ETIMEDOUT)
        {
            return 0;
@@ -4645,7 +4677,7 @@ osMessagePut (osMessageQId queue_id, uint32_t
info, uint32_t millisec)
        res = (reinterpret_cast<message_queue&> (*queue_id))
            .timed_send (
                (const char*)&info, sizeof (uint32_t),
-                clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)),
+                clock_systick::ticks_cast (((uint64_t) millisec *
1000u)),
                0);
        // osOK, osErrorTimeoutResource, osErrorParameter
    }

@@ -4743,7 +4775,7 @@ osMessageGet (osMessageQId queue_id, uint32_t
millisec)
        res = (reinterpret_cast<message_queue&> (*queue_id))
            .timed_receive (
                (char*)&event.value.v, sizeof (uint32_t),
-                clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)),
+                clock_systick::ticks_cast (((uint64_t) millisec *
1000u)),
                nullptr);
        // result::event_message when message;
        // result::event_timeout when timeout;
@@ -4881,7 +4913,7 @@ osMailAlloc (osMailQId mail_id, uint32_t
millisec)
    }
    ret = (reinterpret_cast<memory_pool&> (mail_id->pool))
        .timed_alloc (
-            clock_systick::ticks_cast ((uint64_t)(millisec *
1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec *
1000u)));

```

```

    }
#pragma GCC diagnostic pop
    return ret;
@@ -5044,7 +5076,7 @@ osMailGet (osMailQId mail_id, uint32_t millisec)
    res = (reinterpret_cast<message_queue*> (mail_id->queue))
        .timed_receive (
            (char*)&event.value.p, sizeof (void*),
-           clock_systick::ticks_cast ((uint64_t)(millisec *
+           clock_systick::ticks_cast ((uint64_t) millisec *
            1000u)),
            nullptr);
    // osEventMail for ok, osEventTimeout
}

```

• **GitHub CLI:**

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/cmsis-c-wrapper-
  cleanups \
  --title "fix(rtos): CMSIS-RTOS v1 one-shot timer default,
  polymorphic delete & 64-bit timeouts"

```

PR #10: POSIX I/O File Descriptor Table Critical Section Mutexing

- **Single Theme:** Thread-safe file descriptor allocation and deallocation.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/posix-io/file-descriptors-manager.cpp
- **Problem & Solution:**
 - Protects file descriptor table allocation, validation, and deallocation under `rtos::interrupts::critical_section` ics; locks.
 - Adds null-pointer checks for `descriptors_array__` preventing segfaults if queries occur before descriptor table initialization.
- **Exact Unified Diff Applied:**

```
diff --git a/src/posix-io/file-descriptors-manager.cpp b/src/posix-io/
file-descriptors-manager.cpp
index 836539f2..8a65dbcd 100644
--- a/src/posix-io/file-descriptors-manager.cpp
+++ b/src/posix-io/file-descriptors-manager.cpp
@@ -16,6 +16,7 @@
#include <cmsis-plus/posix-io/file-descriptors-manager.h>
#include <cmsis-plus/posix-io/io.h>
#include <cmsis-plus/posix-io/socket.h>
+#include <cmsis-plus/rtos/os.h>

#include <cmsis-plus/diag/trace.h>

@@ -27,6 +28,7 @@

#ifdef __clang__
#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
#endif

// -----
@@ -89,6 +91,8 @@ namespace os
io*
file_descriptors_manager::io (int fildes)
{
+   rtos::interrupts::critical_section ics;
+
// Check if valid descriptor or buffer not yet initialised
if ((fildes < 0) || (static_cast<std::size_t> (fildes) >=
size__))
|| (descriptors_array__ == nullptr))
@@ -106,7 +110,11 @@ namespace os
bool
file_descriptors_manager::valid (int fildes)
{
-   if ((fildes < 0) || (static_cast<std::size_t> (fildes) >=
size__))
+   rtos::interrupts::critical_section ics;
+
+   if ((fildes < 0) || (static_cast<std::size_t> (fildes) >=
size__)
+       || (descriptors_array__ == nullptr)
+       || (descriptors_array__[fildes] == nullptr))
+   {
return false;
}
@@ -127,6 +135,8 @@ namespace os
return -1;
}

+   rtos::interrupts::critical_section ics;
```

```

+
+         for (std::size_t i = reserved__; i < size__; ++i)
+         {
+             #pragma GCC diagnostic push
@@ -167,6 +177,8 @@ namespace os
+                 return -1;
+         }

+         rtos::interrupts::critical_section ics;
+
+         #pragma GCC diagnostic push
+         #if defined(__clang__)
+         #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
@@ -184,7 +196,11 @@ namespace os
+             trace::printf ("file_descriptors_manager::%s(%d)\n", __func__,
+                             fildes);
+         #endif

-         if ((fildes < 0) || (static_cast<std::size_t> (fildes) >=
+             size__))
+         rtos::interrupts::critical_section ics;
+
+         if ((fildes < 0) || (static_cast<std::size_t> (fildes) >=
+             size__)
+             || (descriptors_array__ == nullptr)
+             || (descriptors_array__[fildes] == nullptr))
+         {
+             errno = EBADF;
+             return -1;
@@ -204,13 +220,19 @@ namespace os
+             file_descriptors_manager::socket (int fildes)
+             {
+                 assert ((fildes >= 0) && (static_cast<std::size_t> (fildes) <
+                     size__));
+
+         rtos::interrupts::critical_section ics;
+
+         #pragma GCC diagnostic push
+         #if defined(__clang__)
+         #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+         #endif
-         auto* const io = descriptors_array__[fildes];
+         auto* const io = (descriptors_array__ != nullptr)
+             ? descriptors_array__[fildes]
+             : nullptr;
+         #pragma GCC diagnostic pop
-         if (io->get_type () != static_cast<posix::io::type_t>
+             (io::type::socket))
+         if (io == nullptr
+             || io->get_type () != static_cast<posix::io::type_t>
+             (io::type::socket))
+         {
+             return nullptr;
+         }
+     }
@@ -220,6 +242,8 @@ namespace os
+         size_t
+         file_descriptors_manager::used (void)
+         {
+             rtos::interrupts::critical_section ics;
+
+             std::size_t count = reserved__;
+             for (std::size_t i = reserved__; i <
+                 file_descriptors_manager::size ();
+                 ++i)
@@ -228,7 +252,7 @@ namespace os
+             #if defined(__clang__)
+             #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+             #endif
-             if (descriptors_array__[i] != nullptr)

```

```
+         if (descriptors_array__ != nullptr &&
            descriptors_array__[i] != nullptr)
        {
            ++count;
        }
```

• **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/posix-io-fd-mutexing \
  --title
  "fix(posix-io): protect file descriptor table under critical
  section locks"
```

PR #11: POSIX I/O Deferred List Critical Section Protection

- **Single Theme:** Thread-safe deferred files, directories, and sockets reuse lists.
- **Repository:** micro-os-plus-iii
- **Files Touched:** include/cmsis-plus/posix-io/file-system.h, include/cmsis-plus/posix-io/net-stack.h
- **Problem & Solution:**
 - Protects deferred_files_list_, deferred_directories_list_, and deferred_sockets_list_ recycling operations with rtos::interrupts::critical_section ics;.
 - Eliminates list corruption during concurrent multi-threaded opens and closes.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/posix-io/file-system.h b/include/
cmsis-plus/posix-io/file-system.h
index 28fb822c..4986ca49 100644
--- a/include/cmsis-plus/posix-io/file-system.h
+++ b/include/cmsis-plus/posix-io/file-system.h
@@ -28,6 +28,7 @@
#include <cmsis-plus/utils/lists.h>

#include <cmsis-plus/diag/trace.h>
+#include <cmsis-plus/rtos/os.h>

#include <mutex>
#include <cstdlibarg>
@@ -821,12 +822,14 @@ namespace os
inline void
file_system::add_deferred_file (file* fil)
{
+   rtos::interrupts::critical_section ics;
   deferred_files_list_.link (*fil);
}

inline void
file_system::add_deferred_directory (directory* dir)
{
+   rtos::interrupts::critical_section ics;
   deferred_directories_list_.link (*dir);
}

@@ -848,16 +851,21 @@ namespace os
{
using file_type = T;

-   file_type* fil;
+   file_type* fil = nullptr;
+   {
+       rtos::interrupts::critical_section ics;
+       if (!deferred_files_list_.empty ())
+       {
+           fil = static_cast<file_type*>
+               (deferred_files_list_.unlink_head ());
+       }
+   }

-   if (deferred_files_list_.empty ())
+   if (fil == nullptr)
+   {
+       fil = new file_type (*this);
+   }
+   else
```

```

        {
-         fil = static_cast<file_type*>
            (deferred_files_list_.unlink_head ());
-
            // Call the constructor before reusing the object,
            fil->~file_type ();

@@ -875,16 +883,21 @@ namespace os
        {
            using file_type = T;

-         file_type* fil;
+         file_type* fil = nullptr;
+         {
+             rtos::interrupts::critical_section ics;
+             if (!deferred_files_list_.empty ())
+             {
+                 fil = static_cast<file_type*>
                    (deferred_files_list_.unlink_head ());
+             }
+         }

-         if (deferred_files_list_.empty ())
+         if (fil == nullptr)
            {
                fil = new file_type (*this, locker);
            }
        else
        {
-         fil = static_cast<file_type*>
            (deferred_files_list_.unlink_head ());
-
            // Call the constructor before reusing the object,
            fil->~file_type ();

@@ -902,13 +915,20 @@ namespace os
        {
            using file_type = T;

-         // Deallocate all remaining elements in the list.
-         while (!deferred_files_list_.empty ())
+         for (;;)
            {
-         file_type* f
-             = static_cast<file_type*>
                (deferred_files_list_.unlink_head ());

-         // Call the destructor and the deallocator.
+         file_type* f = nullptr;
+         {
+             rtos::interrupts::critical_section ics;
+             if (!deferred_files_list_.empty ())
+             {
+                 f = static_cast<file_type*>
                    (deferred_files_list_.unlink_head ());
+             }
+         }
+         if (f == nullptr)
+         {
+             break;
+         }
+         delete f;
            }
        }

@@ -919,17 +939,22 @@ namespace os
        {
            using directory_type = T;

```

```

-     directory_type* dir;
-
-     if (deferred_directories_list_.empty ())
+     directory_type* dir = nullptr;
+     {
+         rtos::interrupts::critical_section ics;
+         if (!deferred_directories_list_.empty ())
+         {
+             dir = static_cast<directory_type*> (
+                 deferred_directories_list_.unlink_head ());
+         }
+     }
+
+     if (dir == nullptr)
+     {
+         dir = new directory_type (*this);
+     }
+     else
+     {
-         dir = static_cast<directory_type*> (
-             deferred_directories_list_.unlink_head ());
-
-         // Call the constructor before reusing the object,
-         dir->directory_type ();

```

@@ -947,17 +972,22 @@ namespace os

```

{
    using directory_type = T;

-     directory_type* dir;
-
-     if (deferred_directories_list_.empty ())
+     directory_type* dir = nullptr;
+     {
+         rtos::interrupts::critical_section ics;
+         if (!deferred_directories_list_.empty ())
+         {
+             dir = static_cast<directory_type*> (
+                 deferred_directories_list_.unlink_head ());
+         }
+     }
+
+     if (dir == nullptr)
+     {
+         dir = new directory_type (*this, locker);
+     }
+     else
+     {
-         dir = static_cast<directory_type*> (
-             deferred_directories_list_.unlink_head ());
-
-         // Call the constructor before reusing the object,
-         dir->directory_type ();

```

@@ -975,13 +1005,21 @@ namespace os

```

{
    using directory_type = T;

-     // Deallocate all remaining elements in the list.
-     while (!deferred_directories_list_.empty ())
+     for (;;)
+     {
-         directory_type* d = static_cast<directory_type*> (
-             deferred_directories_list_.unlink_head ());
-
-         // Call the destructor and the deallocator.

```

```

+         directory_type* d = nullptr;
+         {
+             rtos::interrupts::critical_section ics;
+             if (!deferred_directories_list_.empty ())
+             {
+                 d = static_cast<directory_type*> (
+                     deferred_directories_list_.unlink_head ());
+             }
+         }
+         if (d == nullptr)
+         {
+             break;
+         }
+         delete d;
+     }
+ }
diff --git a/include/cmsis-plus/posix-io/net-stack.h b/include/cmsis-plus/posix-io/net-stack.h
index ebced3da..c2d4f3af 100644
--- a/include/cmsis-plus/posix-io/net-stack.h
+++ b/include/cmsis-plus/posix-io/net-stack.h
@@ -26,6 +26,7 @@
#include <cmsis-plus/utils/lists.h>

#include <cmsis-plus/diag/trace.h>
+#include <cmsis-plus/rtos/os.h>

#include <cstdint>
#include <cassert>
@@ -468,6 +469,7 @@ namespace os
    inline void
    net_stack::add_deferred_socket (class socket* sock)
    {
+        rtos::interrupts::critical_section ics;
        deferred_sockets_list_.link (*sock);
    }

@@ -483,17 +485,22 @@ namespace os
    {
        using socket_type = T;

-        socket_type* sock;
-
-        if (deferred_sockets_list_.empty ())
+        socket_type* sock = nullptr;
+        {
+            rtos::interrupts::critical_section ics;
+            if (!deferred_sockets_list_.empty ())
+            {
+                sock = static_cast<socket_type*> (
+                    deferred_sockets_list_.unlink_head ());
+            }
+        }

+        if (sock == nullptr)
        {
            sock = new socket_type (*this);
        }
        else
        {
-            sock = static_cast<socket_type*> (
-                deferred_sockets_list_.unlink_head ());
-
            // Call the constructor before reusing the object,
            sock->~socket_type ();

```

```

@@ -501,12 +508,21 @@ namespace os
    new (sock) socket_type (*this);

    // Deallocate all remaining elements in the list.
    while (!deferred_sockets_list_.empty ())
+   for (;;)
    {
        socket_type* s = static_cast<socket_type*> (
            deferred_sockets_list_.unlink_head ());

        // Call the destructor and the deallocator.
        socket_type* s = nullptr;
        {
            rtos::interrupts::critical_section ics;
            if (!deferred_sockets_list_.empty ())
            {
                s = static_cast<socket_type*> (
                    deferred_sockets_list_.unlink_head ());
            }
            if (s == nullptr)
            {
                break;
            }
            delete s;
        }
    }
}
@@ -519,17 +535,22 @@ namespace os
{
    using socket_type = T;

    socket_type* sock;

    if (deferred_sockets_list_.empty ())
    socket_type* sock = nullptr;
    {
        rtos::interrupts::critical_section ics;
        if (!deferred_sockets_list_.empty ())
        {
            sock = static_cast<socket_type*> (
                deferred_sockets_list_.unlink_head ());
        }
    }

    if (sock == nullptr)
    {
        sock = new socket_type (*this, locker);
    }
    else
    {
        sock = static_cast<socket_type*> (
            deferred_sockets_list_.unlink_head ());

        // Call the constructor before reusing the object,
        sock->~socket_type ();
    }
}
@@ -537,12 +558,21 @@ namespace os
    new (sock) socket_type (*this, locker);

    // Deallocate all remaining elements in the list.
    while (!deferred_sockets_list_.empty ())
+   for (;;)
    {
        socket_type* s = static_cast<socket_type*> (
            deferred_sockets_list_.unlink_head ());
    }

```

```

-         // Call the destructor and the deallocator.
+         socket_type* s = nullptr;
+         {
+             rtos::interrupts::critical_section ics;
+             if (!deferred_sockets_list_.empty ())
+             {
+                 s = static_cast<socket_type*> (
+                     deferred_sockets_list_.unlink_head ());
+             }
+         }
+         if (s == nullptr)
+         {
+             break;
+         }
+         delete s;
+     }
}

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/posix-io-deferred-
  lists-locking \
  --title "fix(posix-io): critical section protection for deferred
  files, directories, and sockets lists"

```

PR #12: Block Device Sector Size Validation Check

- **Single Theme:** Block device sector size query validation in `vfcntl()`.
- **Repository:** `micro-os-plus-iii`
- **File Touched:** `src/posix-io/block-device.cpp`
- **Problem & Solution:**
 - Corrects inverted checks in `vfcntl()` where queries for `block_logical_size_bytes_`, `block_physical_size_bytes_`, and `num_blocks_tested != 0` instead of `== 0`.
 - Inverting the condition allows valid non-zero block sizes to be reported and returns `EINVAL` only when the size is zero or the output pointer is null.
- **Exact Unified Diff Applied:**

```
diff --git a/src/posix-io/block-device.cpp b/src/posix-io/block-
device.cpp
index 020245ee..2565cf4d 100644
--- a/src/posix-io/block-device.cpp
+++ b/src/posix-io/block-device.cpp
@@ -153,7 +153,7 @@ namespace os
    // Get logical device sector size (to be used for read/
    writes).
    {
        std::size_t* sz = va_arg (args, std::size_t*);
-       if (sz == nullptr || impl ().block_logical_size_bytes_ !=
0)
+       if (sz == nullptr || impl ().block_logical_size_bytes_ ==
0)
        {
            errno = EINVAL;

@@ -171,7 +171,7 @@ namespace os
    // Get physical device sector size (internally used for
    erase).
    {
        std::size_t* sz = va_arg (args, std::size_t*);
-       if (sz == nullptr || impl ().block_physical_size_bytes_ !=
= 0)
+       if (sz == nullptr || impl ().block_physical_size_bytes_
== 0)
        {
            errno = EINVAL;

@@ -189,7 +189,7 @@ namespace os
    // Get device size in bytes.
    {
        uint64_t* sz = va_arg (args, uint64_t*);
-       if (sz == nullptr || impl ().num_blocks_ != 0)
+       if (sz == nullptr || impl ().num_blocks_ == 0)
        {
            errno = EINVAL;
```

• GitHub CLI:

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
--base xpack-development --head dan-maio:fix/block-device-size-
check \
--title "fix(posix-io): correct inverted block size validation in
block device vfcntl"
```

PR #13: ARMv8-M Architecture Semihosting Trap Instruction Guards

- **Single Theme:** ARMv8-M mainline/baseline semihosting trap instructions.
- **Repository:** micro-os-plus-iii
- **File Touched:** include/cmsis-plus/arm/semihosting.h
- **Problem & Solution:**
 - Adds `__ARM_ARCH_8M_MAIN__` and `__ARM_ARCH_8M_BASE__` macro guards for semihosting trap instructions (bkpt).
 - Adds `#elif defined(SEMIHOST_TRAP_HLT)` support for ARMv8 HLT semihosting traps.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/arm/semihosting.h b/include/cmsis-plus/arm/semihosting.h
index de63417e..37c85d61 100644
--- a/include/cmsis-plus/arm/semihosting.h
+++ b/include/cmsis-plus/arm/semihosting.h
@@ -73,7 +73,8 @@ extern "C"
 #endif
 // For thumb only architectures use the BKPT instruction instead of
 // SWI.
 #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
- || defined(__ARM_ARCH_6M__)
+ || defined(__ARM_ARCH_6M__) || defined(__ARM_ARCH_8M_MAIN__) \
+ || defined(__ARM_ARCH_8M_BASE__)
 #define AngelSWIInsn "bkpt"
 #define AngelSWIAsm bkpt
 #else
@@ -103,6 +104,9 @@ extern "C"
     " mov r1, %[arg] \n"
 #if defined(OS_DEBUG_SEMIHOSTING_FAULTS)
     " " AngelSWITestFault " \n"
+ #elif defined(SEMIHOST_TRAP_HLT)
+     " .arm \n"
+     " .inst 0xE10F0070 \n"
 #else
     " " AngelSWIInsn " %[swi] \n"
 #endif
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/armv8m-semihosting-
  guards \
  --title
    "fix(arm): ARMv8-M semihosting trap instruction architecture
    guards"
```

PR #14: ARMv8-M Mainline Architecture Exception Handlers Support

- **Single Theme:** ARMv8-M Mainline (Cortex-M33) exception handling and VTOR setup.
- **Repository:** micro-os-plus-iii
- **Files Touched:** src/startup/exception-handlers.c, include/cmsis-plus/cortexm/exception-handlers.h
- **Problem & Solution:**
 - Extends exception handling routines (VTOR relocation register setup at 0xE000ED08, register dump routines, BusFault, UsageFault, SVC, DebugMon, and PendSV handlers) to include defined(__ARM_ARCH_8M_MAIN__).
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/cortexm/exception-handlers.h b/
include/cmsis-plus/cortexm/exception-handlers.h
index 66da4b04..96f40aee 100644
--- a/include/cmsis-plus/cortexm/exception-handlers.h
+++ b/include/cmsis-plus/cortexm/exception-handlers.h
@@ -34,7 +34,8 @@ extern "C"
     extern void
         HardFault_Handler (void);

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

     extern void
         MemManage_Handler (void);
@@ -48,6 +49,13 @@ extern "C"
     extern void
         DebugMon_Handler (void);

+
+#endif
+
+#if defined(__ARM_ARCH_8M_MAIN__)
+
+    extern void
+        SecureFault_Handler (void);
+
+#endif

     extern void
@@ -70,17 +78,18 @@ extern "C"
         uint32_t lr;
         uint32_t pc;
         uint32_t psr;
-#if defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7EM__) || defined(__ARM_ARCH_8M_MAIN__)
         uint32_t s[16];
-#endif
     } exception_stack_frame_t;

     #if defined	TRACE)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
         void
         dump_exception_stack (exception_stack_frame_t* frame, uint32_t
             cfsr,
                                     uint32_t mmfar, uint32_t bfar, uint32_t lr);
-#endif // defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#endif // defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) ||
+    defined(__ARM_ARCH_8M_MAIN__)
```

```

#if defined(__ARM_ARCH_6M__)
    void
    dump_exception_stack (exception_stack_frame_t* frame, uint32_t lr);
@@ -90,7 +99,8 @@ extern "C"
    void
    HardFault_Handler_C (exception_stack_frame_t* frame, uint32_t lr);

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

    void
    UsageFault_Handler_C (exception_stack_frame_t* frame, uint32_t lr);
@@ -98,7 +108,7 @@ extern "C"
    void
    BusFault_Handler_C (exception_stack_frame_t* frame, uint32_t lr);

-#endif // defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#endif // defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) ||
+    defined(__ARM_ARCH_8M_MAIN__)

#if defined(__cplusplus)
}
diff --git a/src/startup/exception-handlers.c b/src/startup/exception-
handlers.c
index 98be7d14..533ffcd7 100644
--- a/src/startup/exception-handlers.c
+++ b/src/startup/exception-handlers.c
@@ -91,7 +91,8 @@ Reset_Handler (void)
    // SCB->VTOR
    // https://developer.arm.com/documentation/dui0552/a/cortex-m3-
    peripherals/system-control-block/vector-table-offset-register
    // Mandatory when running from RAM. Not available on Cortex-M0.
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
    *((uint32_t*)0xE00ED08)
        = ((uint32_t)interrupt_vectors & (uint32_t)(~0x3F));
#endif
@@ -119,14 +120,16 @@ void __attribute__ ((section (".after_vectors"),
weak))
NMI_Handler (void)
{
    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)
@@ -139,7 +142,8 @@ NMI_Handler (void)

#if defined	TRACE)

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

// The values of BFAR and MMFAR remain unchanged if the BFARVALID or

```

```

// MMARVALID is set. However, if a new fault occurs during the
@@ -183,7 +187,8 @@ dump_exception_stack (exception_stack_frame_t*
    frame, uint32_t cfsr,
    trace_printf (" LR/EXC_RETURN = %08X\n", lr);
}

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */

#if defined(__ARM_ARCH_6M__)

@@ -209,7 +214,8 @@ dump_exception_stack (exception_stack_frame_t*
    frame, uint32_t lr)

// -----

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

#if defined(OS_USE_SEMIHOSTING_SYSCALLS) \
    || defined(OS_USE_TRACE_SEMIHOSTING_STDOUT) \
@@ -470,14 +476,16 @@ HardFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),
#endif /* defined(TRACE) */

#if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
    if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
    {
        __BKPT (0);
    }
#else
    __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

while (true)
@@ -486,7 +494,8 @@ HardFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),
}
}

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */

#if defined(__ARM_ARCH_6M__)

@@ -532,14 +541,16 @@ HardFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),
#endif /* defined(TRACE) */

#if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
    if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
    {
        __BKPT (0);
    }
#else
    __BKPT (0);

```

```

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

while (true)
@@ -550,20 +561,23 @@ HardFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),

#endif /* defined(__ARM_ARCH_6M__) */

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)

void __attribute__((section(".after_vectors"), weak))
MemManage_Handler (void)
{
    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

while (true)
@@ -603,14 +617,16 @@ BusFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),
#endif /* defined	TRACE) */

#if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

while (true)
@@ -665,14 +681,16 @@ UsageFault_Handler_C (exception_stack_frame_t*
    frame __attribute__((unused)),
#endif /* defined	TRACE) */

#if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else

```

```

    __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

    while (true)
@@ -687,14 +705,16 @@ void __attribute__ ((section (".after_vectors"),
    weak))
    SVC_Handler (void)
    {
        #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
            if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
            {
                __BKPT (0);
            }
        #else
            __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
        #endif /* defined(DEBUG) */

        while (true)
@@ -703,7 +723,8 @@ SVC_Handler (void)
    }
}

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

void __attribute__ ((section (".after_vectors"), weak))
DebugMon_Handler (void)
@@ -721,20 +742,23 @@ DebugMon_Handler (void)
    }
}

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */

void __attribute__ ((section (".after_vectors"), weak))
PendSV_Handler (void)
{
    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
            if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
            {
                __BKPT (0);
            }
        #else
            __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
        #endif /* defined(DEBUG) */

```

```
while (true)
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \  
--base xpack-development --head dan-maio:feat/armv8m-main-exception-  
handlers \  
--title "feat(startup): ARMv8-M Mainline architecture exception  
handlers and VTOR support"
```

PR #15: Semihosting Syscall fstat() Fallback Mode & Board Console Mirror

- **Single Theme:** Semihosting filesystem attribute fallback handling and physical UART console mirror hook.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/semihosting/c-syscalls-semihosting.cpp
- **Problem & Solution:**
 - Preserves existing file mode in __semihosting_stat(), setting S_IFCHR only if (st->st_mode & S_IFMT) == 0.
 - Introduces weak os_board_console_mirror() hook in __posix_write() to allow board UART drivers to duplicate console output.
- **Exact Unified Diff Applied:**

```
diff --git a/src/semihosting/c-syscalls-semihosting.cpp b/src/semihosting/c-syscalls-semihosting.cpp
index e3ff04c4..8b6d034c 100644
--- a/src/semihosting/c-syscalls-semihosting.cpp
+++ b/src/semihosting/c-syscalls-semihosting.cpp
@@ -263,9 +263,12 @@ __semihosting_stat (int fd, struct stat* st)
     return -1;
 }

- /* Always assume a character device,
-  with 1024 byte blocks. */
- st->st_mode |= S_IFCHR;
+ /* If the caller did not already specify a file type,
+  default to character device, with 1024 byte blocks. */
+ if ((st->st_mode & S_IFMT) == 0)
+ {
+     st->st_mode |= S_IFCHR;
+ }
+ st->st_blksize = 1024;

int res;
@@ -437,9 +440,32 @@ __posix_read (int fildes, void* buf, size_t
    nbyte)
    return nbyte - res;
}

+/**
+ * Optional board console mirror.
+ *
+ * A board whose console is a physical UART (not the debugger)
+ * defines this to
+ * copy the console stream there, so the application's stdout/stderr
+ * is visible
+ * on the terminal as well as in the debugger's semihosting console.
+ * The
+ * semihosting write stays the primary path; the board only adds the
+ * UART copy.
+ * Weak: boards that do not need it link this no-op and it is never
+ * called.
+ */
+extern "C" void
+os_board_console_mirror (int fildes, const void* buf, size_t nbyte)
+ __attribute__ ((weak));
+
+ssize_t
+__posix_write (int fildes, const void* buf, size_t nbyte)
+{
+ // The board console, if any, gets stdout/stderr whatever happens
+ // to the
+ // semihosting side below: on some ports the monitor handles are
+ // not set up,
+ // and a dropped printf() would otherwise never be seen. Done
+ // before the fd
```

```
+ // lookup so it does not depend on it.
+ if ((fildes == 1 || fildes == 2)
+     && (os_board_console_mirror != nullptr))
+ {
+     os_board_console_mirror (fildes, buf, nbyte);
+ }
+
+ struct fdent* pfd;
+ pfd = __semihosting_findslot (fildes);
+ if (pfd == NULL)
```

• **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/semihosting-fstat-
  mirror \
  --title "fix(semihosting): preserve existing fstat st_mode and add
  board console mirror hook"
```

PR #16: Software Timer Callback Execution Decoupling & Drift Prevention

- **Single Theme:** Deadlock-free software timer callback execution and periodic drift elimination.
- **Repository:** micro-os-plus-iii
- **Files Touched:** src/rtos/internal/os-lists.cpp, src/rtos/os-timer.cpp, include/cmsis-plus/rtos/os-thread.h
- **Problem & Solution:**
 - Invokes timer callbacks **outside the critical section** in os-lists.cpp by unlinking the node inside the critical section and invoking node->action() after exit, preventing deadlocks when callbacks acquire mutexes.
 - In os-timer.cpp, guards against cumulative drift under heavy load by checking if (timer_node_.timestamp <= now) and advancing from now + period_ under a critical section.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index 9cd7a05a..3aecb6a9 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -2081,6 +2081,14 @@ namespace os
     inline int* __attribute__((always_inline))
     __errno (void)
     {
+        // A timer callback runs inside the tick ISR and a libc call
+        // (e.g. printf) still touches errno; thread() asserts in
+        // handler mode,
+        // so hand back a scratch int instead of the current
+        // thread's.
+        if (interrupts::in_handler_mode ())
+        {
+            static int isr_errno;
+            return &isr_errno;
+        }
+        return &this_thread::thread ().errno_;
     }

diff --git a/src/rtos/internal/os-lists.cpp b/src/rtos/internal/os-lists.cpp
index 02ea8ab4..32c0c9a0 100644
--- a/src/rtos/internal/os-lists.cpp
+++ b/src/rtos/internal/os-lists.cpp
@@ -499,33 +499,42 @@ namespace os
     // iterate until a node with future time stamp is identified.
     for (;;)
     {
-        // ----- Enter critical section
-        -----
-        interrupts::critical_section ics;
+        timestamp_node* node = nullptr;
+        {
+            // ----- Enter critical section
+            -----
+            interrupts::critical_section ics;

-            if (empty ())
+            if (empty ())
             {
-                break;
+                break;
             }
+            if (empty ())
             {
-                break;
+                break;
             }
         }
     }
 }
```



```
else  
{
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \  
--base xpack-development --head dan-maio:fix/timer-callback-  
decoupling \  
--title "fix(rtos): execute software timer callbacks outside  
critical section & eliminate drift"
```

PR #17: POSIX Mutex Priority Ceiling & Dynamic Boost Protocol

- **Single Theme:** Real-time priority ceiling protocol and dynamic priority inheritance tracking.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/rtos/os-mutex.cpp
- **Problem & Solution:**
 - Elevates and verifies caller priority against ceiling **before** granting mutex ownership (if (protocol_ == protocol::protect && th->priority () > prio_ceiling_) return EINVAL;), ensuring a rejected lock leaves no trace in the thread's owned mutexes list.
 - Dynamically tracks maximum boost among multiple waiters and safely captures thread* owner = owner_; to prevent null-pointer dereferencing when owner releases lock concurrently on another CPU.
- **Exact Unified Diff Applied:**

```
diff --git a/src/rtos/os-mutex.cpp b/src/rtos/os-mutex.cpp
index e8cab9d1..6ce22248 100644
--- a/src/rtos/os-mutex.cpp
+++ b/src/rtos/os-mutex.cpp
@@ -638,6 +638,15 @@ namespace os
     // First lock.
     if (owner_ == nullptr)
     {
+        // Prio ceiling must be at least the priority of the
+        // highest
+        // priority thread. Checked before anything is taken: a
+        // refused
+        // lock must not leave the mutex on the thread's list of
+        // owned
+        // mutexes, nor counted in acquired_mutexes_.
+        if (protocol_ == protocol::protect && th->priority () >
+            prio_ceiling_)
+        {
+            return EINVAL;
+        }
+
         // If the mutex has no owner, own it.
         owner_ = th;
     }
@@ -661,16 +670,6 @@ namespace os

     if (protocol_ == protocol::protect)
     {
-        if (th->priority () > prio_ceiling_)
-        {
-            // No need to keep the lock.
-            owner_ = nullptr;
-
-            // Prio ceiling must be at least the priority of
-            // the
-            // highest priority thread.
-            return EINVAL;
-        }
-
         // POSIX: When a thread owns one or more mutexes
         // initialised with the mutex::protocol::protect
         protocol,
         // it shall execute at the higher of its priority or
         the
     }
@@ -778,7 +777,11 @@ namespace os
     if (protocol_ == protocol::inherit)
     {
         thread::priority_t prio = th->priority ();
-        boosted_prio_ = prio;
     }
 }
```

```

+         if ((boosted_prio_ == thread::priority::none)
+             || (prio > boosted_prio_))
+         {
+             boosted_prio_ = prio;
+         }

+         if (owner_links_.unlinked ())
+         {
@@ -788,13 +791,54 @@ namespace os
+         }

+         // Boost owner priority.
+         if ((boosted_prio_ > owner_>priority_inherited ()))
+         //
+         // The owner is captured HERE, while the kernel lock is
still
+         // held, and the captured pointer -- not the member --
is the
+         // one dereferenced below.
+         //
+         // The reason is the uncritical section itself: it
releases the
+         // kernel lock (`priority_inherited()` ends in a yield,
which
+         // must not run scheduler-locked), and while it is open
the
+         // owner can finish its own `unlock()` on another CPU.
+         // `internal_unlock_()` ends by setting `owner_` to
nullptr, so
+         // re-reading the member after the section -- which is
exactly
+         // what `owner_>priority_inherited(...)` compiles to
-- then
+         // dereferences a null pointer.
+         //
+         // Every SMP port has this window. It was found on the
POSIX
+         // host port, where `scheduler::unlock()`/'`lock()` is
two
+         // `pthread_sigmask()` system calls rather than a
handful of
+         // instructions: that widens the window enough to turn
a rare
+         // race into the common case (smp_test2 faulted in 21
runs out
+         // of 25 on four CPUs).
+         /* class */ thread* owner = owner_;

+         if ((owner != nullptr)
+             && (boosted_prio_ > owner_>priority_inherited ()))
+         {
-         // ----- Enter uncritical section
+         -----
+         scheduler::uncritical_section sucs;
+         {
+         // ----- Enter uncritical section
+         -----
+         scheduler::uncritical_section sucs;

+         // Still the owner? If it released the mutex
while this
+         // section was open there is nothing to inherit,
and
+         // boosting it anyway would leave behind an
inherited
+         // priority that no later unlock() would ever
clear.

+         if (owner_ == owner)
+         {
+             owner_>priority_inherited (boosted_prio_);
+         }
+         // ----- Exit uncritical section
+         -----
+         }

```

```

-         owner_>priority_inherited (boosted_prio_);
-         // ----- Exit uncritical section
-         -----
+         // Released while the section was open? Then that
unlock()
+         // found no waiter to hand the mutex to, and
blocking now
+         // would wait for a wake-up that already happened.
Decide
+         // again, back under the lock.
+         if (owner_ != owner)
+         {
+             return internal_try_lock_ (th);
+         }
+     }

#ifdef OS_TRACE_RTOS_MUTEX
@@ -862,14 +906,8 @@ namespace os
    mutexes_list* thread_mutexes
        = reinterpret_cast<mutexes_list*> (&owner_-
>mutexes_);

-         if (thread_mutexes->empty ())
-         {
-             // If the owner thread has no more mutexes,
-             // clear the inherited priority,
-             // and the assigned priority will take
precedence.
-             boosted_prio_ = thread::priority::none;
-         }
-         else
+         thread::priority_t inherited_prio =
thread::priority::none;
+         if (!thread_mutexes->empty ())
+         {
+             // If the owner thread acquired other mutexes
too,
+             // compute the maximum boosted priority.
@@ -887,10 +925,14 @@ namespace os
+         }
+     }
+ }

#pragma GCC diagnostic pop
-         boosted_prio_ = max_prio;
+         if (max_prio > 0)
+         {
+             inherited_prio = max_prio;
+         }
+     }

    // Delayed until end of critical section.
-         owner_>priority_inherited (boosted_prio_);
+         owner_>priority_inherited (inherited_prio);
+         boosted_prio_ = thread::priority::none;
+     }

    // Delayed until end of critical section.

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:feat/mutex-priority-
  ceiling-protocol \
  --title "feat(rtos): POSIX priority ceiling protocol & dynamic
  priority boost tracking"

```

PR #18: Thread Lifecycle State Machine (state::destroying) & Idle Reaper

- **Single Theme:** Safe asynchronous thread resource reclamation and handler-mode errno fallback.
- **Repository:** micro-os-plus-iii
- **Files Touched:** include/cmsis-plus/rtos/os-c-decls.h, include/cmsis-plus/rtos/os-thread.h, src/rtos/os-thread.cpp, src/rtos/os-idle.cpp
- **Problem & Solution:**
 - Introduces state::destroying = 7 to eliminate use-after-free races during thread cleanup.
 - In os-idle.cpp, transitions terminated threads through state::destroying before unlinking, verifying no core is executing on the thread before reclamation.
 - In __errno(), returns a static scratch int in handler mode to avoid assertion failures when ISRs or timer callbacks call libc routines.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index e8565a72..260d1e52 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -315,7 +315,12 @@ extern "C"
 /**
  * @brief Used to check reused threads.
  */
- os_thread_state_initialising = 6
+ os_thread_state_initialising = 6,
+
+ /**
+  * @brief In process of being destroyed.
+  */
+ os_thread_state_destroying = 7
};

/**
diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index 3a6cb6a9..cac6777f 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -400,7 +400,11 @@ namespace os
 /**
  * @brief Used to check reused threads.
  */
- initializing = 6 //
+ initializing = 6, //
+
+ /**
+  * @brief In process of being destroyed.
+  */
+ destroying = 7 //
};
/* enum */
}; /* struct state */
@@ -741,6 +745,7 @@ namespace os
 friend port::stack::element_t*
 port::scheduler::switch_stacks (port::stack::element_t* sp);

+
#endif
/**
 * @endcond
@@ -875,6 +880,7 @@ namespace os
```

```

        bool th_enable_assert_reuse = false;

+
        // Add more attributes here.

        /**
@@ -1192,6 +1198,7 @@ namespace os
        priority_t
        priority_inherited (void);

+
        #if 0
            // ???
            result_t
@@ -1689,6 +1696,7 @@ namespace os
        priority_t volatile prio_assigned_ = priority::none;
        priority_t volatile prio_inherited_ = priority::none;

+
        bool volatile interrupted_ = false;

        internal::event_flags event_flags_;
diff --git a/src/rtos/os-idle.cpp b/src/rtos/os-idle.cpp
index 23b05974..0db1ab35 100644
--- a/src/rtos/os-idle.cpp
+++ b/src/rtos/os-idle.cpp
@@ -79,6 +79,14 @@ os_rtos_idle_actions (void)
        interrupts::critical_section ics;
        node = const_cast<internal::waiting_thread_node*> (
            scheduler::terminated_threads_list.head ());
+
+        thread* th = node->thread_;
+        if (th->state_ == thread::state::destroying
+            || th->state_ == thread::state::destroyed)
+        {
+            node->unlink ();
+            continue;
+        }
+        th->state_ = thread::state::destroying;
        node->unlink ();
        // ----- Exit critical section
        -----
    }

diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
index 3b8f293a..79b0a71c 100644
--- a/src/rtos/os-thread.cpp
+++ b/src/rtos/os-thread.cpp
@@ -19,6 +19,7 @@
        #include <memory>
        #include <stdexcept>

+
+        // -----
+
        #if defined(__clang__)
@@ -679,10 +680,15 @@ namespace os
        interrupts::critical_section ics;

        // If the thread is not already in the ready list, enqueue
        it.
-        if (ready_node_.next () == nullptr)
+        // In SMP, a running thread has ready_node_.next() ==
+        nullptr;
+        // it must not be re-enqueued while actively executing.
+        if (state_ == state::suspended || state_ ==
+            state::initializing)
        {
-            scheduler::ready_threads_list.link (ready_node_);

```

```

-         // state::ready set in above link().
+         if (ready_node_.next () == nullptr)
+         {
+             scheduler::ready_threads_list_.link (ready_node_);
+             // state::ready set in above link().
+         }
+     }
+     // ----- Exit critical section
+     -----
+ }

@@ -691,6 +697,7 @@ namespace os

    port::scheduler::reschedule ();

+
+ #endif

    instrumentation::thread::resume_return (this);
@@ -702,6 +709,7 @@ namespace os
    *
    * @note Can be invoked from Interrupt Service Routines.
    */
+
+ thread::priority_t
+ thread::priority (void)
+ {
@@ -791,17 +799,24 @@ namespace os

+ #else

-     if (state_ == state::ready)
-     {
-         // ----- Enter critical section
-         -----
-         interrupts::critical_section ics;
+     {
+         // ----- Enter critical section
+         -----
+         interrupts::critical_section ics;

-         // Remove from initial location and reinsert according
-         // to new priority.
-         ready_node_.unlink ();
-         scheduler::ready_threads_list_.link (ready_node_);
-         // ----- Exit critical section
-         -----
-     }
+         // Test and relink under one lock, and only a thread that is
+         // still
+         // linked in the ready list. On SMP another CPU may pick this
+         // thread
+         // between a test made outside the lock and the relink:
+         // link() would
+         // then put a thread being dispatched back in the ready list,
+         // marked
+         // ready, and a third CPU could run the same context at once.
+         if (state_ == state::ready && ready_node_.next () != nullptr)
+         {
+             // Remove from initial location and reinsert according
+             // to new priority.
+             ready_node_.unlink ();
+             scheduler::ready_threads_list_.link (ready_node_);
+         }
+         // ----- Exit critical section
+         -----
+     }

+     // Mandatory, the priority might have been raised, the
+     // task must be scheduled to run.
@@ -875,17 +890,24 @@ namespace os

```

```

#else

-   if (state_ == state::ready)
-   {
-       // ----- Enter critical section
-       -----
-       interrupts::critical_section ics;
+   {
+       // ----- Enter critical section
+       -----
+       interrupts::critical_section ics;

-       // Remove from initial location and reinsert according
-       // to new priority.
-       ready_node_.unlink ();
-       scheduler::ready_threads_list_.link (ready_node_);
-       // ----- Exit critical section
-       -----
-   }
+   // Test and relink under one lock, and only a thread that is
+   // still
+   // linked in the ready list. On SMP another CPU may pick this
+   // thread
+   // between a test made outside the lock and the relink:
+   // link() would
+   // then put a thread being dispatched back in the ready list,
+   // marked
+   // ready, and a third CPU could run the same context at once.
+   if (state_ == state::ready && ready_node_.next () != nullptr)
+   {
+       // Remove from initial location and reinsert according
+       // to new priority.
+       ready_node_.unlink ();
+       scheduler::ready_threads_list_.link (ready_node_);
+   }
+   // ----- Exit critical section
+   -----
+   }

    // Mandatory, the priority might have been raised, the
    // task must be scheduled to run.
@@ -941,7 +963,22 @@ namespace os

#else

-   // TODO: implement
+   {
+       interrupts::critical_section ics;
+
+       if (state_ == state::destroyed)
+       {
+           instrumentation::thread::detach_retval (this, EINVAL);
+           return EINVAL;
+       }
+
+       if (parent_ != nullptr)
+       {
+           child_links_.unlink ();
+           scheduler::top_threads_list_.link (*this);
+           parent_ = nullptr;
+       }
+   }

#endif

@@ -997,13 +1034,37 @@ namespace os
    // Fail if current thread
    assert (this != this_thread::_thread ());

```

```

- while (state_ != state::destroyed)
+ thread* crt_thread = this_thread::_thread ();
+ for (;;)
+ {
-     joiner_ = this_thread::_thread ();
-     this_thread::_thread ()->internal_suspend_ (
-         OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN);
+     {
+         // ----- Enter critical section
+         -----
+         interrupts::critical_section ics;
+
+         // Test, register and suspend under one lock, the same
+         lock
+         // internal_destroy() sets `destroyed` and reads
+         `joiner_`
+         // under. Done in steps, a destroy on another CPU could
+         fall
+         // between them and its wake-up be lost.
+         if (state_ == state::destroyed)
+         {
+             break;
+         }
+         joiner_ = crt_thread;
+
+         // Remove this thread from the ready list, if there.
+         port::this_thread::prepare_suspend ();
+
+         crt_thread->state_ = state::suspended;
+         // ----- Exit critical section
+         -----
+     }
+
+     instrumentation::thread::suspended (
+         crt_thread,
+         OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN);
+
+     port::scheduler::reschedule ();
+ }
+
+ #if defined(OS_TRACE_RTOS_THREAD)
+     trace::printf ("%s() @%p %s joined\n", __func__, this, name
+ ());
+ #endif
@@ -1268,12 +1329,27 @@ namespace os
+     // ----- Exit critical section
+     -----
+ }

- state_ = state::destroyed;
+ {
+     // ----- Enter critical section
+     -----
+     interrupts::critical_section ics;
+
+     // From `destroyed` on, a joiner may return from join() and
+     free
+     // this object, so `joiner_` is read under the same lock that
+     // sets it, and the joiner is made ready here, where it
+     cannot
+     // yet have gone -- resume() would do it after the lock.
+     state_ = state::destroyed;

- if (joiner_ != nullptr)
- {
-     joiner_->resume ();
- }
+ thread* joiner = joiner_;
+ if (joiner != nullptr && joiner->state_ == state::suspended

```

```

+         && joiner->ready_node_.next () == nullptr)
+     {
+         scheduler::ready_threads_list_.link (joiner-
+>ready_node_);
+         // state::ready set in above link().
+     }
+     // ----- Exit critical section
+     -----
+ }
+
+ port::scheduler::reschedule ();
+ }
+ #pragma GCC diagnostic pop

@@ -1308,6 +1384,7 @@ namespace os
+     // ----- Enter critical section
+     -----
+     scheduler::critical_section scs;

+
+     if (state_ == state::destroyed)
+     {
+ #if defined(OS_TRACE_RTOS_THREAD)
+@@ -1318,29 +1395,49 @@ namespace os
+         return result::ok; // Already exited itself
+     }

+     bool we_claimed = false;
+     {
+         // ----- Enter critical section
+         -----
+         interrupts::critical_section ics;

+         // Remove thread from the funeral list and kill it here.
+         ready_node_.unlink ();

+         // If the thread is waiting on an event, remove it from the
+         list.
+         if (waiting_node_ != nullptr)
+         if (state_ != state::destroying && state_ !=
+ state::destroyed)
+         {
+             waiting_node_->unlink ();
+         }
+         we_claimed = true;
+         state_ = state::destroying;

+         // If the thread is waiting on a timeout, remove it from
+         the list.
+         if (clock_node_ != nullptr)
+         {
+             clock_node_->unlink ();
+         }
+         // Remove thread from the funeral list and kill it
+         here.
+         ready_node_.unlink ();

+         child_links_.unlink ();
+         // If the thread is waiting on an event, remove it from
+         the list.
+         if (waiting_node_ != nullptr)
+         {
+             waiting_node_->unlink ();
+         }

+         // If the thread is waiting on a timeout, remove it
+         from the list.
+         if (clock_node_ != nullptr)
+         {
+             clock_node_->unlink ();

```

```

+         }
+
+         child_links_.unlink ();
+     }
+     // ----- Exit critical section
+     -----
+ }
+
+ if (!we_claimed)
+ {
+     // The idle reaper already claimed this thread for
+     // destruction.
+     // Wait with lock released until it is completely
+     // destroyed.
+     while (state_ != state::destroyed)
+     {
+         scheduler::uncritical_section sucs;
+         this_thread::yield ();
+     }
+     instrumentation::thread::kill_retval (this, result::ok);
+     return result::ok;
+ }
+
+ // The must be no more children threads alive.
+ assert (children_.empty ());
+ parent_ = nullptr;
+ @@ -1837,6 +1934,11 @@ namespace os
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+ // NOTE: NOT 'inline'. It is declared non-inline in os-
+ // thread.h, and its
+ // body lives only here (not in a header), so other TUs (e.g.
+ // the C API
+ // wrapper os_this_thread_suspend) that odr-use it need an out-
+ // of-line
+ // definition. Marking it 'inline' meant no TU emitted one ->
+ // undefined
+ // reference unless the caller happened to be pruned by --gc-
+ // sections.
+ void
+ suspend (void)
+ {

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:feat/thread-lifecycle-
  destroying-reaper \
  --title "feat(rtos): thread state::destroying (7), idle reaper &
  handler-mode errno fallback"

```

PR #19: Condition Variable Lost-Signal Elimination & Clock Binding

- **Single Theme:** Lost-signal race elimination in condition variables and clock attribute binding.
- **Repository:** micro-os-plus-iii
- **Files Touched:** include/cmsis-plus/rtos/os-condvar.h, src/rtos/os-condvar.cpp, include/cmsis-plus/diag/instrumentation.h, include/cmsis-plus/rtos/os-c-decls.h, include/cmsis-plus/estd/condition_variable
- **Problem & Solution:**
 - Encloses list node linking and user mutex unlocking inside a single scheduler::critical_section atomic step in wait() and timed_wait(), eliminating lost-signal races.
 - Binds condition variable instances to configurable attribute clocks (clock_ = attr.clock != nullptr ? attr.clock : &sysclock;).
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/diag/instrumentation.h b/include/
      cmsis-plus/diag/instrumentation.h
index 69e0c035..fd0b5cec 100644
--- a/include/cmsis-plus/diag/instrumentation.h
+++ b/include/cmsis-plus/diag/instrumentation.h
@@ -25,6 +25,7 @@
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_EVENT_FLAGS (9u)
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN (10u)
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_USER (11u)
+#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR (12u)

// -----

diff --git a/include/cmsis-plus/estd/condition_variable b/include/
      cmsis-plus/estd/condition_variable
index e4a59eb8..d8ed311c 100644
--- a/include/cmsis-plus/estd/condition_variable
+++ b/include/cmsis-plus/estd/condition_variable
@@ -284,20 +284,22 @@ namespace os
    return cv_status::timeout;
}

- Native_clock::time_point start_tp = Native_clock::now ();
-
os::rtos::clock::duration_t ticks
    = os::estd::chrono::ceil<std::chrono::duration<
        os::rtos::clock::duration_t, typename
        Native_clock::period>> (
        rel_time)
    .count ();

- ncv_.timed_wait (
+ // Trust the kernel's verdict rather than re-deriving it from
+ // the wall
+ // clock: a signalled wake that happened to be preempted beyond
+ // `rel_time`, or a timeout that rounded down to fewer ticks,
+ // would be
+ // misreported by a pure elapsed-time comparison.
+ // `timed_wait()` returns
+ // `result::ok` on a signal and `ETIMEDOUT` when the timeout
+ // expired.
+ os::rtos::result_t res = ncv_.timed_wait (
+     /*(rtos::mutex &)*/ (*(lock.mutex ()->native_handle ())),
+     ticks);

- return (Native_clock::now () - start_tp) < rel_time
-     ? cv_status::no_timeout
-     : cv_status::timeout;
+ return (res == os::rtos::result::ok) ? cv_status::no_timeout
```

```

+                                     : cv_status::timeout;
+
+    }

    template <class Rep_T, class Period_T, class Predicate_T>
diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index 260d1e52..86dc5429 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -1065,7 +1065,7 @@ extern "C"
    const char* name;
    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
        os_internal_threads_waiting_list_t list;
-    // void* clock;
+    void* clock;
    #endif

    /**
diff --git a/include/cmsis-plus/rtos/os-condvar.h b/include/cmsis-plus/rtos/os-condvar.h
index d9b64b22..8faa4e66 100644
--- a/include/cmsis-plus/rtos/os-condvar.h
+++ b/include/cmsis-plus/rtos/os-condvar.h
@@ -278,7 +278,7 @@ namespace os
    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
        internal::waiting_threads_list list_;
-    // clock& clock_;
+    clock* clock_ = nullptr;
    #endif

    /**
diff --git a/src/rtos/os-condvar.cpp b/src/rtos/os-condvar.cpp
index 9c1021a3..b08ff1e7 100644
--- a/src/rtos/os-condvar.cpp
+++ b/src/rtos/os-condvar.cpp
@@ -271,8 +271,8 @@ namespace os
    * Edition](http://pubs.opengroup.org/onlinepubs/9699919799/nframe.html)).
    */
    condition_variable::condition_variable (const char* name,
-                                           const attributes& attr
-                                           __attribute__((unused)))
+                                           const attributes& attr
+                                           : object_named_system{ name }
    {
        instrumentation::condition_variable::create (this);
@@ -284,6 +284,10 @@ namespace os
        // Don't call this from interrupt handlers.
        os_assert_throw (!interrupts::in_handler_mode (), EPERM);

+    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+        clock_ = attr.clock != nullptr ? attr.clock : &sysclock;
+    #endif
+
        instrumentation::condition_variable::create_return (this);
    }

@@ -371,10 +371,10 @@ namespace os
        // Don't call this from interrupt handlers.
        os_assert_err (!interrupts::in_handler_mode (), EPERM);

+    #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+        result_t res = port::condition_variable::signal (this);
+        instrumentation::condition_variable::signal_retval (this, res);
+        return res;
    #endif

```

```
+
+
+         list_.resume_one ();
+
+         instrumentation::condition_variable::signal_retval (this,
+             result::ok);
+         return result::ok;
+
+
+
+     }
+
+
+ /**
+ @ -447,6 +460,14 @@ namespace os
+ // Don't call this from interrupt handlers.
+ os_assert_err (!interrupts::in_handler_mode (), EPERM);
+
+
+
+ #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+ +
+ +     result_t res = port::condition_variable::broadcast (this);
+ +     instrumentation::condition_variable::broadcast_retval (this,
+ +         res);
+ +     return res;
+ +
+ +
+ #else
+ +
+ +     // Wake-up all threads, if any.
+ +     // Need not be inside the critical section,
+ +     // the list is protected by inner `resume_one()`.
+ @ -454,6 +475,8 @@ namespace os
+
+     instrumentation::condition_variable::broadcast_retval (this,
+         result::ok);
+     return result::ok;
+
+
+
+ #endif
+
+
+ }
+
+
+ /**
+ @ -555,6 +578,14 @@ namespace os
+ // Don't call this from critical regions.
+ os_assert_err (!scheduler::locked (), EPERM);
+
+
+
+ #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+ +
+ +     result_t res = port::condition_variable::wait (this, &mutex);
+ +     instrumentation::condition_variable::wait_retval (this, res);
+ +     return res;
+ +
+ +
+ #else
+ +
+ +     thread& crt_thread = this_thread::thread ();
+
+ +     // Prepare a list node pointing to the current thread.
+ @ -562,32 +593,52 @@ namespace os
+ // list and guaranteed to be removed before this function
+ returns.
+ internal::waiting_thread_node node{ crt_thread };
+
+
+
+ // TODO: validate
+
+
+
+ result_t res;
+ res = mutex.unlock ();
+
+ {
+ // ----- Enter critical section
+ -----
+
+ // The link and the unlock must be one step for this CPU's
+ scheduler.
```

```

+         // Once linked, the thread is `suspended`; a tick or an IPI
+         taken
+         // before `mutex.unlock()` would switch it out still OWNING
+         the mutex,
+         // and the thread that could signal it would block on that
+         mutex
+         // forever (smp-pro-cons-test stalled this way on 4 cores, 1
+         run in
+         // about 20). Other CPUs are not held back: they can still
+         take the
+         // mutex, and signal(), as soon as it is released.
+         scheduler::critical_section scs;

-     if (res != result::ok)
-     {
-         instrumentation::condition_variable::wait_retval (this,
-         res);
-         return res;
+         // ----- Enter critical section
+         -----
+         interrupts::critical_section ics;
+
+         // Add this thread to the condition variable waiting list.
+         scheduler::internal_link_node (
+         list_, node,
+         OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR);
+         // state::suspended set in above link().
+         // ----- Exit critical section
+         -----
+     }

-     {
-         // Add this thread to the condition variable waiting list.
-         list_.link (node);
-         node.thread_>waiting_node_ = &node;
-
-         res = mutex.lock ();
+         res = mutex.unlock ();

-         // Remove the thread from the node waiting list,
-         // if not already removed.
-         node.thread_>waiting_node_ = nullptr;
-         node.unlink ();
+         if (res != result::ok)
+         {
+             scheduler::internal_unlink_node (node);
+             instrumentation::condition_variable::wait_retval (this,
+             res);
+             return res;
+         }
+         // ----- Exit critical section
+         -----
+     }

+     port::scheduler::reschedule ();
+
+     // Remove the thread from the condition variable waiting list,
+     // if not already removed by signal() / broadcast().
+     scheduler::internal_unlink_node (node);
+
+     res = mutex.lock ();
+
+     instrumentation::condition_variable::wait_retval (this, res);
+     return res;
+
+ #endif
+ }

+ /**
@@ -715,6 +766,14 @@ namespace os
+     // Don't call this from critical regions.

```

```

os_assert_err (!scheduler::locked (), EPERM);

+#if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+
+    result_t res = port::condition_variable::timed_wait (this,
+        &mutex, timeout);
+    instrumentation::condition_variable::timed_wait_retval (this,
+        res);
+    return res;
+
+#else
+
+    thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.
@@ -722,32 +781,62 @@ namespace os
    // list and guaranteed to be removed before this function
    returns.
    internal::waiting_thread_node node{ crt_thread };

-    // TODO: validate
+    clock* clk = (clock_ != nullptr) ? clock_ : &sysclock;
+    internal::clock_timestamps_list& clock_list = clk->steady_list
+        ();
+    clock::timestamp_t timeout_timestamp = clk->steady_now () +
+        timeout;

+    // Prepare a timeout node pointing to the current thread.
+    internal::timeout_thread_node timeout_node{ timeout_timestamp,
+        crt_thread };

    result_t res;
-    res = mutex.unlock ();
+    {
+        // ----- Enter critical section
+        -----
+        // Link and unlock as one step for this CPU's scheduler; see
+        wait().
+        scheduler::critical_section scs;

-    if (res != result::ok)
+    {
+        instrumentation::condition_variable::timed_wait_retval
+        (this, res);
+        return res;
+        // ----- Enter critical section
+        -----
+        interrupts::critical_section ics;

+        // Add this thread to the condition variable waiting list,
+        // and the clock timeout list.
+        scheduler::internal_link_node (
+            list_, node, clock_list, timeout_node,
+            OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR);
+        // state::suspended set in above link().
+        // ----- Exit critical section
+        -----
+    }

-    {
-        // Add this thread to the condition variable waiting list.
-        list_.link (node);
-        node.thread->waiting_node_ = &node;
-
-        res = mutex.timed_lock (timeout);
+    res = mutex.unlock ();

-        // Remove the thread from the node waiting list,
-        // if not already removed.
-        node.thread->waiting_node_ = nullptr;

```

```

-     node.unlink ();
+     if (res != result::ok)
+     {
+         scheduler::internal_unlink_node (node, timeout_node);
+         instrumentation::condition_variable::timed_wait_retval
+         (this, res);
+         return res;
+     }
+     // ----- Exit critical section
+     -----
+ }

+ port::scheduler::reschedule ();
+
+ // Remove the thread from the condition variable waiting list,
+ // if not already removed by signal() / broadcast() and from
+ // the clock
+ // timeout list, if not already removed by the timer.
+ scheduler::internal_unlink_node (node, timeout_node);
+
+ res = mutex.lock ();
+
+ if (res == result::ok && clk->steady_now () >=
+     timeout_timestamp)
+ {
+     res = ETIMEDOUT;
+ }
+
+ instrumentation::condition_variable::timed_wait_retval (this,
+     res);
+ return res;
+
+ #endif
+ }

+ // -----

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/condvar-atomic-wait-
  clock-binding \
  --title "fix(rtos): eliminate condvar lost-signal race & bind
  timed_wait to attribute clock"

```

PR #20: C++ std::thread Native Handle Synchronization & Functor Lifetime

- **Single Theme:** ISO C++ standard thread wrapper join semantics and closure memory safety.
- **Repository:** micro-os-plus-iii
- **Files Touched:** include/cmsis-plus/estd/thread_internal.h, src/libcpp/thread-cpp.h
- **Problem & Solution:**
 - thread::join() waits directly on the native RTOS thread handle via id_.native_thread_->join() before deleting thread resources.
 - Retains function_object_ pointer in thread_internal.h, ensuring functor closures are not freed until join() completes even after the kernel clears func_args_.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/estd/thread_internal.h b/include/
cmsis-plus/estd/thread_internal.h
index 1d97128e..389498e8 100644
--- a/include/cmsis-plus/estd/thread_internal.h
+++ b/include/cmsis-plus/estd/thread_internal.h
@@ -146,6 +146,12 @@ private:
     using function_object_deleter_t = void (*) (void*);
     function_object_deleter_t function_object_deleter_ = nullptr;

+ // The bound function object, remembered here so it can be deleted
+ // even
+ // after the kernel has cleared the thread's `func_args_` on exit.
+ // Relying
+ // on `native_thread_->function_args()` leaks it whenever the
+ // thread has
+ // already finished by the time `join()` runs (the common case).
+ void* function_object_ = nullptr;
+
 public:
 };

@@ -358,6 +364,10 @@ thread::thread (Callable_T&& f, Args_T&&... args)
     Function_object* funct_obj = new Function_object (std::bind (
         std::forward<Callable_T> (f), std::forward<Args_T> (args)...));

+ // Remember the object ourselves; the kernel may clear its own copy
+ // when
+ // the thread exits, before `join()` gets a chance to read it.
+ function_object_ = funct_obj;
+
     // The function to start the thread is a custom proxy that
     // knows how to get the variadic arguments.
     #pragma GCC diagnostic push
diff --git a/src/libcpp/thread-cpp.h b/src/libcpp/thread-cpp.h
index c14f5bba..af572d23 100644
--- a/src/libcpp/thread-cpp.h
+++ b/src/libcpp/thread-cpp.h
@@ -36,11 +36,14 @@ thread::delete_system_thread (void)
 {
     if (id_ != id ())
     {
-        void* args = id_.native_thread_->function_args ();
-        if (args != nullptr && function_object_deleter_ != nullptr)
+        if (function_object_ != nullptr && function_object_deleter_ !=
+            nullptr)
         {
             // Manually delete the function object used to store
             arguments.
             function_object_deleter_ (args);
+            // `function_object_` is our own copy: the kernel clears
+            its
         }
     }
 }
```

```

+         // `func_args_` on exit, so reading it here would leak
+         whenever the
+         // thread has already finished.
+         function_object_deleter_ (function_object_);
+         function_object_ = nullptr;
+     }

    // Manually delete the system thread.
@@ -68,6 +71,7 @@ thread::swap (thread& t) noexcept
    {
        std::swap (id_, t.id_);
        std::swap (function_object_deleter_, t.function_object_deleter_);
+    std::swap (function_object_, t.function_object_);
    }

    bool
@@ -81,7 +85,28 @@ thread::join ()
    {
        os::trace::printf ("%s() @%p\n", __func__, this);

-    delete_system_thread ();
+    if (id_ != id ())
+    {
+        // Wait for the thread to end, as ISO join() does, before
+        // freeing what
+        // it runs on. Deleting it straight away -- which is what this
+        // did --
+        // frees its bound arguments and kills the system thread; on
+        // one core
+        // the thread had usually finished by then, on SMP it is still
+        // running
+        // on another core.
+        id_.native_thread->join ();
+
+        if (function_object_ != nullptr && function_object_deleter_ !=
+            nullptr)
+        {
+            // Manually delete the function object used to store
+            // arguments.
+            // Use our own pointer, because the kernel clears
+            // `func_args_` when
+            // the thread exits -- which happens before `join()`
+            // returns for any
+            // short-lived thread.
+            function_object_deleter_ (function_object_);
+            function_object_ = nullptr;
+        }
+
+        // Manually delete the system thread, destroyed by now.
+        delete id_.native_thread;
+    }

    id_ = id ();
    os::trace::printf ("%s() @%p joined\n", __func__, this);

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:fix/cpp-std-thread-functor-
  lifetime \
  --title "fix(libcpp): std::thread native handle join barrier &
  functor lifetime retention"

```

PR #21: Message Queue Preemptive Rescheduling on Wakeups

- **Single Theme:** Immediate scheduler preemption upon message queue wakeups.
- **Repository:** micro-os-plus-iii
- **File Touched:** src/rtos/os-mqueue.cpp
- **Problem & Solution:**
 - Calls `port::scheduler::reschedule()` immediately after message send/receive wakes a higher-priority thread across all code paths (`send`, `try_send`, `timed_send`, `receive`, `try_receive`, `timed_receive`).
- **Exact Unified Diff Applied:**

```
diff --git a/src/rtos/os-mqueue.cpp b/src/rtos/os-mqueue.cpp
index 3f134fae..cd46c58a 100644
--- a/src/rtos/os-mqueue.cpp
+++ b/src/rtos/os-mqueue.cpp
@@ -961,18 +961,27 @@ namespace os

    #else

+    bool sent;
+    {
+        // ----- Enter critical section
+        -----
+        interrupts::critical_section ics;

-        if (internal_try_send_ (msg, nbytes, mprio))
-        {
-            instrumentation::message_queue::send_retval (this,
-            result::ok);
-            return result::ok;
-        }
+        sent = internal_try_send_ (msg, nbytes, mprio);
+        // ----- Exit critical section
+        -----
+    }

+    if (sent)
+    {
+        // internal_try_send_() resumed a waiting receiver.
+        Rescheduling here,
+        // with the kernel lock released, lets it run at once -- as
+        the ARM
+        // ports do when PendSV is taken on the way out of the
+        section --
+        // instead of waiting for the next tick.
+        port::scheduler::reschedule ();
+
+        instrumentation::message_queue::send_retval (this,
+        result::ok);
+        return result::ok;
+    }

+    thread& crt_thread = this_thread::thread ();

+    // Prepare a list node pointing to the current thread.
@@ -1083,24 +1092,30 @@ namespace os
    // Don't call this from high priority interrupts.
    assert (port::interrupts::is_priority_valid ());

+    bool sent;
+    {
+        // ----- Enter critical section
+        -----
+        interrupts::critical_section ics;
```

```

-         if (internal_try_send_ (msg, nbytes, mprio))
-         {
-             instrumentation::message_queue::try_send_retval (this,
result::ok);
-             return result::ok;
-         }
-         else
-         {
-             instrumentation::message_queue::try_send_retval (this,
-
EWOULDBLOCK);
-             return EWOULDBLOCK;
-         }
+         sent = internal_try_send_ (msg, nbytes, mprio);
+         // ----- Exit critical section
+         -----
+     }

+     if (sent)
+     {
+         // internal_try_send_() resumed a waiting receiver.
Rescheduling here,
+         // with the kernel lock released, lets it run at once -- as
the ARM
+         // ports do when PendSV is taken on the way out of the
section --
+         // instead of waiting for the next tick.
+         port::scheduler::reschedule ();
+
+         instrumentation::message_queue::try_send_retval (this,
result::ok);
+         return result::ok;
+     }
+
+     instrumentation::message_queue::try_send_retval (this,
EWOULDBLOCK);
+     return EWOULDBLOCK;
+
+ #endif
+ }

@@ -1185,19 +1200,27 @@ namespace os

    // Extra test before entering the loop, with its inherent
weight.
    // Trade size for speed.
+    bool sent;
+    {
+        // ----- Enter critical section
+        -----
+        interrupts::critical_section ics;

-         if (internal_try_send_ (msg, nbytes, mprio))
-         {
-             instrumentation::message_queue::timed_send_retval (this,
result::ok);
-             return result::ok;
-         }
+         sent = internal_try_send_ (msg, nbytes, mprio);
+         // ----- Exit critical section
+         -----
+     }

+     if (sent)
+     {
+         // internal_try_send_() resumed a waiting receiver.
Rescheduling here,
+         // with the kernel lock released, lets it run at once -- as
the ARM
+         // ports do when PendSV is taken on the way out of the
section --
+         // instead of waiting for the next tick.

```

```

+         port::scheduler::reschedule ();
+
+         instrumentation::message_queue::timed_send_retval (this,
+ result::ok);
+         return result::ok;
+     }
+
+     thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.
@@ -1338,18 +1361,26 @@ namespace os

    // Extra test before entering the loop, with its inherent
    weight.
    // Trade size for speed.
+ bool received;
+ {
+     // ----- Enter critical section
+     -----
+     interrupts::critical_section ics;

-     if (internal_try_receive_ (msg, nbytes, mprio))
-     {
-         instrumentation::message_queue::receive_retval (this,
- result::ok);
-         return result::ok;
-     }
+     received = internal_try_receive_ (msg, nbytes, mprio);
+     // ----- Exit critical section
+     -----
+ }

+ if (received)
+ {
+     // internal_try_receive_() resumed a waiting sender.
+     Rescheduling
+     // here, with the kernel lock released, lets it run at once
+     instead
+     // of waiting for the next tick.
+     port::scheduler::reschedule ();
+
+     instrumentation::message_queue::receive_retval (this,
+ result::ok);
+     return result::ok;
+ }
+
+     thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.
@@ -1463,25 +1494,29 @@ namespace os

    // Don't call this from high priority interrupts.
    assert (port::interrupts::is_priority_valid ());

+ bool received;
+ {
+     // ----- Enter critical section
+     -----
+     interrupts::critical_section ics;

-     if (internal_try_receive_ (msg, nbytes, mprio))
-     {
-         instrumentation::message_queue::try_receive_retval (this,
- result::ok);
-         return result::ok;
-     }
-     else
-     {
-         instrumentation::message_queue::try_receive_retval (this,
- EWOULDBLOCK);

```

```

-         return EWOULDBLOCK;
-     }
+     received = internal_try_receive_ (msg, nbytes, mprio);
+     // ----- Exit critical section
+     -----
+ }

+ if (received)
+ {
+     // internal_try_receive_() resumed a waiting sender.
+     Rescheduling
+     // here, with the kernel lock released, lets it run at once
+     instead
+     // of waiting for the next tick.
+     port::scheduler::reschedule ();
+
+     instrumentation::message_queue::try_receive_retval (this,
+ result::ok);
+     return result::ok;
+ }
+
+ instrumentation::message_queue::try_receive_retval (this,
+ EWOULDBLOCK);
+ return EWOULDBLOCK;
+
+ #endif
+ }

@@ -1578,19 +1613,27 @@ namespace os

    // Extra test before entering the loop, with its inherent
    weight.
    // Trade size for speed.
+ bool received;
+ {
+     // ----- Enter critical section
+     -----
+     interrupts::critical_section ics;

-     if (internal_try_receive_ (msg, nbytes, mprio))
-     {
-         instrumentation::message_queue::timed_receive_retval
- (this,
-
- result::ok);
-         return result::ok;
-     }
+     received = internal_try_receive_ (msg, nbytes, mprio);
+     // ----- Exit critical section
+     -----
+ }

+ if (received)
+ {
+     // internal_try_receive_() resumed a waiting sender.
+     Rescheduling
+     // here, with the kernel lock released, lets it run at once
+     instead
+     // of waiting for the next tick.
+     port::scheduler::reschedule ();
+
+     instrumentation::message_queue::timed_receive_retval (this,
+ result::ok);
+     return result::ok;
+ }
+
+ thread& crt_thread = this_thread::thread ();

```

```
// Prepare a list node pointing to the current thread.
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \  
  --base xpack-development --head dan-maio:fix/mqueue-preemptive-  
    reschedule \  
  --title "fix(rtos): invoke port::scheduler::reschedule() after  
    message queue wakeups"
```

PR #22: High-Resolution Monotonic Clock Port Synchronization

- **Single Theme:** Cross-repository monotonic hardware clock interface and SysTick overflow detection.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-cortexm, micro-os-plus-iii-posix-arch
- **Files Touched:** Kernel os-decls.h & os-clocks.cpp, Cortex-M os-inlines.h, POSIX-Arch os-inlines.h
- **Problem & Solution:**
 - Kernel declares has_hardware_counter() and hardware_counter() in os-decls.h and fast-paths clock_highres::now().
 - Cortex-M implements pending SysTick inspection via SCB->ICSR & SCB_ICSR_PENDSTSET_Msk.
 - POSIX-arch implements host CLOCK_MONOTONIC timestamping using timespec tp; (-Werror=redundant-tags).
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/rtos/os-decls.h b/include/cmsis-plus/rtos/os-decls.h
index 5d625d5b..2e39d590 100644
--- a/include/cmsis-plus/rtos/os-decls.h
+++ b/include/cmsis-plus/rtos/os-decls.h
@@ -22,6 +22,11 @@
 #endif
 // Include the non-portable portable types, enums and constants
 // declarations.
 #include <cmsis-plus/rtos/port/os-decls.h>
+// Port compat (develop/xbpack-development): the C-ABI declarations
+// were split
+// out of the C++ <os.h> chain, but kernel C++ sources
+// (os_systick_handler in
+// os-clocks.cpp) and the ports (os_thread_t) still rely on them
+// being visible
+// via <os.h>, as in v7.0.1. Needed by both the SMP and single-CPU
+// builds.
+#include <cmsis-plus/rtos/os-c-decls.h>

#include <cmsis-plus/diag/trace.h>

@@ -1025,6 +1030,12 @@ namespace os
     static void
     start (void);

+    static constexpr bool
+    has_hardware_counter (void) noexcept;
+
+    static uint64_t
+    hardware_counter (void) noexcept;
+
+    static uint32_t
+    cycles_per_tick (void);

diff --git a/src/rtos/os-clocks.cpp b/src/rtos/os-clocks.cpp
index f91f2ae3..94b457ff 100644
--- a/src/rtos/os-clocks.cpp
+++ b/src/rtos/os-clocks.cpp
@@ -793,11 +793,19 @@ namespace os
     clock::timestamp_t
     clock_highres::now (void)
     {
-        // ----- Enter critical section
-        -----
-        interrupts::critical_section ics;
+        if constexpr (port::clock_highres::has_hardware_counter ())
+        {
+            return port::clock_highres::hardware_counter ();
         }
     }
 }
```

```

+     }
+     else
+     {
+         // Prevent inconsistent values.
+         // ----- Enter critical section
+         -----
+         interrupts::critical_section ics;
+
-         return steady_count_ + port::clock_highres::cycles_since_tick
+         ();
-         // ----- Exit critical section
+         -----
+         return steady_count_ +
+         port::clock_highres::cycles_since_tick ();
+         // ----- Exit critical section
+         -----
+     }
+ }

// -----

diff --git a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/
os-inlines.h b/micro-os-plus-iii-cortexm/include/cmsis-plus/
rtos/port/os-inlines.h
index bcf1c4..33a1a69 100644
--- a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
inlines.h
+++ b/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
inlines.h
@@ -95,12 +95,14 @@ namespace os
     return locked (state::unlocked);
 }

+
+     inline bool __attribute__ ((always_inline))
+     locked (void)
+     {
+         return lock_state != state::unlocked;
+     }
+
+
+     inline void __attribute__ ((always_inline))
+     wait_for_interrupt (void)
+     {
@@ -230,6 +232,7 @@ namespace os

#endif

+
+         // Apparently not required by architecture, but used by
+         // FreeRTOS, with an unconvincing motivation ("... ensure
+         // the code is completely within the specified behaviour
@@ -244,6 +247,7 @@ namespace os
     inline void __attribute__ ((always_inline))
     critical_section::exit (rtos::interrupts::state_t state)
     {
+
+     #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)

+     #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
@@ -362,6 +366,18 @@ namespace os
        ;
    }

+
+     inline constexpr bool __attribute__ ((always_inline))
+     clock_highres::has_hardware_counter (void) noexcept
+     {
+         return false;
+     }
+
+

```

```

+     inline uint64_t __attribute__ ((always_inline))
+     clock_highres::hardware_counter (void) noexcept
+     {
+         return 0;
+     }
+
+     inline uint32_t __attribute__ ((always_inline))
+     clock_highres::input_clock_frequency_hz (void)
+     {
@@ -391,7 +407,7 @@ namespace os
        // yet processed, so the total cycles count in steady_count_
        // does not yet reflect the correct value and needs to be
        // adjusted by one full cycle length.
-       if (SysTick->CTRL & SCB_ICSR_PENDSTSET_Msk)
+       if ((SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) != 0)
        {
            // Sample the decrementing counter again to validate the
            // initial sample.

diff --git a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/
port/os-inlines.h b/micro-os-plus-iii-posix-arch/include/
cmsis-plus/rtos/port/os-inlines.h
index 93944a5..0ec7ab7 100644
--- a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
inlines.h
+++ b/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
inlines.h
@@ -129,6 +129,21 @@ namespace os

        //
        =====

+     inline constexpr bool __attribute__ ((always_inline))
+     clock_highres::has_hardware_counter (void) noexcept
+     {
+         return true;
+     }
+
+     inline uint64_t __attribute__ ((always_inline))
+     clock_highres::hardware_counter (void) noexcept
+     {
+         timespec tp;    // NB: no 'struct' - baseline gate has
+         -Werror=redundant-tags
+         ::clock_gettime (CLOCK_MONOTONIC, &tp);
+         return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
+             + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
+     }
+
+     } /* namespace port */
+     } /* namespace rtos */
+     } /* namespace os */

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base xpack-development --head dan-maio:feat/highres-clock-port-
  sync \
  --title
  "feat(rtos): cross-repo high-resolution monotonic clock port
  synchronization"

```

PR #23: Multi-Core SMP Kernel Infrastructure, Port Contracts & native-smp

- **Single Theme:** Multi-core SMP scheduling engine, per-CPU current thread arrays, affinity matching, and host dual-core platform.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-cortexm, micro-os-plus-iii-posix-arch
- **Base Branch:** step/13 | **Head Branch:** step/14
- **Problem & Solution:**
 - Declares `current_thread_[OS_NCPU]` array indexed by `port_cpu_id()`, per-CPU idle threads `os_idle_thread_core[OS_NCPU]`, and affinity validation in `os-core.cpp`.
 - Implements 5-stage stack pointer claim/publish protocol preventing stack corruption across cores.
 - Adds tests/platforms/native-smp/ setting `OS_USE_SMP_SCHEDULER=1` via `target_compile_definitions` for real host dual-core verification.
- **Exact Unified Diff Applied:**

```
diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index 86dc5429..e3cddb30 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -316,7 +316,6 @@ extern "C"
     * @brief Used to check reused threads.
     */
     os_thread_state_initialising = 6,
-
-    /**
     * @brief In process of being destroyed.
     */
@@ -546,6 +545,10 @@ extern "C"
     */
     bool th_enable_assert_reuse;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t th_cpu_affinity;
+#endif
+
     } os_thread_attr_t;

     /**
@@ -592,6 +595,9 @@ extern "C"
     os_thread_state_t state;
     os_thread_prio_t prio_assigned;
     os_thread_prio_t prio_inherited;
+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t cpu_affinity;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
     bool interrupted;
     os_internal_evflags_t event_flags;
     #if defined(OS_INCLUDE_RTOS_CUSTOM_THREAD_USER_STORAGE)
diff --git a/include/cmsis-plus/rtos/os-sched.h b/include/cmsis-plus/rtos/os-sched.h
index 0a2b4e78..f3b6ab60 100644
--- a/include/cmsis-plus/rtos/os-sched.h
+++ b/include/cmsis-plus/rtos/os-sched.h
@@ -55,7 +55,11 @@ namespace os

     #if !defined(OS_USE_RTOS_PORT_SCHEDULER)
         extern bool is_preemptive_;
     #if defined(OS_USE_SMP_SCHEDULER)
         extern thread* volatile current_thread_[OS_NCPU];
     #else
```

```

extern thread* volatile current_thread_;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
extern internal::ready_threads_list ready_threads_list_;
#endif /* !defined(OS_USE_RTOS_PORT_SCHEDULER) */

diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index cac6777f..385e6cb2 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -745,6 +745,14 @@ namespace os
    friend port::stack::element_t*
        port::scheduler::switch_stacks (port::stack::element_t* sp);

+#if defined(OS_USE_SMP_SCHEDULER)
+    // The SMP picker accesses state_ / context_ directly.
+    friend void
+        rtos::scheduler::internal_switch_threads (void);
+    // The idle reaper reads context_ to know a terminated
+    thread's
+    // context is no longer live on another CPU (see os-
+    idle.cpp).
+    friend void ::os_rtos_idle_actions (void);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    #endif
    /**
@@ -880,6 +888,9 @@ namespace os

        bool th_enable_assert_reuse = false;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t th_cpu_affinity = 0xFFFFFFFFu;
+#endif

        // Add more attributes here.

@@ -1198,6 +1209,13 @@ namespace os
        priority_t
        priority_inherited (void);

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t
+    cpu_affinity (void) const;
+
+    void
+    cpu_affinity (uint32_t mask);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    #if 0
        // ???
@@ -1696,6 +1714,9 @@ namespace os
        priority_t volatile prio_assigned_ = priority::none;
        priority_t volatile prio_inherited_ = priority::none;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

        bool volatile interrupted_ = false;

diff --git a/src/rtos/os-c-wrapper.cpp b/src/rtos/os-c-wrapper.cpp
index 4bd9e1bb..1c37050d 100644
--- a/src/rtos/os-c-wrapper.cpp
+++ b/src/rtos/os-c-wrapper.cpp
@@ -239,6 +239,8 @@ @@ static_assert (os_thread_state_terminated ==
    thread::state::terminated,
        "adjust os_thread_state_terminated");

```

```

static_assert (os_thread_state_destroyed == thread::state::destroyed,
               "adjust os_thread_state_destroyed");
+static_assert (os_thread_state_destroying ==
               thread::state::destroying,
+               "adjust os_thread_state_destroying");

static_assert (os_timer_once == timer::run::once, "adjust
os_timer_once");
static_assert (os_timer_periodic == timer::run::periodic,
@@ -3538,6 +3540,12 @@ osThreadCreate (const osThreadDef_t*
thread_def, void* args)
    thread::attributes attr;
    attr.th_priority = thread_def->tpriority;
    attr.th_stack_size_bytes = thread_def->stacksize;
+if defined(OS_USE_SMP_SCHEDULER)
+ // CMSIS-RTOS v1 assumes a single-core priority scheduling model;
+ // pin threads created through the CMSIS-RTOS v1 API to Core 0 at
    construction.
+ attr.th_cpu_affinity = (1u << 0);
+endif
+
    // Creating thread with invalid priority should fail (validator
    requirement).
    if (thread_def->tpriority >= osPriorityError)
    {
@@ -3575,6 +3583,12 @@ osThreadCreate (const osThreadDef_t*
thread_def, void* args)
        args, attr);
#pragma GCC diagnostic pop

+if defined(OS_USE_SMP_SCHEDULER)
+ // CMSIS-RTOS v1 assumes a single-core priority scheduling
    model;
+ // pin threads created through the CMSIS-RTOS v1 API to
    Core 0.
+ th->cpu_affinity (1u << 0);
+endif
+
    // No need to yield here, already done by constructor.
    return reinterpret_cast<osThreadId> (th);
}
diff --git a/src/rtos/os-core.cpp b/src/rtos/os-core.cpp
index c9d6859e..69befb2d 100644
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -24,6 +24,14 @@

// -----

+if defined(OS_USE_SMP_SCHEDULER)
+// Provided by the SMP port (e.g. Cortex-A7); returns the current
    core index.
+pragma GCC diagnostic push
+pragma GCC diagnostic ignored "-Wredundant-decls"
+extern "C" unsigned port_cpu_id(void);
+pragma GCC diagnostic pop
+endif /* defined(OS_USE_SMP_SCHEDULER) */
+
namespace
{
    #if defined(OS_HAS_INTERRUPTS_STACK)
@@ -130,10 +138,21 @@ namespace os

    #pragma GCC diagnostic push
    #pragma GCC diagnostic ignored "-Wcast-align"
+if defined(OS_USE_SMP_SCHEDULER)
+ // One running thread per CPU, indexed by port_cpu_id().
+ thread* volatile current_thread_[OS_NCPU]
+ = {reinterpret_cast<thread*> (&tiny_thread)};
+else

```

```

        thread* volatile current_thread_
            = reinterpret_cast<thread*> (&tiny_thread);
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
        #pragma GCC diagnostic pop

+ #if defined(OS_USE_SMP_SCHEDULER)
+     // One idle thread per CPU (secondary cores register theirs at
+     // boot).
+     thread* os_idle_thread_core[OS_NCPU] = {nullptr};
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
        #pragma GCC diagnostic push
        #if defined(__clang__)
        #pragma clang diagnostic ignored "-Wglobal-constructors"
@@ -462,6 +481,45 @@ namespace os

        #if !defined(OS_USE_RTOS_PORT_SCHEDULER)

+ #if defined(OS_USE_SMP_SCHEDULER)
+     // Affinity gate: a per-core idle thread is pinned to its own
+     // core,
+     // everything else follows its cpu_affinity() bitmask.
+     //
+     // Identity against os_idle_thread_core[] is the authoritative
+     // test: it
+     // covers every core whatever the thread is named. The name
+     // rules are
+     // only the fallback for the boot window, between the moment a
+     // secondary
+     // idle thread is linked into the ready list by its constructor
+     // and the
+     // moment it is registered in os_idle_thread_core[]. Without a
+     // rule for
+     // core OS_NCPU-1 an unregistered idle thread inherits the
+     // default
+     // 0xFFFFFFFF affinity, so another core can pick it out of the
+     // ready list
+     // and run it on the same stack as its own core does.
+     static bool
+     is_thread_allowed_on_cpu (thread* th, unsigned cpu)
+     {
+         for (unsigned c = 0; c < OS_NCPU; ++c)
+         {
+             if (th == scheduler::os_idle_thread_core[c])
+             {
+                 return (cpu == c);
+             }
+         }
+         const char* name = th->name ();
+         if (name != nullptr && name[0] == 'i' && name[1] == 'd'
+             && name[2] == 'l' && name[3] == 'e')
+         {
+             if (name[4] == '\0' || name[4] == '0')
+             {
+                 return (cpu == 0);
+             }
+             if (name[4] >= '1' && name[4] <= '9' && name[5] == '\0')
+             {
+                 return (cpu == static_cast<unsigned> (name[4] -
+ '0'));
+             }
+         }
+         return (th->cpu_affinity () & (1u << cpu)) != 0;
+     }
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
        void
        internal_switch_threads (void)
        {
@@ -490,7 +548,11 @@ namespace os

```

```

        scheduler::statistics::cpu_cycles_ += delta;

        // Accumulate durations to old thread.
#ifdef OS_USE_SMP_SCHEDULER
+        scheduler::current_thread_[port_cpu_id()]-
>statistics_.cpu_cycles_ += delta;
#else
        scheduler::current_thread_>statistics_.cpu_cycles_ += delta;
#endif /* defined(OS_USE_SMP_SCHEDULER) */

        // Remember the timestamp for the next context switch.
        scheduler::statistics::switch_timestamp_ = now;
@@ -501,6 +563,77 @@ namespace os
        // current thread and return the top priority thread.
        if (!locked ())
        {
#ifdef OS_USE_SMP_SCHEDULER
+        unsigned cpu = port_cpu_id ();
+        thread* old_thread = scheduler::current_thread_[cpu];
+
+        instrumentation::thread::suspended (
+        old_thread,
+        OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_SWITCH);
+
+        bool is_old_idle = false;
+        for (unsigned c = 0; c < OS_NCPU; ++c)
+        {
+            if (old_thread == scheduler::os_idle_thread_core[c])
+            {
+                is_old_idle = true;
+                break;
+            }
+        }
+
+        if (!is_old_idle)
+        {
+            old_thread->internal_relink_running_ ();
+        }
+        else
+        {
+            old_thread->state_ = thread::state::ready;
+        }
+
+        // Affinity-aware pick from the ready list. Skip threads
+        whose
+        // stack_ptr is still nullptr (their context is live in
+        another
+        // CPU's registers -- publish is deferred to the asm
+        restore path);
+        // this CPU's own outgoing thread may always be re-
+        picked.
+        thread* next_thread = nullptr;
+        auto* sentinel =
+        reinterpret_cast<internal::waiting_thread_node*> (
+        &scheduler::ready_threads_list_);
+        auto* node = const_cast<internal::waiting_thread_node*> (
+        static_cast<const volatile
+        internal::waiting_thread_node*> (
+        scheduler::ready_threads_list_.head ()));
+
+        while (node != nullptr && node != sentinel)
+        {
+            thread* th = node->thread_;
+            if (th != nullptr && is_thread_allowed_on_cpu (th,
+            cpu)
+
+            && (th == old_thread
+            || (th->state_ != thread::state::running
+            && th->context_.port_.stack_ptr !=
+            nullptr)))
+        {

```

```

+         next_thread = th;
+         node->unlink ();
+         next_thread->state_ = thread::state::running;
+         break;
+     }
+     node = static_cast<internal::waiting_thread_node*> (
+         node->next ());
+ }
+
+     if (next_thread != nullptr)
+     {
+         scheduler::current_thread[cpu] = next_thread;
+     }
+     else
+     {
+         scheduler::current_thread[cpu]
+             = scheduler::os_idle_thread_core[cpu];
+         if (scheduler::os_idle_thread_core[cpu] != nullptr)
+         {
+             scheduler::os_idle_thread_core[cpu]->state_
+                 = thread::state::running;
+         }
+     }
+
+     instrumentation::thread::active
+         (scheduler::current_thread[cpu]);
+ #else
+
+         instrumentation::thread::suspended (
+             scheduler::current_thread_,
+             OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_SWITCH);
+ @@ -512,6 +645,7 @@ namespace os
+             = scheduler::ready_threads_list_.unlink_head ();
+
+             instrumentation::thread::active
+                 (scheduler::current_thread_);
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+     }
+
+     // ***** Pointer switched to new thread! *****
+ @@ -527,7 +661,11 @@ namespace os
+         scheduler::statistics::context_switches++;
+
+         // Increment new thread context switches.
+ #if defined(OS_USE_SMP_SCHEDULER)
+         scheduler::current_thread_[port_cpu_id()]-
+             >statistics_.context_switches++;
+ #else
+         scheduler::current_thread_->statistics_.context_switches++;
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+ #endif /* defined(OS_INCLUDE_RTOS_STATISTICS_THREAD_CONTEXT_SWITCHES)
+ */
+ }
+
+ diff --git a/src/rtos/os-idle.cpp b/src/rtos/os-idle.cpp
+ index 0db1ab35..10272366 100644
+ --- a/src/rtos/os-idle.cpp
+ +++ b/src/rtos/os-idle.cpp
+ @@ -80,6 +80,28 @@ os_rtos_idle_actions (void)
+         node = const_cast<internal::waiting_thread_node*> (
+             scheduler::terminated_threads_list_.head ());
+         thread* th = node->thread_;
+ #if defined(OS_USE_SMP_SCHEDULER)
+ +         // A thread links itself here in internal_exit() and only
+ +         then
+ +         // switches away, so on SMP it may still be running on
+ +         another CPU.
+ +         // Destroying it now frees the stack that CPU is executing
+ +         on, and

```

```

+         // lets the joiner delete the object under it. Reap it only
+         // once no
+         // CPU has it current and its context has been saved -- the
+         // same
+         // rule internal_switch_threads() applies to the ready list
+         // -- and
+         // otherwise leave it linked for the next idle pass.
+         bool live = (__atomic_load_n (&th->context_.port_.stack_ptr,
+         __ATOMIC_ACQUIRE)
+         == nullptr);
+         for (unsigned c = 0; c < OS_NCPU && !live; ++c)
+         {
+             live = (__atomic_load_n (&scheduler::current_thread_[c],
+             __ATOMIC_ACQUIRE)
+             == th);
+         }
+         if (live)
+         {
+             break;
+         }
+     }
+     if (th->state_ == thread::state::destroying
+         || th->state_ == thread::state::destroyed)
+     {
diff --git a/src/rtos/os-main.cpp b/src/rtos/os-main.cpp
index 67782880..546c709e 100644
--- a/src/rtos/os-main.cpp
+++ b/src/rtos/os-main.cpp
@@ -165,6 +165,9 @@ int

        thread::attributes attr = thread::initializer;
        attr.th_stack_size_bytes = OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES;
+    #if defined(OS_USE_SMP_SCHEDULER)
+    + attr.th_cpu_affinity = (1u << 0);
+    #endif
        #pragma GCC diagnostic push
        #if defined(__clang__)
        #pragma clang diagnostic ignored "-Wcast-function-type-strict"
@@ -176,6 +179,12 @@ int

        #endif /* defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS) */

+    #if defined(OS_USE_SMP_SCHEDULER)
+    + // Pin main thread to core 0 so per-core peripherals (NVIC,
+    +   SysTick) initialized
+    + // on the boot core remain consistent for the main thread
+    +   lifecycle.
+    + os_main_thread->cpu_affinity (1u << 0);
+    #endif
+
+    #if !defined(OS_USE_RTOS_PORT_SCHEDULER)
+    + os_startup_create_thread_idle ();
+    #endif /* !defined(OS_USE_RTOS_PORT_SCHEDULER) */
diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
index 79b0a71c..7255e75c 100644
--- a/src/rtos/os-thread.cpp
+++ b/src/rtos/os-thread.cpp
@@ -19,6 +19,19 @@
@@ -19,6 +19,19 @@
#include <memory>
#include <stdexcept>

+    #if defined(OS_USE_SMP_SCHEDULER)
+    #pragma GCC diagnostic push
+    #pragma GCC diagnostic ignored "-Wredundant-decls"
+    extern "C" unsigned port_cpu_id(void);
+    #pragma GCC diagnostic pop
+    extern "C" void port_smp_ipi(unsigned cpu) __attribute__((weak));
+

```

```

+extern "C" void __attribute__((weak))
+port_smp_ipi (unsigned cpu)
+{
+ (void) cpu;
+}
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

// -----

@@ -272,6 +285,9 @@ namespace os
// Must be explicit here, since they are not done in the
members
// declarations to allow th_enable_assert_reuse.
state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+ cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

instrumentation::thread::create_return (this);
}

@@ -286,6 +302,9 @@ namespace os
// Must be explicit here, since they are not done in the
members
// declarations to allow th_enable_assert_reuse.
state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+ cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

instrumentation::thread::create_return (this);
}

@@ -440,6 +459,9 @@ namespace os
#endif /* DEBUG */

state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+ cpu_affinity_ = attr.th_cpu_affinity;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

allocator_ = &allocator;

@@ -578,7 +600,11 @@ namespace os

if (!scheduler::started ())
{
+#if defined(OS_USE_SMP_SCHEDULER)
+ scheduler::current_thread_[port_cpu_id()] = this;
+#else
+ scheduler::current_thread_ = this;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
}

// Add to ready list, but do not yield yet.
@@ -697,6 +723,23 @@ namespace os

port::scheduler::reschedule ();

+#if defined(OS_INTEGER_RTOS_PORT_NCPU) && (OS_INTEGER_RTOS_PORT_NCPU
> 1)
+ // If the waking CPU is not in this thread's affinity mask,
+ // actively trigger a reschedule IPI to an eligible core so it
+ // doesn't wait up to 1 ms for its next local clock tick.
+ unsigned this_cpu = port_cpu_id ();
+ if ((cpu_affinity () & (1u << this_cpu)) == 0)
+ {
+ for (unsigned c = 0; c < OS_NCPU; ++c)
+ {

```

```

+         if (c != this_cpu && (cpu_affinity () & (1u << c)) !=
+             0)
+         {
+             port_smp_ipi (c);
+             break;
+         }
+     }
+ }
+
+ #endif
+
+ #endif
+
+ @@ -709,6 +752,19 @@ namespace os
+     *
+     * @note Can be invoked from Interrupt Service Routines.
+     */
+ #if defined(OS_USE_SMP_SCHEDULER)
+ + uint32_t
+ + thread::cpu_affinity (void) const
+ + {
+ +     return cpu_affinity_;
+ + }
+
+ + void
+ + thread::cpu_affinity (uint32_t mask)
+ + {
+ +     cpu_affinity_ = mask;
+ + }
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     thread::priority_t
+     thread::priority (void)
+
+ @@ -1064,6 +1120,30 @@ namespace os
+         port::scheduler::reschedule ();
+     }
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+ + // On SMP, the joined thread may still be running on another
+ + // CPU
+ + // completing its final reschedule() context switch. Wait until
+ + // it is
+ + // no longer current on any CPU before returning, so the caller
+ + // can
+ + // safely reclaim the thread object and stack memory.
+ + for (;;)
+ + {
+ +     bool still_running = false;
+ +     for (unsigned c = 0; c < OS_NCPU; ++c)
+ +     {
+ +         if (__atomic_load_n (&scheduler::current_thread[c],
+ +                               __ATOMIC_ACQUIRE) == this)
+ +         {
+ +             still_running = true;
+ +             break;
+ +         }
+ +     }
+ +     if (!still_running)
+ +     {
+ +         break;
+ +     }
+ +     this_thread::yield ();
+ + }
+ #endif
+
+ #if defined(OS_TRACE_RTOS_THREAD)
+     trace::printf ("%s() @%p %s joined\n", __func__, this, name
+ ());
+
+ @@ -1384,6 +1464,69 @@ namespace os

```

```

// ----- Enter critical section
-----

scheduler::critical_section scs;

#ifdef OS_USE_SMP_SCHEDULER
+
+ // On SMP the thread may be running on another CPU --
+ typically
+ // still inside its own internal_exit_(), right after the
+ event that
+ // let this caller go on -- or be terminated and already
+ taken off
+ // the funeral list by an idle reaper, which destroys it
+ outside the
+ // lock. Destroying it here in either case frees the stack
+ another
+ // CPU runs on, or destroys it twice. So wait, with the lock
+ // released, until it is off every CPU and unclaimed; from
+ then on
+ // the lock held keeps it so.
+ for (;;)
+ {
+     if (__atomic_load_n (&state_, __ATOMIC_ACQUIRE) ==
state::destroyed)
+     {
+         break;
+     }
+
+     bool busy = (__atomic_load_n (&context_.port_.stack_ptr,
+                                     __ATOMIC_ACQUIRE)
+                 == nullptr);
+     unsigned busy_cpu = OS_NCPU;
+     for (unsigned c = 0; c < OS_NCPU; ++c)
+     {
+         if (__atomic_load_n (&scheduler::current_thread_[c],
+                               __ATOMIC_ACQUIRE)
+             == this)
+         {
+             busy = true;
+             busy_cpu = c;
+             break;
+         }
+     }
+
+     thread::state_t st = __atomic_load_n (&state_,
+__ATOMIC_ACQUIRE);
+     if (st == state::destroying)
+     {
+         busy = true;
+     }
+     else if (st == state::terminated && ready_node_.unlinked
())
+     {
+         // Idle reaper claimed it and will destroy it.
+         busy = true;
+     }
+
+     if (!busy)
+     {
+         break;
+     }
+
+     // Actively trigger reschedule IPI if running on another
core
+     if (busy_cpu < OS_NCPU && busy_cpu != port_cpu_id ())
+     {
+         port_smp_ipi (busy_cpu);
+     }
+
+     {
+         // ----- Enter uncritical section
+         -----

```

```

+         scheduler::uncritical_section succs;
+
+         this_thread::yield ();
+         // ----- Exit uncritical section
+         -----
+     }
+ }
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */

    if (state_ == state::destroyed)
    {
@@ -1845,7 +1988,22 @@ namespace os

    #else

+ #if defined(OS_USE_SMP_SCHEDULER)
+ {
+     // Read the core id and that core's current thread with
+     // this core's
+     // interrupts masked. With them enabled, a preemption
+     // between the
+     // two reads can move the caller to another core, and it
+     // would get
+     // back the thread now running on the core it left -- as a
+     // mutex
+     // owner, a waiter, or the errno it writes. The IRQ
+     // critical section
+     // is the per-core mask every port provides; it also takes
+     // the
+     // kernel lock, recursively, so a caller already inside one
+     // pays
+     // nothing more.
+     interrupts::critical_section ics;
+     th = scheduler::current_thread_[port_cpu_id ()];
+ }
+ #else
+     th = scheduler::current_thread_;
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */

    #endif

    return th;

diff --git a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/
os-c-decls.h b/micro-os-plus-iii-cortexm/include/cmsis-plus/
rtos/port/os-c-decls.h
index 2eacaf3..348df94 100644
--- a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-c-
decls.h
+++ b/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-c-
decls.h
@@ -48,6 +48,13 @@ typedef struct
    os_port_thread_stack_element_t* stack_ptr;
} os_port_thread_context_t;

+ #if defined(OS_USE_SMP_SCHEDULER)
+
+ /* The kernel lock's owner when nobody holds it (os-decls.h,
+    smp_klock_t). */
+ #define SMP_NO_OWNER 0xFFFFFFFFu
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+ #define OS_INTEGER_RTOS_STACK_FILL_MAGIC (0xEFBEADDE)

+ #define OS_HAS_INTERRUPTS_STACK
diff --git a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/
os-decls.h b/micro-os-plus-iii-cortexm/include/cmsis-plus/
rtos/port/os-decls.h
index 8036342..1747acc 100644
--- a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
decls.h
+++ b/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
decls.h

```

```

@@ -116,8 +116,34 @@ namespace os
    constexpr state_t init = unlocked;
    } /* namespace state */

+#if defined(OS_USE_SMP_SCHEDULER)
+
+    // The kernel's SMP scheduler. Both cores declare it here:
+    // upstream's
+    // at OS_NCPU=1 (os-core.cpp), and the RP2350's at OS_NCPU=1
+    // or 2
+    // (os-core-rp2350.cpp, whose include-rp2350/ has no os-
+    // decls.h of its
+    // own). One lock state per CPU, as the SMP kernel indexes
+    // it.
+    extern state_t lock_state[OS_NCPU];
+    extern uint32_t lock_primask[OS_NCPU];
+
+    // The kernel lock, recursive. owner/depth record who holds
+    // it and how
+    // deeply, the bookkeeping the SMP ports keep. What excludes
+    // the other
+    // core is the core's business: with one CPU, masking
+    // interrupts IS the
+    // mutual exclusion, so there is no lock word and no LDREX/
+    // STREX; on
+    // the RP2350 it is SIO hardware spinlock 0 (include-
+    // rp2350/.../os-inlines.h).
+    struct smp_klock_t
+    {
+        volatile uint32_t owner;
+        volatile uint32_t depth;
+    };
+    extern smp_klock_t _smp_klock;
+
+#else
+
+    extern state_t lock_state;

+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+    } /* namespace scheduler */

    using thread_context_t = struct context_s
diff --git a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/
os-inlines.h b/micro-os-plus-iii-cortexm/include/cmsis-plus/
rtos/port/os-inlines.h
index 33a1a69..813d114 100644
--- a/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
inlines.h
+++ b/micro-os-plus-iii-cortexm/include/cmsis-plus/rtos/port/os-
inlines.h
@@ -64,6 +64,9 @@ namespace os
    trace::printf ("0/0+");
    #endif
    trace::printf ("", preemptive);
+#if defined(OS_USE_SMP_SCHEDULER)
+    trace::printf ("", SMP %u CPU", static_cast<unsigned>
(OS_NCPU));
+#endif
    #if (defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)) \
        && defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
        trace::printf ("", BASEPRI(%u)",
@@ -95,6 +98,47 @@ namespace os
    return locked (state::unlocked);
    }

+#if defined(OS_USE_SMP_SCHEDULER)
+
+    // One CPU: it is always CPU 0 (os-core.cpp asserts OS_NCPU
+    // == 1).
+    inline unsigned __attribute__ ((always_inline))

```

```

port_cpu_id (void)
{
    return 0;
}

inline bool __attribute__ ((always_inline))
locked (void)
{
    return lock_state[port_cpu_id ()] != state::unlocked;
}

// Enter/leave the kernel lock. The caller has already masked
// interrupts (critical_section, switch_stacks); with one CPU
that is
// the exclusion, and this only records the owner and the
nesting
// depth. Nothing spins: there is no other CPU to wait for.
inline void __attribute__ ((always_inline))
_smp_klock_enter (void)
{
    _smp_klock.owner = port_cpu_id ();
    _smp_klock.depth = _smp_klock.depth + 1;
}

inline void __attribute__ ((always_inline))
_smp_klock_exit (void)
{
    if (_smp_klock.depth > 0)
    {
        uint32_t d = _smp_klock.depth - 1;
        _smp_klock.depth = d;
        if (d == 0)
        {
            _smp_klock.owner = SMP_NO_OWNER;
        }
    }
}

+ #else

    inline bool __attribute__ ((always_inline))
    locked (void)
@@ -102,6 +146,7 @@ namespace os
        return lock_state != state::unlocked;
    }

+ #endif /* defined(OS_USE_SMP_SCHEDULER) */

    inline void __attribute__ ((always_inline))
    wait_for_interrupt (void)
@@ -232,6 +277,10 @@ namespace os

# endif

+ #if defined(OS_USE_SMP_SCHEDULER)
+     // Interrupts are masked: take the kernel lock.
+     scheduler::smp_klock_enter ();
+ # endif

    // Apparently not required by architecture, but used by
    // FreeRTOS, with an unconvincing motivation ("... ensure
@@ -247,6 +296,10 @@ namespace os
    inline void __attribute__ ((always_inline))
    critical_section::exit (rtos::interrupts::state_t state)
    {
+ #if defined(OS_USE_SMP_SCHEDULER)

```

```

+         // Release the kernel lock while interrupts are still
+         masked.
+         scheduler::_smp_klock_exit ();
+     #endif

    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)

diff --git a/micro-os-plus-iii-cortexm/src/rtos/os-core.cpp b/micro-
os-plus-iii-cortexm/src/rtos/os-core.cpp
index d88d5ec..ad9b65a 100644
--- a/micro-os-plus-iii-cortexm/src/rtos/os-core.cpp
+++ b/micro-os-plus-iii-cortexm/src/rtos/os-core.cpp
@@ -62,12 +62,40 @@ static_assert
    (OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY

    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */

    // The kernel's SMP scheduler on ONE CPU. This core has no second
    core to
    start, no cross-core lock word and no IPI: the SMP branch below
    exists so
    the kernel's SMP code paths (per-CPU current thread and lock
    state, the
    affinity-aware picker, the stack_ptr liveness rule) run on a
    single Cortex-M.
    More CPUs need a board that supplies a lock and an IPI (see
    CMakeLists.txt).
    #if defined(OS_USE_SMP_SCHEDULER)
    static_assert (OS_NCPU == 1,
    +         "the upstream Cortex-M core runs the SMP scheduler on
    +         1 CPU "
    +         "only");
    #endif /* defined(OS_USE_SMP_SCHEDULER) */
    +
    // -----

    #if defined(__ARM_ARCH_6M__)
    extern uint32_t __vectors_start;
    #endif

    #if defined(OS_USE_SMP_SCHEDULER)
    // The kernel's idle thread, registered as CPU 0's idle in start().
    extern os::rtos::thread* os_idle_thread;
    +
    namespace os
    +{
    +    namespace rtos
    +    {
    +        namespace scheduler
    +        {
    +            // The kernel's per-CPU idle threads (kernel os-core.cpp).
    +            extern thread* os_idle_thread_core[OS_NCPU];
    +        } /* namespace scheduler */
    +    } /* namespace rtos */
    +} /* namespace os */
    #endif /* defined(OS_USE_SMP_SCHEDULER) */
    +
    // -----

    extern "C"
    @@ -358,7 +386,12 @@ namespace os

        namespace scheduler
        {
    #if defined(OS_USE_SMP_SCHEDULER)
    +        state_t lock_state[OS_NCPU];
    +        smp_klock_t _smp_klock = { SMP_NO_OWNER, 0 };
    #else
    +        state_t lock_state;
    #endif /* defined(OS_USE_SMP_SCHEDULER) */

```

```

        result_t
        initialize (void)
@@ -494,7 +527,15 @@ namespace os
        rtos::thread* pth = reinterpret_cast<rtos::thread*>
        (&fake_thread);

        // Make the fake thread look like the current thread.
+#if defined(OS_USE_SMP_SCHEDULER)
+        rtos::scheduler::current_thread_[0] = pth;
+
+        // The picker falls back to the CPU's idle thread; CPU 0's
+        is the
+        // kernel's.
+        rtos::scheduler::os_idle_thread_core[0] = ::os_idle_thread;
+#else
        rtos::scheduler::current_thread_ = pth;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

        // Trigger the PendSV; the exception will happen a bit
        later,
        // after re-enabling the interrupts.
@@ -507,7 +548,11 @@ namespace os

        #endif

+#if defined(OS_USE_SMP_SCHEDULER)
+        lock_state[0] = state::init;
+#else
        lock_state = state::init;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

        // Enable all interrupts; allow PendSV to occur.
        __enable_irq ();
@@ -531,8 +576,17 @@ namespace os
        {
            rtos::interrupts::critical_section ics;

+#if defined(OS_USE_SMP_SCHEDULER)
+            // With one CPU the scheduler lock stays what it is
+            upstream: a
+            // flag the picker reads. It holds no interrupt mask; no
+            other
+            // core can run the picker meanwhile.
+            unsigned cpu = port_cpu_id ();
+            tmp = lock_state[cpu];
+            lock_state[cpu] = state;
+#else
            tmp = lock_state;
            lock_state = state;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
        }

        return tmp;
@@ -830,6 +884,44 @@ namespace os
        SCB->ICSR = SCB_ICSR_PENDSVCLR_Msk;
        #pragma GCC diagnostic pop

+#if defined(OS_USE_SMP_SCHEDULER)
+
+        // Interrupts are masked: take the kernel lock for the
+        picker.
+        __smp_klock_enter ();
+
+        unsigned cpu = port_cpu_id ();
+        rtos::thread* old_thread =
+        rtos::scheduler::current_thread_[cpu];
+
+

```

```

+         // Save the current SP in the initial context. This also
+         publishes
+         // the outgoing thread as switchable again: the SMP picker
+         skips a
+         // thread whose stack_ptr is null (its context is live on a
+         CPU).
+         old_thread->context_.port_.stack_ptr = sp;
+
+         rtos::scheduler::internal_switch_threads ();
+
+         rtos::thread* new_thread =
+         rtos::scheduler::current_thread_[cpu];
+
+         if (new_thread == nullptr)
+         {
+             // No ready thread and no idle thread -- fatal. Release
+             the
+             // kernel lock first so the picker cannot be entered by
+             anything
+             // else, then halt with interrupts masked (do not
+             restore the
+             // saved mask: the scheduler is already torn).
+             _smp_klock_exit ();
+             __asm__ volatile ("cpsid if" ::: "memory");
+             for (;;)
+             __asm__ volatile ("wfi");
+         }
+
+         // Prepare a local copy of the new thread SP, and mark its
+         context
+         // live on this CPU (the same thread, when it was re-
+         picked).
+         stack::element_t* out_sp = new_thread-
+         >context_.port_.stack_ptr;
+         new_thread->context_.port_.stack_ptr = nullptr;
+
+         _smp_klock_exit ();
+
+     +#else
+
+         rtos::thread* old_thread =
+         rtos::scheduler::current_thread_;
+
+         // Save the current SP in the initial context.
+@@ -851,6 +943,8 @@ namespace os
+         stack::element_t* out_sp
+         = rtos::scheduler::current_thread_-
+         >context_.port_.stack_ptr;
+
+     +#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     +
+     +#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+
+     +#if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+@@ -886,6 +980,17 @@ namespace os
+
+     +// -----
+
+     +#if defined(OS_USE_SMP_SCHEDULER)
+     +// The kernel's CPU index (declared extern "C" by the kernel).
+     +extern "C" unsigned
+     +port_cpu_id (void)
+     +{
+     +    return os::rtos::port::scheduler::port_cpu_id ();
+     +}
+     +#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     +
+     +// -----
+
+     +
+     +using namespace os::rtos;

```

```

/**
diff --git a/micro-os-plus-iii-posix-arch/CMakeLists.txt b/micro-os-
plus-iii-posix-arch/CMakeLists.txt
index ae85397..ba48588 100644
--- a/micro-os-plus-iii-posix-arch/CMakeLists.txt
+++ b/micro-os-plus-iii-posix-arch/CMakeLists.txt
@@ -49,6 +49,9 @@ target_include_directories(micro-os-plus-iii-posix-
arch-interface INTERFACE
    target_sources(micro-os-plus-iii-posix-arch-interface INTERFACE
        src/diag/trace-posix.cpp
        src/rtos/os-core.cpp
+   src/host_cpu.cpp
+   src/free-store.cpp
+   src/exception_handler.cpp
    )

    target_compile_definitions(micro-os-plus-iii-posix-arch-interface
INTERFACE
diff --git a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/
port/os-c-decls.h b/micro-os-plus-iii-posix-arch/include/
cmsis-plus/rtos/port/os-c-decls.h
index c989c07..a274f67 100644
--- a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-c-
decls.h
+++ b/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-c-
decls.h
@@ -1,29 +1,15 @@
/*
+ * os-c-decls.h - the width-dependent half of the POSIX port
contract.
+ *
+ * This file is part of the µOS++ project (https://micro-os-
plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
license.
+ *
- * If a copy of the license was not distributed with this file, it
can
- * be obtained from https://opensource.org/licenses/mit/.
- */
-
-/*
- * The initial CMSIS++ RTOS API was inspired by CMSIS RTOS API v1.x,
- * Copyright (c) 2013 ARM LIMITED
- */
-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * It is included in `cmsis-plus/rtos/os-c-decls.h` to customise
- * it with port specific declarations.
- *
- * These structures (which basically contain handlers)
- * are conditionally included in the system objects
- * when they are implemented using the port native objects.
+ * Derived from upstream micro-os-plus-iii-posix-arch v1.0.1. What
changed and
+ * why is in os-decls.h, beside the declarations it changed.
*/

#ifndef CMSIS_PLUS_RTOS_PORT_OS_C_DECLS_H
@@ -69,17 +55,39 @@ typedef uint32_t os_port_clock_duration_t;
// Must match port::clock::offset_t
typedef uint64_t os_port_clock_offset_t;

```

```

+typedef uint64_t os_port_thread_stack_element_t;
+typedef uint64_t os_port_thread_stack_allocation_element_t;
+
+/*
+ * The thread context.
+ *
+ * `stack_ptr` FIRST and `ucontext` second, and the order is not
+ * cosmetic.
+ *
+ * Upstream's context was the ucontext alone, because upstream had
+ * one CPU and
+ * therefore never had to ask whether a saved context was safe to
+ * resume. The
+ * SMP scheduler does ask, in the kernel, in a line no port may edit:
+ *
+ * src/rtos/os-core.cpp, internal_switch_threads():
+ *     && (th == old_thread || th->context_.port_.stack_ptr !=
+ *         nullptr)
+ *
+ * On the ARM ports that field is the outgoing stack pointer,
+ * published by the
+ * assembly restore path only once the CPU has left the outgoing
+ * thread's
+ * stack. Here it is the same gate carrying the same meaning -- "this
+ * context
+ * is fully saved, another CPU may claim it" -- and nothing more: the
+ * register
+ * state lives in the ucontext. A CPU clears it when it claims a
+ * thread and
+ * the NEXT context to run on that CPU publishes it, which is the
+ * whole of the
+ * deferred-publish rule. See host_cpu.cpp.
+ */
+typedef struct
+{
+    - os_impl_ucontext_t ucontext; //
+    + os_port_thread_stack_element_t* stack_ptr;
+    + os_impl_ucontext_t ucontext;
+} os_port_thread_context_t;

+typedef bool os_port_scheduler_state_t;

+// Signal set (true if signal blocked)
+/* True when this CPU's interrupt signals are blocked. A per-thread
+   signal
+ * mask IS per-CPU interrupt masking, because a host thread IS a CPU.
+ */
+typedef bool os_port_irq_state_t;

+typedef uint64_t os_port_thread_stack_element_t;
+typedef uint64_t os_port_thread_stack_allocation_element_t;
+
+
+#endif /* CMSIS_PLUS_RTOS_PORT_OS_C_DECLS_H */
diff --git a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/
      port/os-decls.h b/micro-os-plus-iii-posix-arch/include/cmsis-
      plus/rtos/port/os-decls.h
index 4ee273a..825f285 100644
--- a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
      decls.h
+++ b/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
      decls.h
@@ -1,51 +1,56 @@
+/*
+ * os-decls.h - the C++ half of the POSIX port contract.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ */

```

```

- * If a copy of the license was not distributed with this file, it
  can
- * be obtained from https://opensource.org/licenses/mit/.
- */
-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * It is included in `cmsis-plus/rtos/os.h` to customise
- * it with POSIX specific declarations.
+ * This port does NOT link micro-os-plus::port-smp-decls, and that
  target's
+ * own README says why: the shared copy is written for ports whose
  state types
+ * are integers and whose thread context is a bare stack pointer.
  Neither is
+ * true here -- the interrupt state is a signal-mask bit, the
  scheduler state
+ * is a bool, and the context carries a ucontext. The SMP
  declarations the
+ * kernel actually reads (lock_state[], _smp_klock,
  _port_ctx_pending[]) are
+ * reproduced below with the same names, the same types and the same
+ * volatility, so the kernel cannot tell the two apart.
  */

#ifndef CMSIS_PLUS_RTOS_PORT_OS_DECLS_H_
#define CMSIS_PLUS_RTOS_PORT_OS_DECLS_H_

-// -----
-
#if defined(OS_USE_OS_APP_CONFIG_H)
#include <cmsis-plus/os-app-config.h>
#endif

#include <cmsis-plus/rtos/port/os-c-decls.h>

-// -----
+#if !defined(OS_NCPU)
+#define OS_NCPU (1)
+#endif

#if !defined(OS_INTEGER_SYSTICK_FREQUENCY_HZ)
#define OS_INTEGER_SYSTICK_FREQUENCY_HZ (1000)
#endif

+/*
+ * Host stacks are large because a signal frame lands on them.
+ *
+ * Preemption here is a signal delivered on the running thread's own
  stack --
+ * deliberately, with no SA_ONSTACK, so that the whole signal frame
  migrates
+ * with the thread when another CPU resumes it (see host_cpu.cpp).
  That frame
+ * is several kilobytes of ucontext plus the FPU state, so upstream's
  32 KiB
+ * minimum is kept rather than the 2 KiB the ARM ports use.
+ */
#if !defined(OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)
#define OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES (32 * 1024)
#endif

#if !defined(OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES)
-#if defined(__linux__)
#define OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES \
  (2 * OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)
-#else
-#define OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES \
-  (OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)

```

```

-#endif /* defined(__linux__) */
#endif

#if !defined(OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES)
@@ -58,59 +63,54 @@
(OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES)
#endif

-// -----
+#if !defined(OS_INTEGER_RTOS_STACK_FILL_MAGIC)
+#define OS_INTEGER_RTOS_STACK_FILL_MAGIC (0xEFBEADDEEFBEADDEULL)
+#endif

-#ifdef __cplusplus
+/* Kept identical to the ARM ports so the same code reads on both. */
+#if !defined(SMP_NO_OWNER)
+#define SMP_NO_OWNER (0xFFFFFFFFu)
+#endif

-// -----
+#ifdef __cplusplus

#include <signal.h>
-// Platform definitions
#include <sys/time.h>

#include <cstdint>
#include <cstddef>

-// -----
-
#pragma GCC diagnostic push

#if defined(__clang__)
#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wreserved-identifier"
#endif

-// -----
-
namespace os
{
    namespace rtos
    {
        namespace port
        {
-            // -----

namespace stack
{
-            // Assume 64-bits core.
using element_t = os_port_thread_stack_element_t;

-            // Align stack to 8 bytes.
using allocation_element_t =
os_port_thread_stack_allocation_element_t;

-            // Initial value for the minimum stack size in bytes.
constexpr std::size_t min_size_bytes
= OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES;

-            // Initial value for the default stack size in bytes.
constexpr std::size_t default_size_bytes
= OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES;

```

```

-     constexpr element_t magic = 0xEFBEADDEEFBEADDE;
+     constexpr element_t magic = OS_INTEGER_RTOS_STACK_FILL_MAGIC;
} /* namespace stack */

namespace interrupts
{
-     // True if signal blocked
    using state_t = os_port_irq_state_t;

    namespace state
@@ -118,8 +118,19 @@ namespace os
        constexpr state_t init = false;
    } /* namespace state */

-     extern sigset_t clock_set;
+     /* The signals this port treats as interrupts: the per-CPU
+     tick and
+     * the IPI. Blocking them IS masking interrupts on this CPU.
+     */
+     extern sigset_t irq_set;
+
+     /* Per-CPU handler-mode flag. AArch64 has the same array for
+     the same
+     * reason: there are no banked modes to ask. */
+     extern "C" volatile bool _in_isr[OS_NCPU];

+     /* The Arm CMSIS RTOS validator's host shim raises its
+     "interrupt" with
+     * kill(SIGUSR1) and increments this around the handler. The
+     port has
+     * no banked mode to ask either, so a non-zero value also
+     means handler
+     * mode (see in_handler_mode()). Defined in src/rtos/os-
+     core.cpp. */
+     extern "C" volatile uint32_t signal_nesting;
} /* namespace interrupts */

namespace scheduler
@@ -133,23 +144,46 @@ namespace os
    constexpr state_t init = unlocked;
} /* namespace state */

-     extern state_t lock_state;
+     extern volatile state_t lock_state[OS_NCPU];

+     /* Byte-for-byte the ARM ports' recursive kernel lock (lock
+     word,
+     * owner CPU, nesting depth). The ARM timer leaf lock
+     (smp_tlock_t,
+     * port_tmr_lock/unlock) is deliberately NOT carried here: it
+     exists
+     * for a kernel timer back-end that takes it, and this port's
+     timers
+     * are POSIX timers driven through host_cpu, so it would be
+     dead. */
+     struct smp_klock_t
+     {
+         volatile uint32_t lock;
+         volatile uint32_t owner;
+         volatile uint32_t depth;
+     };

+     extern smp_klock_t _smp_klock;
+     extern volatile unsigned _port_ctx_pending[OS_NCPU];
} /* namespace scheduler */

namespace clock
{
-     constexpr unsigned int signal_number = SIGALRM;
- } /* namespace clock */
+     /* Not SIGALRM. ITIMER_REAL and SIGALRM are process-wide;
+     this port

```

```

+     * gives every CPU its own timer_create() timer, and a real-
+     time
+     * signal because those queue rather than coalesce.
+     *
+     * Functions, not constants: glibc's SIGRTMIN expands to a
+     call to
+     * __libc_current_sigrtmin(), so it is not a constant
+     expression. */
+     inline int
+     signal_number (void)
+     {
+         return SIGRTMIN;
+     }

-     using thread_context_t = struct thread_context_s
-     {
-         // On POSIX, the context is saved on standard (although
-         deprecated)
-         // ucontext_t structures. It requires _XOPEN_SOURCE=700L to
-         compile.
-         os_impl_ucontext_t ucontext; //
-     };
+     inline int
+     ipi_signal_number (void)
+     {
+         return SIGRTMIN + 1;
+     }
+ } /* namespace clock */

-     //
+     using thread_context_t = os_port_thread_context_t;

        } /* namespace port */
    } /* namespace rtos */
@@ -157,10 +191,6 @@ namespace os

#pragma GCC diagnostic pop

-// -----
-
#endif /* __cplusplus */

-// -----
-
#endif /* CMSIS_PLUS_RTOS_PORT_OS_DECLS_H */
diff --git a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/
port/os-inlines.h b/micro-os-plus-iii-posix-arch/include/
cmsis-plus/rtos/port/os-inlines.h
index 0ec7ab7..e9e0e69 100644
--- a/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
inlines.h
+++ b/micro-os-plus-iii-posix-arch/include/cmsis-plus/rtos/port/os-
inlines.h
@@ -1,63 +1,56 @@
/*
+ * os-inlines.h - the hot half of the POSIX port: masking and the
+ kernel lock.
+ *
+ * This file is part of the µOS++ project (https://micro-os-
+ plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ license.
+ *
+ * If a copy of the license was not distributed with this file, it
+ can
+ be obtained from https://opensource.org/licenses/mit/.
+ */

```

```

-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * If contains the scheduler implementation that uses
- * functions from the POSIX API (macOS and GNU/Linux).
+ * Read this beside micro-os-plus-iii-aarch64/include/cmsis-plus/
+ * rtos/port/
+ * os-inlines.h. The two files have the same shape function for
+ * function; only
+ * the mechanism differs, and that is the whole point of the port
+ * seam:
+ *
- * This file is included in all src/os-*.cpp files.
+ * concern      AArch64      here
+ * -----
+ * CPU id        MRS MPIDR_EL1, & 3      thread_local, set at
+ * CPU start
+ * IRQ mask      MSR DAIFSET/DAIFCLR, #2
+ * pthread_sigmask(irq_set)
+ * IRQ state     MRS DAIF      sigismember(old, tick)
+ * atomics        LDAXR/STLXR + DMB ISH    __atomic_* , seq_cst
+ * in-handler     _in_isr[]      _in_isr[] (identical)
+ */

#ifndef CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_
#define CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_

-// -----
-
-#if defined(OS_USE_OS_APP_CONFIG_H)
-#include <cmsis-plus/os-app-config.h>
-#endif

-#include <cmsis-plus/rtos/os-c-decls.h>

-// -----
-
-#ifdef __cplusplus

-// -----
-
-#include <stdlib.h>
-#include <string.h>
-#include <sys/utsname.h>
-#include <sys/time.h>
-
-// For Linux
-#include <pthread.h>
-#include <signal.h>
-#include <unistd.h>
-#include <time.h>

-#include <cmsis-plus/diag/trace.h>

-// -----
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wunused-parameter"

-#if defined(__clang__)
-#pragma clang diagnostic ignored "-Wc++98-compat"
-#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
-#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
-#endif

-// -----

```

```

-
-extern "C" uint32_t signal_nesting;
+// Out of line on purpose; see _this_cpu below.
+extern "C" unsigned
+port_cpu_id (void);

namespace os
{
@@ -65,10 +58,44 @@ namespace os
{
    namespace port
    {
-        //
+        /*
+         * Which CPU is running this code.
+         *
+         * This is the one deliberate use of native thread-local
+         * storage in the
+         * whole port, and it is the one use that is correct by
+         * construction: a
+         * host thread IS a CPU, so storage private to a host thread is
+         * storage
+         * private to a CPU. It answers "where am I", never "what was I
+         * doing".
+         *
+         * Everything else must obey the opposite rule. A µOS++ thread
+         * migrates
+         * between CPUs, so `errno` and any `thread_local` it touches
+         * belong to
+         * whichever host thread happens to be running it at that
+         * instant. No
+         * port or application state may live there across a switch
+         * point --
+         * read errno only inside the critical section that made the
+         * call.
+         *
+         * That rule covers the ADDRESS of `_this_cpu` too, and the
+         * compiler does
+         * not know it. The thread pointer is constant for the life of
+         * a host
+         * thread, so clang computes `&_this_cpu` once per function and
+         * keeps it
+         * in a callee-saved register. A function that blocks --
+         * semaphore::
+         * wait() loops around reschedule() -- then resumes on another
+         * host
+         * thread and reads the CPU id of the one it left. It takes the
+         * kernel
+         * lock as that CPU, skips the acquire when that CPU already
+         * owns it,
+         * and two CPUs run inside the lock; the depth count strands it
+         * held and
+         * every CPU spins for ever (clang 16/17 -O2: smp_test3, smp-
+         * pipeline).
+         * GCC reloads the thread pointer at each access and never
+         * showed it.
+         *
+         * So the read is never inlined: each call of port_cpu_id()
+         * computes the
+         * address afresh, and the empty asm makes every call a side
+         * effect that
+         * can be neither merged with another nor hoisted, LTO
+         * included.
+         */
+extern thread_local unsigned _this_cpu;

namespace scheduler
{
+    inline unsigned __attribute__((always_inline))
+    port_cpu_id_inline (void)
+    {
+        return ::port_cpu_id ();
+    }

    inline port::scheduler::state_t __attribute__((always_inline))

```

```

lock (void)
@@ -85,27 +112,54 @@ namespace os
    inline bool __attribute__ ((always_inline))
    locked (void)
    {
-       return lock_state != state::unlocked;
+       return lock_state[port_cpu_id_inline ()] !=
state::unlocked;
    }

+
+    /* SMP: acquire the scheduler lock word.
+    *
+    * A spin, exactly as on ARM, and for the same reason: this
lock is
+    * held for a handful of instructions and is taken from
inside signal
+    * handlers, where a pthread_mutex would be a function call
into code
+    * that is not async-signal-safe. The pause hint keeps a
spinning CPU
+    * from starving the one that holds it when the host
oversubscribes. */
    inline void __attribute__ ((always_inline))
    wait_for_interrupt (void)
+    _smp_klock_raw_acquire (void)
    {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-    trace::printf ("%s() \n", __func__);
+    while (__atomic_exchange_n (&_smp_klock.lock, 1u,
__ATOMIC_ACQUIRE)
+        != 0u)
+    {
+#if defined(__x86_64__) || defined(__i386__)
+        __builtin_ia32_pause ();
+
+#elif defined(__aarch64__) || defined(__arm__)
+        __asm__ volatile("yield" ::: "memory");
+
+endif
+        pause ();
+    }
+
+    inline void __attribute__ ((always_inline))
+    _smp_klock_raw_release (void)
+    {
+        __atomic_store_n (&_smp_klock.lock, 0u, __ATOMIC_RELEASE);
+    }

+
+    inline unsigned __attribute__ ((always_inline))
port_smp_depth (void)
    {
        return _smp_klock.depth;
    }

+
+    void
wait_for_interrupt (void);
+
+} /* namespace scheduler */

namespace interrupts
{
-
    inline bool __attribute__ ((always_inline))
in_handler_mode (void)
    {
-        return (signal_nesting > 0);
+        const unsigned cpu = scheduler::port_cpu_id_inline ();
+        return ((cpu < OS_NCPU) && _in_isr[cpu]) ||
(signal_nesting != 0);
    }
}

```

```

        inline bool __attribute__ ((always_inline))
@@ -114,9 +168,71 @@ namespace os
        return true;
    }

-    } /* namespace interrupts */
+    /*
+     * Enter an IRQ critical section: mask this CPU's interrupts
+     * and take
+     * the kernel lock. Same two steps, same order, as AArch64's.
+     *
+     * pthread_sigmask, not sigprocmask. In a multithreaded
+     * process the
+     * behaviour of sigprocmask is unspecified (POSIX.1-2017, and
+     * glibc
+     * documents it as such), and upstream's port used it
+     * throughout
+     * because upstream had exactly one thread. The replacement
+     * is not a
+     * workaround but the better abstraction: a per-thread signal
+     * mask is
+     * per-CPU interrupt masking, which is what a critical
+     * section has
+     * always meant.
+     */
+    inline rtos::interrupts::state_t __attribute__
+    ((always_inline))
+    critical_section::enter (void)
+    {
+        sigset_t old;
+        ::pthread_sigmask (SIG_BLOCK, &irq_set, &old);
+
+        const unsigned cpu = scheduler::port_cpu_id_inline ();
+        if (scheduler::_smp_klock.owner != cpu)
+        {
+            scheduler::_smp_klock_raw_acquire ();
+            scheduler::_smp_klock.owner = cpu;
+        }
+        scheduler::_smp_klock.depth = scheduler::_smp_klock.depth +
1;
+
+        return ::sigismember (&old, clock::signal_number ()) != 0;
+    }
+
+    inline void __attribute__ ((always_inline))
+    critical_section::exit (rtos::interrupts::state_t state)
+    {
+        const unsigned cpu = scheduler::port_cpu_id_inline ();
+        if (scheduler::_smp_klock.owner == cpu
+            && scheduler::_smp_klock.depth > 0)
+        {
+            const uint32_t d = scheduler::_smp_klock.depth - 1;
+            scheduler::_smp_klock.depth = d;
+            if (d == 0)
+            {
+                scheduler::_smp_klock.owner = SMP_NO_OWNER;
+                scheduler::_smp_klock_raw_release ();
+            }
+        }
+
+        ::pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK,
+&irq_set,
+
+                            nullptr);
+    }
+
+    inline rtos::interrupts::state_t __attribute__
+    ((always_inline))
+    uncritical_section::enter (void)
+    {
+        sigset_t old;

```

```

+         ::pthread_sigmask (SIG_UNBLOCK, &irq_set, &old);
+         return ::sigismember (&old, clock::signal_number ()) != 0;
+     }
+
+     inline void __attribute__ ((always_inline))
+     uncritical_section::exit (rtos::interrupts::state_t state)
+     {
+         ::pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK,
+                             &irq_set,
+
+                             nullptr);
+     }
+
+ //
+ // =====
+ } /* namespace interrupts */
+
+ namespace this_thread
+ {
+@@ -124,11 +240,8 @@ namespace os
+     prepare_suspend (void)
+     {
+     }
+
+ } /* namespace this_thread */
+
+ //
+ // =====
+
+ inline constexpr bool __attribute__ ((always_inline))
+ clock_highres::has_hardware_counter (void) noexcept
+ {
+@@ -138,7 +251,7 @@ namespace os
+     inline uint64_t __attribute__ ((always_inline))
+     clock_highres::hardware_counter (void) noexcept
+     {
+         timespec tp; // NB: no 'struct' - baseline gate has
+         - Werror=redundant-tags
+         + timespec tp;
+         ::clock_gettime (CLOCK_MONOTONIC, &tp);
+         return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
+             + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
+@@ -150,10 +263,6 @@ namespace os
+
+ #pragma GCC diagnostic pop
+
+ -// -----
+ -
+ #endif /* __cplusplus */
+
+ -// -----
+ -
+ #endif /* CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_ */
+ diff --git a/micro-os-plus-iii-posix-arch/include/
+         exception_handler.hpp b/micro-os-plus-iii-posix-arch/include/
+         exception_handler.hpp
+ new file mode 100644
+ index 0000000..27423a1
+ --- /dev/null
+ +++ b/micro-os-plus-iii-posix-arch/include/exception_handler.hpp
+@@ -0,0 +1,29 @@
+ +/**
+ + * @file exception_handler.hpp
+ + * @brief Synchronous-fault reporting for the POSIX port.
+ + *
+ + * On the ARM ports a synchronous exception lands in a vector
+         installed by
+ + * startup.S and is reported by port_fatal_exception(). The host
+         equivalent of
+ + * a synchronous exception is a signal -- SIGSEGV, SIGBUS, SIGFPE,
+         SIGILL --

```

```

+ * so exception::init() installs handlers for those and reports
+ * through the
+ * same function, with the same name, so a test that faults says so
+ * instead of
+ * dying silently with no output.
+ *
+ * This matters more here than on the silicon, not less: a host
+ * process that
+ * segfaults under a test runner produces an exit status and nothing
+ * else, and
+ * with several CPUs running there is no way afterwards to tell which
+ * one
+ * faulted or in which thread.
+ */
+ #pragma once
+
+ #include <stdint>
+
+ namespace exception {
+
+ void init();
+
+ } // namespace exception
+
+ extern "C" void port_fatal_exception(std::uint64_t type,
+                                     std::uint64_t esr,
+                                     std::uint64_t elr, std::uint64_t
+                                     far,
+                                     std::uint64_t spsr);
diff --git a/micro-os-plus-iii-posix-arch/include/host_cpu.hpp b/
micro-os-plus-iii-posix-arch/include/host_cpu.hpp
new file mode 100644
index 0000000..55861b6
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/include/host_cpu.hpp
@@ -0,0 +1,129 @@
+ /*
+ * host_cpu.hpp - the CPU model of the POSIX port: a host thread IS a
+ * CPU.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ *
+ * On the ARM ports the equivalent of this header is split between
+ * the silicon
+ * (the generic timer, the GIC/mailbox IPI, MPIDR) and the board (the
+ * spin
+ * table that releases the secondary cores). Here the silicon is the
+ * host
+ * kernel, so it is all one file, and it is port code rather than
+ * board code.
+ */
+
+ #ifndef MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_
+ #define MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_
+
+ #include <cstdint>
+
+ #include <cmsis-plus/rtos/os.h>
+ #include <cmsis-plus/rtos/port/os-decls.h>
+
+ namespace host_cpu
+ {
+     using element_t = os::rtos::port::stack::element_t;
+
+     /*
+      * "Which thread is running on this CPU."
+      */

```

```

+ * The kernel spells this two different ways: under the SMP
+   scheduler
+ * `scheduler::current_thread_` is an array indexed by CPU, and
+   without it
+ * a single pointer (os-sched.h, guarded on OS_USE_SMP_SCHEDULER,
+   which
+ * uos_add_app() derives from NCPU > 1).
+ *
+ * One accessor keeps the difference in one place, so the rest of
+   the port
+ * is written once and serves both. This is the same dual-branch
+   arrangement
+ * the cortexm port uses to run its three STM32 boards at OS_NCPU=1
+   off the
+ * RP2350's SMP core.
+ */
+ inline os::rtos::thread* volatile&
+   current_thread (unsigned cpu)
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+   return os::rtos::scheduler::current_thread_[cpu];
+ #else
+   (void)cpu;
+   return os::rtos::scheduler::current_thread_;
+ #endif
+ }
+
+ /* Install the tick and IPI signal handlers, process-wide. Called
+   once,
+ * from port::scheduler::initialize(). */
+ void
+ install_handlers (void);
+
+ /* Give the calling CPU an alternate stack for fault reporting. Per
+   host
+ * thread, because sigaltstack() is, and that is correct here:
+   reporting a
+ * fault does not migrate, so it may use storage the CPU owns. */
+ void
+ install_fault_stack (void);
+
+ /* Arm the calling CPU's own periodic timer, at
+ * OS_INTEGER_SYSTICK_FREQUENCY_HZ. Every CPU has one; only CPU 0
+   advances
+ * the kernel clock with it. */
+ void
+ start_this_cpu_tick (void);
+
+ /* Create the OS_NCPU-1 secondary host threads. Each sets its own
+   CPU index,
+ * arms its own tick and enters the scheduler, which is what a
+   secondary
+ * core does after the spin table releases it. */
+ void
+ start_secondary_cpus (void);
+
+ /* Ask another CPU to re-pick. pthread_kill() of the IPI signal,
+   which is
+ * this port's SGI / mailbox doorbell. */
+ void
+ send_ipi (unsigned cpu);
+
+ /* Build a never-run context's entry point. Wraps the kernel's
+   thread body
+ * in the trampoline that discharges this CPU's deferred publish
+   before the
+ * body runs -- a context that has never run still arrives on a CPU
+   that
+ * has just left another thread behind. */
+ void
+ make_entry (os_impl_ucontext_t* ctx, void* func, void* args);
+

```

```

+ /* Leave `*addr = val` for whoever runs next on this CPU. See the
+   long
+   * comment on port::scheduler::switch_stacks(). */
+ void
+ defer_publish (unsigned cpu, element_t** addr, element_t* val);
+
+ /* Discharge this CPU's deferred publish, if it has one. Called on
+   every
+   * arrival into a context: after swapcontext(), and in the
+   trampoline. */
+ void
+ publish_pending (void);
+
+ /* -----
+   * AddressSanitizer and the context switch.
+   *
+   * ASan keeps a shadow record of which stack is live so that it can
+   tell a
+   * genuine overflow from an ordinary function return. swapcontext()
+   moves
+   * the stack pointer to somewhere ASan has never seen, and without
+   being
+   * told, ASan reports the first thing the resumed thread touches as
+   a
+   * stack-buffer-overflow -- a false positive on every single
+   switch, which
+   * makes the tool useless rather than merely noisy.
+   *
+   * The fix is ASan's own fiber interface, which exists for exactly
+   this:
+   *
+   *   start_switch (&save, bottom, size)   before swapcontext /
+   setcontext
+   *   finish_switch (save)                  first thing on arrival
+   *
+   * `save` is where ASan parks the outgoing fibre's "fake stack"
+   (its
+   * use-after-return bookkeeping). It must survive until that
+   context is
+   * resumed, so switch_stacks() keeps it in a local -- which lives
+   on the
+   * outgoing thread's own stack and is therefore still there, and
+   still
+   * correct, whenever and on whichever CPU that thread comes back.
+   *
+   * Both compile to nothing when ASan is off, so the calls stay
+   unguarded at
+   * the call sites and the switch code reads the same either way.
+   * ----- */
+ void
+ asan_start_switch (void** save, const void* bottom, std::size_t
+   size);
+
+ void
+ asan_finish_switch (void* save);
+
+} /* namespace host_cpu */
+
+#endif /* MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_ */
diff --git a/micro-os-plus-iii-posix-arch/include/hw_result.hpp b/
micro-os-plus-iii-posix-arch/include/hw_result.hpp
new file mode 100644
index 00000000..2102b90
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/include/hw_result.hpp
@@ -0,0 +1,53 @@
+/**
+ * @file hw_result.hpp
+ * @brief Bounded "done" helper, POSIX host.
+ *
+ * The sibling of micro-os-plus-iii-aarch64/include/hw_result.hpp. A
+ test
+ * prints its RESULT line and then calls ok()/fail().
+ *

```

```

+ * On the silicon ports these compile to a semihosted SYS_EXIT under
+ * SEMIHOST
+ * and to nothing otherwise, so a plain image keeps its idle-forever
+ * behaviour. Here there is no "otherwise": the host always has an
+ * exit
+ * status, and a test process that idled for ever would have to be
+ * killed by
+ * the runner and could not report anything. So they always end the
+ * run, and
+ * the verdict reaches the shell as the exit code.
+ *
+ * _exit(), not exit(). Other CPUs are still running threads; exit()
+ * would run
+ * static destructors underneath them and turn a clean PASS into a
+ * crash in
+ * the teardown.
+ *
+ * But _exit() does not flush stdio either, so these flush first.
+ * Most tests
+ * here print through uart::uart1, which is write(2) and therefore
+ * already on
+ * its way out; a test carried from upstream's own suite prints with
+ * printf(),
+ * and to a pipe -- which is how the runner captures it -- stdout is
+ * fully
+ * buffered. Without the flush its entire output, verdict included,
+ * is
+ * discarded at the exit. (Found exactly that way: mutex-stress ran,
+ * passed and
+ * printed nothing.)
+ *
+ * Pattern in a test:
+ *
+ *     uart1 << "\nRESULT: " << (ok ? "PASS" : "FAIL") << "\n";
+ *     if (ok) hw_result::ok (); else hw_result::fail ();
+ *     for (;;) sysclock.sleep_for (1000);    // never reached
+ */
+
+ #pragma once
+
+ #include <cstdio>
+ #include <unistd.h>
+
+ namespace hw_result
+ {
+     [[noreturn]] inline void
+     ok () noexcept
+     {
+         std::fflush (nullptr);
+         ::_exit (0);
+     }
+
+     [[noreturn]] inline void
+     fail () noexcept
+     {
+         std::fflush (nullptr);
+         ::_exit (1);
+     }
+ } // namespace hw_result
diff --git a/micro-os-plus-iii-posix-arch/src/board-contract.cpp b/
micro-os-plus-iii-posix-arch/src/board-contract.cpp
new file mode 100644
index 0000000..40805d4
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/src/board-contract.cpp
@@ -0,0 +1,43 @@
+/*
+ * board-contract.cpp - no code. It fails the build the moment a
+ * board has
+ *
+ * not stated something the shared sources need.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.

```

```

+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ *
+ * The same file exists in the three cross-compiled projects and
+ * exists here
+ * for the same reason: a board that leaves something unset must
+ * produce a
+ * compile error naming the board and the thing, not a silent
+ * inheritance of
+ * another board's answer -- or, worse, a link that succeeds and a
+ * test that
+ * proves nothing.
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+#include <cmsis-plus/rtos/port/os-decls.h>
+
+#if !defined(OS_NCPU)
+#error "The board does not set OS_NCPU."
+#endif
+
+#if OS_NCPU < 1
+#error "OS_NCPU must be at least 1."
+#endif
+
+#if !defined(PORT_GREETING)
+#error "The board does not define PORT_GREETING; the test banners
+    print it."
+#endif
+
+// A host CPU is a host thread, so OS_NCPU above the number the
+// machine can
+// actually run in parallel is not an error -- it oversubscribes,
+// which is a
+// legitimate and useful way to shake out races. It IS worth saying
+// out loud,
+// because a suite that suddenly takes ten times as long has usually
+// done this
+// by accident.
+#if OS_NCPU > 16
+#warning "OS_NCPU above 16 oversubscribes almost any host; expect the
+    tick to drift."
+#endif
+
+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/micro-os-plus-iii-posix-arch/src/exception_handler.cpp b/
micro-os-plus-iii-posix-arch/src/exception_handler.cpp
new file mode 100644
index 0000000..1dc7f9b
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/src/exception_handler.cpp
@@ -0,0 +1,239 @@
+/*
+ * exception_handler.cpp - synchronous faults, reported rather than
+ * silent.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+// Clang -Weverything hardening for the SMP host port (the
+// destination harness is

```

```

+// stricter than the smp branch's own build): nullptr/alias/range-
+// for, per-CPU
+// array indexing, signal macros, fatal hooks and the ucontext
+// makecontext cast.
+// Placed before the includes so the port headers are covered too.
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+#pragma clang diagnostic ignored "-Wmissing-noreturn"
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif
+
+#include <csignal>
+#include <cstring>
+#include <initializer_list>
+#include <dlfcn.h>
+#include <execinfo.h>
+#include <ucontext.h>
+#include <unistd.h>
+
+#include <cmsis-plus/rtos/os.h>
+#include <cmsis-plus/rtos/port/os-decls.h>
+#include <cmsis-plus/rtos/port/os-inlines.h>
+#include <exception_handler.hpp>
+#include <host_cpu.hpp>
+
+namespace
+{
+ // Everything below runs in a signal handler after the process is
+ // already
+ // broken, so it uses write(2) and nothing else: no printf, no
+ // malloc, no
+ // locks. A reporting path that can itself deadlock reports
+ // nothing.
+
+ void
+ emit (const char* s)
+ {
+     std::size_t n = 0;
+     while (s[n] != '\0')
+     {
+         ++n;
+     }
+     ssize_t r = ::write (2, s, n);
+     (void)r;
+ }
+
+ void
+ emit_hex (std::uint64_t v)
+ {
+     static const char digits[] = "0123456789ABCDEF";
+     char out[19] = "0x";
+     int n = 2;
+     for (int i = 15; i >= 0; --i)
+     {
+         out[n++] = digits[(v >> (i * 4)) & 0xF];
+     }
+     out[n] = '\0';
+     emit (out);
+ }
+
+ /*
+  * The program counter of the faulting instruction, from the
+  * machine context
+  * the kernel handed the handler. The ARM ports report ELR for the
+  * same

```

```

+ * reason: on an SMP fault the faulting address alone rarely
+   identifies the
+ * bug, and `addr2line -e <image> <pc>` names the line outright.
+ */
+ /*
+ * The load bias of this image, captured in exception::init() --
+   i.e. in a
+ * sane context, because dladdr() is not async-signal-safe.
+   Reported
+ * alongside the raw pc so that a position-independent executable
+   (which is
+ * the default here) can still be looked up with the static
+   addresses in the
+ * ELF file: `addr2line -e <image> <pc-bias>`.
+ */
+ std::uint64_t g_load_bias = 0;
+
+ std::uint64_t
+ fault_pc (void* ucontext)
+ {
+     if (ucontext == nullptr)
+     {
+         return 0;
+     }
+     ucontext_t* uc = static_cast<ucontext_t*> (ucontext);
+ #if defined(__x86_64__)
+     return static_cast<std::uint64_t> (uc-
+         >uc_mcontext.gregs[REG_RIP]);
+ #elif defined(__i386__)
+     return static_cast<std::uint64_t> (uc-
+         >uc_mcontext.gregs[REG_EIP]);
+ #elif defined(__aarch64__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.pc);
+ #elif defined(__arm__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.arm_pc);
+ #else
+     (void)uc;
+     return 0;
+ #endif
+ }
+
+ void
+ fault_handler (int sig, siginfo_t* info, void* ucontext)
+ {
+     port_fatal_exception (static_cast<std::uint64_t> (sig), 0,
+         fault_pc (ucontext),
+         reinterpret_cast<std::uint64_t> (
+             info != nullptr ? info->si_addr :
+             nullptr),
+         0);
+ }
+ } /* anonymous namespace */
+
+ extern "C" void
+ port_fatal_exception (std::uint64_t type, std::uint64_t esr,
+     std::uint64_t elr, std::uint64_t far,
+     std::uint64_t spsr)
+ {
+     (void)esr;
+     (void)spsr;
+
+     emit ("\n!!! FATAL EXCEPTION on CPU ");
+     {
+         const unsigned cpu = port_cpu_id ();
+         char c = static_cast<char> ('0' + (cpu % 10));
+         ssize_t r = ::write (2, &c, 1);
+         (void)r;
+     }
+     emit (" -- signal ");

```

```

+ emit (::strsignal (static_cast<int> (type)));
+ emit (" at ");
+ emit_hex (far);
+ emit (" pc ");
+ emit_hex (elr);
+ emit (" (static ");
+ emit_hex (elr - g_load_bias);
+ emit (")\n");
+
+ /*
+  * The scheduler state, which is the whole point of reporting at
+  * all: on a
+  * machine with several CPUs, "it crashed" says nothing without
+  * knowing
+  * which thread each CPU was running and who held the kernel lock.
+  * The ARM
+  * ports print the same two facts from their claim tripwire.
+  */
+ for (unsigned c = 0; c < OS_NCPU; ++c)
+ {
+     emit (" cpu");
+     {
+         char d = static_cast<char> ('0' + (c % 10));
+         ssize_t r = ::write (2, &d, 1);
+         (void)r;
+     }
+     emit (" ");
+     os::rtos::thread* th = host_cpu::current_thread (c);
+     emit (th == nullptr ? "(null)"
+         : (th->name () != nullptr ? th->name () :
+            "(anon)"));
+     emit ("\n");
+ }
+ emit (" klock: owner=");
+ emit_hex (os::rtos::port::scheduler::_smp_klock.owner);
+ emit (" depth=");
+ emit_hex (os::rtos::port::scheduler::_smp_klock.depth);
+ emit (" lock=");
+ emit_hex (os::rtos::port::scheduler::_smp_klock.lock);
+ emit ("\n");
+
+ /*
+  * The call chain. Not async-signal-safe in the strict sense
+  * (backtrace()
+  * may take the loader lock on its first call), but the process is
+  * already
+  * dead and a chain of return addresses is what turns "a null
+  * pointer was
+  * dereferenced" into "this line dereferenced it".
+  */
+ {
+     void* frames[24];
+     int n = ::backtrace (frames, 24);
+     emit (" backtrace (static):\n");
+     for (int i = 0; i < n; ++i)
+     {
+         emit (" ");
+         emit_hex (reinterpret_cast<std::uint64_t> (frames[i]) -
+             g_load_bias);
+         emit ("\n");
+     }
+ }
+
+ // Re-raise with the handler removed, so the shell sees the real
+ // cause and a
+ // core file is still produced.
+ ::signal (static_cast<int> (type), SIG_DFL);
+ ::raise (static_cast<int> (type));
+ ::_exit (128 + static_cast<int> (type));
+}

```

```

+
+namespace exception
+{
+    void
+    init ()
+    {
+        {
+            Dl_info info;
+            if (::dladdr (reinterpret_cast<void*> (&port_fatal_exception),
+                &info)
+                != 0)
+            {
+                g_load_bias = reinterpret_cast<std::uint64_t>
+                    (info.dli_fbase);
+            }
+        }
+
+        struct sigaction sa; // `sigaction` is also a function: the tag
+            is required
+        std::memset (&sa, 0, sizeof (sa));
+        sa.sa_flags = SA_SIGINFO | SA_ONSTACK;
+        ::sigemptyset (&sa.sa_mask);
+        sa.sa_sigaction = fault_handler;
+
+        // SA_ONSTACK here, and NOT on the tick -- the two are opposite
+        // cases and
+        // the difference matters.
+        //
+        // The tick must run on the thread's own stack, so its frame
+        // migrates with
+        // the thread when another CPU resumes it (host_cpu.cpp). A fault
+        // handler
+        // never migrates: it reports and the process ends. And it must
+        // not need
+        // the faulting stack, because a blown or wild stack is precisely
+        // one of
+        // the faults worth reporting -- without an alternate stack that
+        // case
+        // cannot be reported at all, which is how the first SMP fault
+        // found here
+        // came out as a silent exit 139.
+        //
+        // The stack itself is per CPU and installed by host_cpu when the
+        // CPU
+        // starts, because sigaltstack() is per host thread.
+        for (int sig : { SIGSEGV, SIGBUS, SIGFPE, SIGILL })
+        {
+            ::sigaction (sig, &sa, nullptr);
+        }
+    }
+} /* namespace exception */
+
+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/micro-os-plus-iii-posix-arch/src/free-store.cpp b/micro-
os-plus-iii-posix-arch/src/free-store.cpp
new file mode 100644
index 00000000..416e750
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/src/free-store.cpp
@@ -0,0 +1,148 @@
+/*
+ * free-store.cpp - the application free store on the POSIX host.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ */

```

```

+ * -----
+ * Why this file exists at all, and why it does NOT just call
+ *   malloc().
+ *
+ * The kernel's own src/startup/initialise-free-store.cpp is guarded
+ *   by
+ *   `#if defined(__ARM_EABI__)` from top to bottom, so on a host it
+ *   compiles to
+ *   nothing and os_startup_initialize_free_store() is undefined --
+ *   which is
+ *   exactly what the carried tests call. Relaxing that guard would
+ *   mean editing
+ *   the kernel, which is kept merge-clean against upstream; so the
+ *   port
+ *   supplies the hook, which is what a port is for.
+ *
+ * Upstream's posix-arch had no such hook and let the system malloc()
+ *   serve,
+ *   and its NOTES.md says so. That was correct for upstream, which had
+ *   ONE host
+ *   thread. It is not correct here, and the difference is the whole
+ *   subject of
+ *   this port:
+ *
+ *   A µOS++ thread may be preempted inside an allocation and resumed
+ *   on a
+ *   DIFFERENT CPU -- that is, a different host thread. glibc's
+ *   malloc is
+ *   thread-safe by taking an arena lock, and that lock would then be
+ *   released
+ *   by a host thread that did not take it. Nothing in the C library
+ *   promises
+ *   that works.
+ *
+ * Using µOS++'s own memory resource removes the question rather than
+ *   betting
+ *   on it: the allocator is the kernel's, its mutual exclusion is the
+ *   kernel's
+ *   scheduler lock, and the scheduler lock already migrates correctly
+ *   because
+ *   making it do so is what the rest of this port is about. It also
+ *   makes the
+ *   host behave like the three silicon ports, which is the point of
+ *   testing on
+ *   it -- a first_fit_top over a fixed block is what every board does,
+ *   so an
+ *   allocation pattern that exhausts the heap here exhausts it there
+ *   too,
+ *   instead of being quietly absorbed by a host with gigabytes to
+ *   spare.
+ * -----
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+// Clang -Weverything hardening for the SMP host port (the
+//   destination harness is
+//   stricter than the smp branch's own build): nullptr/alias/range-
+//   for, per-CPU
+// array indexing, signal macros, fatal hooks and the ucontext
+//   makecontext cast.
+// Placed before the includes so the port headers are covered too.
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+#pragma clang diagnostic ignored "-Wmissing-noreturn"
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif
+
+#include <cstdint>
+#include <new>
+
+#include <cmsis-plus/rtos/os.h>

```

```

#include <cmsis-plus/rtos/os-hooks.h>
#include <cmsis-plus/memory/first-fit-top.h>
#include <cmsis-plus/estd/memory_resource>
+
+using namespace os;
+
+#if defined(OS_TYPE_APPLICATION_MEMORY_RESOURCE)
+using application_memory_resource =
+    OS_TYPE_APPLICATION_MEMORY_RESOURCE;
+#else
+using application_memory_resource = os::memory::first_fit_top;
+#endif
+
+namespace
+{
+    // Same storage trick as the kernel's ARM version: the resource
+    // itself must
+    // outlive everything and must not need the free store to exist
+    // first.
+    alignas (application_memory_resource) char
+        application_free_store[sizeof (application_memory_resource)];
+} // namespace
+
+/*
+ * The out-of-memory hooks, for the same reason as the hook above:
+ * the
+ * kernel's definitions live in the __ARM_EABI__-guarded file. Weak,
+ * so an
+ * application that wants to reset the machine, dump the heap or
+ * coalesce
+ * before giving up replaces them by defining its own -- which is
+ * exactly the
+ * contract they have on the silicon ports.
+ */
+void __attribute__ ((weak))
+os_rtos_application_out_of_memory_hook (void)
+{
+    estd::__throw_bad_alloc ();
+}
+
+#if defined(OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES)
+
+void __attribute__ ((weak))
+os_rtos_system_out_of_memory_hook (void)
+{
+    estd::__throw_bad_alloc ();
+}
+
+#endif /* defined(OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES) */
+
+/*
+ * Weak no-ops, so an application that brings up no hardware still
+ * links.
+ * Every carried test defines both, and its definitions win.
+ */
+void __attribute__ ((weak))
+os_startup_initialize_hardware_early (void)
+{
+}
+
+void __attribute__ ((weak))
+os_startup_initialize_hardware (void)
+{
+}
+
+void __attribute__ ((weak))
+os_startup_initialize_free_store (void* heap_address,
+    std::size_t heap_size_bytes)
+{

```

```

+ #if !defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS)
+
+ new (&application_free_store)
+     application_memory_resource{ "app-heap", heap_address,
+                                   heap_size_bytes };
+
+ reinterpret_cast<rtos::memory::memory_resource*>
+     (&application_free_store)
+     ->out_of_memory_handler
+     (os_rtos_application_out_of_memory_hook);
+
+
+ estd::pmr::set_default_resource (
+     reinterpret_cast<estd::pmr::memory_resource*>
+     (&application_free_store));
+
+
+ // No sbrk() adjustment. On a bare-metal target the kernel's
+ // version pushes
+ // sbrk past the free store so newlib's malloc cannot collide with
+ // it; here
+ // the block is ordinary static storage the host already owns, so
+ // there is
+ // nothing to push it past.
+
+
+ #else
+
+ (void)heap_address;
+ (void)heap_size_bytes;
+ #endif /* !defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS) */
+
+
+ #endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/micro-os-plus-iii-posix-arch/src/host_cpu.cpp b/micro-os-
plus-iii-posix-arch/src/host_cpu.cpp
new file mode 100644
index 0000000..a45a5ec
--- /dev/null
+++ b/micro-os-plus-iii-posix-arch/src/host_cpu.cpp
@@ -0,0 +1,545 @@
+/*
+ * host_cpu.cpp - host threads as CPUs: bring-up, per-CPU tick, IPI.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT
+ * license.
+ */
+
+
+ #if defined(__APPLE__) || defined(__linux__)
+
+ // Clang -Weverything hardening for the SMP host port (the
+ // destination harness is
+ // stricter than the smp branch's own build): nullptr/alias/range-
+ // for, per-CPU
+ // array indexing, signal macros, fatal hooks and the ucontext
+ // makecontext cast.
+ // Placed before the includes so the port headers are covered too.
+ #if defined(__clang__)
+ #pragma clang diagnostic ignored "-Wc++98-compat"
+ #pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+ #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+ #pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+ #pragma clang diagnostic ignored "-Wmissing-noreturn"
+ #pragma clang diagnostic ignored "-Wunreachable-code"
+ #pragma clang diagnostic ignored "-Wunreachable-code-return"
+ #pragma clang diagnostic ignored "-Wcast-function-type-strict"
+ #endif
+
+
+ #include <cerrno>
+ #include <cstdio>

```

```

#include <cstdlib>
#include <cstring>
+
#include <pthread.h>
#include <signal.h>
#include <sys/syscall.h>
#include <time.h>
#include <unistd.h>
+
#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/rtos/port/os-inlines.h>
+
#include <host_cpu.hpp>
+
+/* Defined by the board (test/boards/<board>/src/smp.cpp), with the
   same
+ * meaning as on the ARM boards: 0 = not started, 3 = scheduler
   entered. The
+ * tests wait on it before they start timing anything. A board that
   does not
+ * define it fails to link, which is the point. */
extern "C" volatile uint32_t g_core_stage[OS_NCPU];
+
+// Port-side weak default so a build that does not link the SMP test
   board still
+// resolves g_core_stage[]. The SMP test board (boards/native/src/
   smp.cpp)
+// provides a strong definition that overrides this one when present.
extern "C"
+{
+  __attribute__((weak)) volatile uint32_t g_core_stage[OS_NCPU] =
    {};
+}
+
+namespace
+{
+  using namespace os::rtos::port;
+
+  /* Every CPU's host thread, so one can be signalled by another. */
+  pthread_t g_cpu_thread[OS_NCPU];
+  volatile bool g_cpu_up[OS_NCPU];
+
+  /* The deferred-publish slot, one per CPU. */
+  struct publish_slot
+  {
+    host_cpu::element_t** addr;
+    host_cpu::element_t* val;
+  };
+
+  publish_slot g_publish[OS_NCPU];
+
+  /* What a never-run context must call. Recovered by the trampoline
     from the
+   * two pointers makecontext() carries for it. */
+  using body_t = void (*) (void*);
+
+  [[noreturn]] void
+  fatal (const char* what)
+  {
+    std::fprintf (stderr, "\n!!! posix-arch: %s: %s\n", what,
+                  std::strerror (errno));
+    std::abort ();
+  }
+
+  /*
+   * The interrupt path.
+   *
+   * Deliberately the same shape as boards/rpi-zero-2w/src/rtos/
     port_isr.cpp:

```

```

+  * flag handler mode, do the work, mark a switch pending, clear the
+    flag,
+  *
+  * Two things are deliberate and neither is an oversight:
+  *
+  * - NO_SA_ONSTACK. The handler runs on the µOS++ thread's own
+    stack, so
+  *   the whole signal frame is part of the context that
+    swapcontext() saves
+  *   and therefore migrates with the thread when another CPU
+    resumes it. An
+  *   alternate stack belongs to the host thread and would be left
+    behind.
+  *   This is why OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES is 32 KiB
+    here.
+  *
+  * - The switch happens INSIDE the handler, not after sigreturn.
+    Returning
+  *   first would mean the switch only ever happened at a
+    cooperative point,
+  *   which is exactly the limitation upstream's NOTES.md records
+    ("the
+  *   scheduler runs in cooperative mode only"). swapcontext() from
+    a
+  *   handler is well defined here because the handler frame lives
+    on the
+  *   thread's stack: resuming the context resumes the handler,
+    which then
+  *   returns through sigreturn normally.
+  */
+ void
+ irq_epilogue (unsigned cpu)
+ {
+   if (scheduler::_port_ctx_pending[cpu] == 0)
+   {
+     return;
+   }
+
+   if (os::rtos::scheduler::locked ())
+   {
+     // Still masked by the kernel; the next tick will find it
+     pending.
+     return;
+   }
+
+   if (scheduler::_smp_klock.owner == cpu &&
+       scheduler::_smp_klock.depth > 0)
+   {
+     return;
+   }
+
+   scheduler::_port_ctx_pending[cpu] = 0;
+   scheduler::switch_stacks (nullptr);
+ }
+
+ /* Put `errno` back, out of line.
+  *
+  * Out of line on purpose. `errno` is `*_errno_location()`, which
+  * is native
+  * TLS, and the caller read it BEFORE a swapcontext() that may have
+  * resumed
+  * this thread on a different CPU -- that is, on a different host
+  * thread,
+  * with a different errno. A compiler that cached the address
+  * across the
+  * switch would write the value into the host thread the thread
+  * LEFT. This
+  * port bans native TLS across a switch point for exactly that
+  * reason, and a
+  * noinline call is how the ban is honoured here: the address is
+  * resolved
+  * inside this function, after the switch, on whichever CPU is
+  * running now.
+  */
+ [[gnu::noinline]] void

```

```

+ restore_errno (int value)
+ {
+     errno = value;
+ }
+
+ void
+ tick_handler (int, siginfo_t*, void*)
+ {
+     /* errno belongs to the THREAD, and on this port the thread is
+        the uOS++
+
+     * one, not the host thread it is borrowing.
+     *
+     * This handler does not, in general, return to where it was
+        raised: its
+     * epilogue switches contexts, so control leaves on one thread's
+        stack and
+     * comes back -- possibly on another CPU, possibly much later --
+        when THIS
+     * thread is resumed. Everything the handler and every thread
+        scheduled in
+     * between did to errno is therefore visible to the interrupted
+        code
+     * unless it is put back. On silicon there is no errno to spoil;
+        here
+     * there is.
+     *
+     * `saved` is a local, so like the ASan fibre save it rides the
+     * interrupted thread's own stack and is still correct wherever
+        that
+     * thread comes back. */
+     const int saved = errno;
+
+     const unsigned cpu = port_cpu_id ();
+     if (cpu >= OS_NCPU)
+     {
+         restore_errno (saved);
+         return;
+     }
+
+     interrupts::_in_isr[cpu] = true;
+
+ #if defined(__linux__)
+     // Only CPU 0 advances the kernel clock. Every CPU tickles itself
+        into
+     // re-picking. This is the BCM2837 arrangement exactly: four
+        cores take
+     // the 1 ms PPI, one calls os_systick_handler().
+     if (cpu == 0)
+     {
+         os_systick_handler ();
+     }
+ #else
+     // On non-Linux where per-thread timers are not available
+        (SIGEV_SIGNAL
+     // process-directed delivery), guarantee os_systick_handler() is
+        called
+     // regardless of which host thread received the tick signal so
+        the clock
+     // does not stall.
+     os_systick_handler ();
+ #endif
+
+     scheduler::_port_ctx_pending[cpu] = 1;
+
+     interrupts::_in_isr[cpu] = false;
+
+     irq_epilogue (cpu);
+
+     // Resumed. See the comment at the top of this function.
+     restore_errno (saved);
+ }
+
+

```

```

+ void
+ ipi_handler (int, siginfo_t*, void*)
+ {
+     // Same reasoning as tick_handler(); this one switches contexts
+     // too.
+     const int saved = errno;
+
+     const unsigned cpu = port_cpu_id ();
+     if (cpu >= OS_NCPU)
+     {
+         restore_errno (saved);
+         return;
+     }
+
+     interrupts::_in_isr[cpu] = true;
+     scheduler::_port_ctx_pending[cpu] = 1;
+     interrupts::_in_isr[cpu] = false;
+
+     irq_epilogue (cpu);
+
+     restore_errno (saved);
+ }
+
+ /* One per CPU, never shared: a CPU reporting a fault is not going
+  * anywhere
+  * else while it does so. SIGSTKSZ is a minimum, and the reporting
+  * path
+  * formats numbers, so it gets room. */
+ char g_fault_stack[OS_NCPU][64 * 1024];
+
+ void
+ arm_tick (void)
+ {
+     /* struct */ sigevent sev;
+     std::memset (&sev, 0, sizeof (sev));
+
+ #if defined(__linux__)
+     // SIGEV_THREAD_ID, not ITIMER_REAL.
+     //
+     // ITIMER_REAL delivers SIGALRM to an ARBITRARY thread of the
+     // process, so
+     // with N CPUs the tick would land on whichever host thread the
+     // kernel
+     // happened to choose -- a tick is not a per-CPU event any more,
+     // and the
+     // preemption of CPU 2 could be charged to CPU 0. A per-thread
+     // timer is
+     // what a per-core timer actually is.
+     sev.sigev_notify = SIGEV_THREAD_ID;
+     sev._sigev_un._tid = static_cast<int> (::syscall (SYS_gettid));
+ #else
+     // On non-Linux (e.g. Darwin/BSD), per-thread timer creation is
+     // not available.
+     // Arm the process timer only on CPU 0 to prevent N timers firing
+     // concurrently.
+     if (port_cpu_id () != 0)
+     {
+         return;
+     }
+     sev.sigev_notify = SIGEV_SIGNAL;
+ #endif
+     sev.sigev_signo = clock::signal_number ();
+
+     timer_t timer;
+     if (::timer_create (CLOCK_MONOTONIC, &sev, &timer) != 0)
+     {
+         fatal ("timer_create");
+     }
+
+

```

```

+     const long period_ns = 1000000000L /
+         OS_INTEGER_SYSTICK_FREQUENCY_HZ;
+
+     /* struct */ itimerspec its;
+     its.it_value.tv_sec = 0;
+     its.it_value.tv_nsec = period_ns;
+     its.it_interval.tv_sec = 0;
+     its.it_interval.tv_nsec = period_ns;
+
+     if (::timer_settime (timer, 0, &its, nullptr) != 0)
+     {
+         fatal ("timer_settime");
+     }
+ }
+
+ void*
+ secondary_cpu_body (void* arg)
+ {
+     const unsigned cpu
+         = static_cast<unsigned> (reinterpret_cast<uintptr_t> (arg));
+
+     _this_cpu = cpu;
+
+     host_cpu::install_fault_stack ();
+
+     ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+
+     // A context to switch away from, as port::scheduler::start()
+     // builds for
+     // CPU 0. A secondary core does exactly this after the spin table
+     // releases it: give itself somewhere to save, then reschedule.
+     static os_thread_t fake_thread[OS_NCPU];
+     std::memset (&fake_thread[cpu], 0, sizeof (os_thread_t));
+     fake_thread[cpu].name = "fake_thread_sec";
+     host_cpu::current_thread (cpu)
+         = reinterpret_cast<os::rtos::thread*> (&fake_thread[cpu]);
+
+     g_cpu_up[cpu] = true;
+
+     host_cpu::start_this_cpu_tick ();
+
+     g_core_stage[cpu] = 3;
+
+     scheduler::reschedule ();
+
+     // Only if this CPU never found anything to run.
+     ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr);
+     for (;;)
+     {
+         ::pause ();
+     }
+
+     return nullptr;
+ }
+
+ [[noreturn]] void
+ trampoline (void* func, void* args)
+ {
+     // libucontext contexts carry no uc_sigmask, so guarantee the
+     // masked
+     // start here, before the deferred publish (unmasked again
+     // below).
+     ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+     // Arrival. The switch that brought us here started on another
+     // stack; this
+     // tells ASan the move is complete and that THIS stack is the
+     // live one.
+     // nullptr because a context that has never run has no outgoing
+     // fibre

```

```

+ // bookkeeping of its own to restore.
+ host_cpu::asan_finish_switch (nullptr);
+
+ // A context that has never run still arrives on a CPU that has
+ // just left
+ // another thread behind, so the publish is owed here too -- and
+ // it is
+ // owed BEFORE this thread can be interrupted, or the tick's own
+ // switch
+ // would overwrite the slot and strand the outgoing thread.
+ context::create
+ // starts every new context with this CPU's interrupts masked for
+ // exactly
+ // this window; it ends here.
+ host_cpu::publish_pending ();
+
+
+ ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr);
+
+
+ reinterpret_cast<body_t> (func) (args);
+
+
+ // The kernel's thread body does not return.
+ std::fprintf (stderr, "\n!!! posix-arch: thread body
+ returned\n");
+ std::abort ();
+ }
+
+
+} /* anonymous namespace */
+
+namespace host_cpu
+{
+ void
+ install_handlers (void)
+ {
+ struct sigaction sa; // `sigaction` is also a function: the tag
+ // is required
+ std::memset (&sa, 0, sizeof (sa));
+
+ // SA_RESTART so a preempted read()/write() resumes rather than
+ // failing
+ // with EINTR; the tests print through write(), a thousand times
+ // a second.
+ // No SA_ONSTACK -- see irq_epilogue() above.
+ sa.sa_flags = SA_SIGINFO | SA_RESTART;
+ ::sigemptyset (&sa.sa_mask);
+ ::sigaddset (&sa.sa_mask, clock::signal_number ());
+ ::sigaddset (&sa.sa_mask, clock::ipi_signal_number ());
+
+ sa.sa_sigaction = tick_handler;
+ if (::sigaction (clock::signal_number (), &sa, nullptr) != 0)
+ {
+ fatal ("sigaction(tick)");
+ }
+
+ sa.sa_sigaction = ipi_handler;
+ if (::sigaction (clock::ipi_signal_number (), &sa, nullptr) != 0)
+ {
+ fatal ("sigaction(ipi)");
+ }
+
+ g_cpu_thread[0] = ::pthread_self ();
+ g_cpu_up[0] = true;
+ g_core_stage[0] = 3;
+
+ install_fault_stack ();
+ }
+
+ void
+ install_fault_stack (void)
+ {
+ const unsigned cpu = port_cpu_id ();

```

```

+   if (cpu >= OS_NCPU)
+   {
+       return;
+   }
+
+   stack_t ss;
+   std::memset (&ss, 0, sizeof (ss));
+   ss.ss_sp = g_fault_stack[cpu];
+   ss.ss_size = sizeof (g_fault_stack[cpu]);
+   ss.ss_flags = 0;
+   ::sigaltstack (&ss, nullptr);
+ }
+
+ void
+ start_this_cpu_tick (void)
+ {
+     arm_tick ();
+ }
+
+ void
+ start_secondary_cpus (void)
+ {
+     for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)
+     {
+         pthread_attr_t attr;
+         ::pthread_attr_init (&attr);
+
+         if (::pthread_create (
+             &g_cpu_thread[cpu], &attr, secondary_cpu_body,
+             reinterpret_cast<void*> (static_cast<uintptr_t>
+ (cpu)))
+             != 0)
+         {
+             fatal ("pthread_create(cpu)");
+         }
+
+         ::pthread_attr_destroy (&attr);
+     }
+
+     // Wait for every CPU to have an index and a context to save
+     // into, the
+     // way the ARM ports wait on the per-core stage flags before
+     // proceeding.
+     for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)
+     {
+         while (!g_cpu_up[cpu])
+         {
+             ::usleep (100);
+         }
+     }
+ }
+
+ void
+ send_ipi (unsigned cpu)
+ {
+     if (cpu < OS_NCPU && g_cpu_up[cpu])
+     {
+         ::pthread_kill (g_cpu_thread[cpu], clock::ipi_signal_number
+ ());
+     }
+ }
+
+ void
+ make_entry (os_impl_ucontext_t* ctx, void* func, void* args)
+ {
+     +#pragma GCC diagnostic push
+     +#if defined(__clang__)
+     +#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"

```

```

+ #endif
+ // Two pointer arguments. glibc carries them whole -- verified on
+ // this
+ // host before it was relied on, because the interface is
+ // declared with
+ // int arguments and a truncated `args` would be a pointer to
+ // nothing.
+ os_impl_makecontext (
+     ctx, reinterpret_cast<void (*) (void)> (trampoline), 2, func,
+     args);
+ #pragma GCC diagnostic pop
+ }
+
+ void
+ defer_publish (unsigned cpu, element_t** addr, element_t* val)
+ {
+     g_publish[cpu].addr = addr;
+     g_publish[cpu].val = val;
+ }
+
+ /* See the long comment in host_cpu.hpp. Declared by hand rather
+    than through
+    * <sanitizer/asan_interface.h>, so that a toolchain without the
+    header still
+    * builds the port -- the symbols come from the ASan runtime, which
+    is only
+    * linked when -fsanitize=address is on, and the calls are compiled
+    out
+    * otherwise.
+    *
+    * __has_feature is clang's (and GCC >= 14's): it is tested in its
+    own #if,
+    * because an older GCC rejects `__has_feature (x)` even behind a
+    * `defined (__has_feature) &&`. */
+ #if defined(__SANITIZE_ADDRESS__)
+ #define UOS_HAVE_ASAN 1
+ #elif defined(__has_feature)
+ #if __has_feature(address_sanitizer)
+ #define UOS_HAVE_ASAN 1
+ #endif
+ #endif
+ #if defined(UOS_HAVE_ASAN)
+ extern "C" void
+ __sanitizer_start_switch_fiber (void** fake_stack_save, const void*
+     bottom,
+
+     std::size_t size);
+ extern "C" void
+ __sanitizer_finish_switch_fiber (void* fake_stack_save, const void**
+     bottom_old,
+
+     std::size_t* size_old);
+ #endif
+
+ void
+ asan_start_switch (void** save, const void* bottom, std::size_t
+     size)
+ {
+     #if defined(UOS_HAVE_ASAN)
+     __sanitizer_start_switch_fiber (save, bottom, size);
+     #else
+     (void)save;
+     (void)bottom;
+     (void)size;
+     #endif
+ }
+
+ void
+ asan_finish_switch (void* save)
+ {
+     #if defined(UOS_HAVE_ASAN)
+     __sanitizer_finish_switch_fiber (save, nullptr, nullptr);
+     #else

```

```

+     (void)save;
+ #endif
+ }
+
+ void
+ publish_pending (void)
+ {
+     // The CPU index must be re-read: this runs after a swapcontext()
+     // that
+     // may well have been performed by a different CPU.
+     const unsigned cpu = port_cpu_id ();
+     if (cpu >= OS_NCPU)
+     {
+         return;
+     }
+
+     element_t** addr = g_publish[cpu].addr;
+     if (addr == nullptr)
+     {
+         return;
+     }
+
+     g_publish[cpu].addr = nullptr;
+     __atomic_store_n (addr, g_publish[cpu].val, __ATOMIC_RELEASE);
+ }
+
+ } /* namespace host_cpu */
+
+ extern "C" void port_smp_ipi (unsigned cpu);
+
+ extern "C" void
+ port_smp_ipi (unsigned cpu)
+ {
+     host_cpu::send_ipi (cpu);
+ }
+
+ #endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp b/
micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp
index 91689ca..f59b35a 100644
--- a/micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp
+++ b/micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp
@@ -1,34 +1,82 @@
/*
 * os-core.cpp - the port's half of the µOS++ III SMP scheduler,
 * POSIX host.
 *
 * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
 * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
 * Copyright (c) 2026 Dan. All rights reserved.
 *
 * Permission to use, copy, modify, and/or distribute this software
 * for any purpose is hereby granted, under the terms of the MIT
 * license.
 *
 * If a copy of the license was not distributed with this file, it
 * can
 * be obtained from https://opensource.org/licenses/mit/.
 * -----
 * The model: a host thread IS a CPU.
 *
 * OS_NCPU host threads are created at startup and never destroyed.
 * Each one
 * runs the scheduler and is, for every purpose the kernel can
 * observe, a
 * core: it has its own interrupt mask (its signal mask), its own
 * tick
 * (its own timer_create timer), its own handler-mode flag and its
 * own entry

```

```

+ * in lock_state[] and _port_ctx_pending[].
```

```

+ *
+ * µOS++ threads remain ucontext contexts switched WITHIN a CPU,
+ *   which is
+ * upstream's machinery, preserved deliberately. What is new is that
+ *   a context
+ * saved by one CPU may be resumed by another -- and that is the only
+ *   genuinely
+ * hard part of this port, because it is a data race unless the
+ *   handover is
+ * ordered. See switch_stacks() below.
```

```

+ *
+ * Read this file beside micro-os-plus-iii-aarch64/src/rtos/os-
+ *   core.cpp. The
+ * two have the same functions doing the same things in the same
+ *   order.
```

```

+ * -----
+ * */

#if defined(__APPLE__) || defined(__linux__)

-// -----
+// Clang -Weverything hardening for the SMP host port (the
+//   destination harness is
+// stricter than the smp branch's own build): nullptr/alias/range-
+//   for, per-CPU
+// array indexing, signal macros, fatal hooks and the ucontext
+//   makecontext cast.
+// Placed before the includes so the port headers are covered too.
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+#pragma clang diagnostic ignored "-Wmissing-noreturn"
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif

#include <cassert>
#include <cstdlib>
#include <cstring>

#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/rtos/os-hooks.h>
#include <cmsis-plus/rtos/port/os-inlines.h>

-#include <sys/time.h>
#include <host_cpu.hpp>

-// -----
+#include <sys/utsname.h>
#include <sys/time.h>
#include <unistd.h>

#if defined(__clang__)
#pragma clang diagnostic ignored "-Wc++98-compat"
#endif

-// -----
+extern os::rtos::thread* os_idle_thread;

-uint32_t signal_nesting;
+#if defined(OS_USE_SMP_SCHEDULER)
+namespace os
+{
+  namespace rtos
+  {
+    {
+      namespace scheduler
```

[illegible]

```

-         os_impl_makecontext (ctx, reinterpret_cast<func_t> (func), 1,
-         args);
-#pragma GCC diagnostic pop
-
-         // context->port_.saved = false;
-     }
-
-#pragma GCC diagnostic pop
+     thread_local unsigned _this_cpu = 0;
-
-     //
-     -----
-
-     namespace interrupts
-     {
-         //
-         -----
-
-         sigset_t clock_set;
-
-         //
-         -----
-
-         // Enter an IRQ critical section
-         rtos::interrupts::state_t
-         critical_section::enter (void)
-         {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-             trace::printf ("c ");
-#endif
-             sigset_t old;
-             sigprocmask (SIG_BLOCK, &clock_set, &old);
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-             return sigismember (&old, clock::signal_number);
-#pragma GCC diagnostic pop
-         }
-
-         // Exit an IRQ critical section
-         void
-         critical_section::exit (rtos::interrupts::state_t state)
-         {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-             trace::printf (" c");
-#endif
-             sigprocmask (state ? SIG_BLOCK : SIG_UNBLOCK, &clock_set,
-             nullptr);
-         }
+         sigset_t irq_set;
-
-         //
-         =====
-
-         // Enter an IRQ uncritical section
-         rtos::interrupts::state_t
-         uncritical_section::enter (void)
-         {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-             trace::printf ("u ");
-#endif
-             sigset_t old;
-             sigprocmask (SIG_UNBLOCK, &clock_set, &old);
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-             return sigismember (&old, clock::signal_number);
-#pragma GCC diagnostic pop
-         }

```

```

-
- // Exit an IRQ critical section
- void
- uncritical_section::exit (rtos::interrupts::state_t state)
- {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-     trace::printf (" u}");
-#endif
-     sigprocmask (state ? SIG_BLOCK : SIG_UNBLOCK, &clock_set,
-                 nullptr);
- }
+ extern "C" volatile bool _in_isr[OS_NCPU];
+ volatile bool _in_isr[OS_NCPU] = {};

+ extern "C" volatile uint32_t signal_nesting;
+ volatile uint32_t signal_nesting = 0;
} /* namespace interrupts */

- //
- -----

- namespace scheduler
- {
+ volatile state_t lock_state[OS_NCPU] = {};

- //
- -----

- state_t lock_state;
+ smp_klock_t _smp_klock = { 0, SMP_NO_OWNER, 0 };
+ volatile unsigned _port_ctx_pending[OS_NCPU] = {};

- //
- -----

@@ -179,96 +115,301 @@ namespace os
/* struct */ utsname name;
if (::uname (&name) != -1)
{
-     trace::printf ("POSIX synthetic, running on %s %s %s",
+     trace::printf ("POSIX synthetic SMP, running on %s %s
%s",
name.machine, name.sysname,
name.release);
}
else
{
-     trace::printf ("POSIX synthetic");
+     trace::printf ("POSIX synthetic SMP");
}

-     trace::puts ("; non-preemptive");
+     trace::printf ("; %d CPU%s, %d Hz tick, preemptive\n",
OS_NCPU,
(OS_NCPU == 1) ? "" : "s",
OS_INTEGER_SYSTICK_FREQUENCY_HZ);
}

result_t
initialize (void)
{
-     signal_nesting = 0;
+ // Must be done before the first critical section: every
mask
+ // operation in this port names this set.
+ ::sigemptyset (&interrupts::irq_set);
+ ::sigaddset (&interrupts::irq_set, clock::signal_number
());
+ ::sigaddset (&interrupts::irq_set, clock::ipi_signal_number
());

```

```

-        // Must be done before the first critical section.
-        sigemptyset (&interrupts::clock_set);
+        for (unsigned c = 0; c < OS_NCPU; ++c)
+        {
+            lock_state[c] = state::init;
+            _port_ctx_pending[c] = 0;
+            interrupts::_in_isr[c] = false;
+        }

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-#endif
-        sigaddset (&interrupts::clock_set, clock::signal_number);
-#pragma GCC diagnostic pop
+        host_cpu::install_handlers ();
+
+        /*
+         * The application's hardware hooks.
+         *
+         * On a bare-metal target the kernel's own src/startup/
+         startup.cpp
+         * calls these two before main(), and every carried test
+         relies on
+         * that: os_startup_initialize_hardware() is where a test
+         brings up
+         * its console, prints its banner, installs the free store
+         and calls
+         * exception::init().
+         *
+         * Here there is no such startup. Upstream's NOTES.md
+         states the
+         * rule -- "For portability reasons, execution starts in
+         the main()
+         * function" -- and startup.cpp is __ARM_EABI__-guarded
+         from end to
+         * end, so nothing calls them at all. The first thing every
+         test's
+         * main() calls is scheduler::initialize(), so this is
+         where that
+         * startup belongs, and the tests need no edit.
+         *
+         * Found by running: without this the banner never printed,
+         the
+         * application free store was never installed (the kernel's
+         * malloc resource stayed in place), and exception::init()
+         never
+         * ran -- so the first SMP fault reported nothing at all.
+         *
+         * After install_handlers(), deliberately:
+         exception::init() wants
+         * the alternate signal stack that call sets up.
+         */
+        os_startup_initialize_hardware_early ();
+        os_startup_initialize_hardware ();
+
+        // Interrupts masked until start(), as every port does.
+        ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set,
+        nullptr);

        return result::ok;
    }

    //
    -----

-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wdeprecated-declarations"
-
-    void

```

```

-         start (void)
+         state_t
+         locked (state_t state)
        {
-         {
-             rtos::interrupts::critical_section ics;
-
-             // Determine the next thread.
-             rtos::scheduler::current_thread_
-             = rtos::scheduler::ready_threads_list_.unlink_head
        (
        );
-         }
+         os_assert_throw (!interrupts::in_handler_mode (), EPERM);

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif
-         os_impl_ucontext_t* new_context
-         = reinterpret_cast<os_impl_ucontext_t*> (&(
-             rtos::scheduler::current_thread_
-             >context_.port_.ucontext));
-#pragma GCC diagnostic pop
-
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() ctx %p %s\n",
-             __func__,
-
-             new_context,
-             rtos::scheduler::current_thread_>name ());
-#endif
+         if (state == state::locked)
+         {
+             // Block the tick BEFORE reading the CPU id. The tick
+             handler
+             // swapcontext(s), and the thread can resume on another
+             host
+             // thread: read before the block, the id could name the
+             CPU it
+             // left, and lock_state[] and the kernel lock would be
+             taken for
+             // that CPU, never to be released by the matching
+             unlock.
+             ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set,
+             nullptr);
+             const unsigned cpu = port_cpu_id ();
+             state_t tmp = lock_state[cpu];
+             if (tmp != state::locked)
+             {
+                 if (_smp_klock.owner != cpu)
+                 {
+                     _smp_klock_raw_acquire ();
+                     _smp_klock.owner = cpu;
+                 }
+                 _smp_klock.depth = _smp_klock.depth + 1;
+                 lock_state[cpu] = state::locked;
+             }
+             return tmp;
+         }
+         else
+         {
+             // Holding the lock, the tick is blocked and the thread
+             cannot
+             // move; not holding it, both CPUs read "unlocked"
+             anyway.
+             const unsigned cpu = port_cpu_id ();
+             state_t tmp = lock_state[cpu];
+             if (tmp != state::unlocked)
+             {
+                 lock_state[cpu] = state::unlocked;

```

```

+         if (_smp_klock.owner == cpu && _smp_klock.depth >
+             0)
+         {
+             const uint32_t d = _smp_klock.depth - 1;
+             _smp_klock.depth = d;
+             if (d == 0)
+             {
+                 _smp_klock.owner = SMP_NO_OWNER;
+                 _smp_klock_raw_release ();
+             }
+         }
+         ::pthread_sigmask (SIG_UNBLOCK,
+ &interrupts::irq_set,
+                             nullptr);
+     }
+     return tmp;
+ }
+ }

```

```

-     lock_state = state::init;
+ //

```

```

-#if defined NDEBBUG
-     os_impl_setcontext (new_context);
-#else
-     int res = os_impl_setcontext (new_context);
-     assert (res == 0);
-#endif
-     abort ();
+ void
+ wait_for_interrupt (void)
+ {
+     // The idle thread's "WFI". sigsuspend() unblocks this
+     // CPU's tick
+     // and IPI and sleeps until one of them is delivered AND
+     // handled, so
+     // an idle CPU costs nothing while still being preemptible.
+     sigset_t mask;
+     ::pthread_sigmask (SIG_SETMASK, nullptr, &mask);
+     ::sigdelset (&mask, clock::signal_number ());
+     ::sigdelset (&mask, clock::ipi_signal_number ());
+     ::sigsuspend (&mask);
+ }

```

```

//

```

```

-     state_t
-     locked (state_t state)
+ /*
+  * The context switch.
+  *
+  * Named switch_stacks() because that is what the kernel
+  * calls this
+  * seam on every port, and because it is a friend of
+  * rtos::thread --
+  * which is how it may touch context_ at all. Unlike the ARM
+  * ports it
+  * performs the switch itself rather than returning a stack
+  * pointer to
+  * an assembly restore path; there is no assembly here to
+  * return to.
+  *
+  * THE DEFERRED PUBLISH.
+  *
+  * The SMP picker in the kernel skips any thread whose
+  * stack_ptr is
+  * null, meaning "not safe to claim". The window that rule
+  * exists for
+  * is real here too: between the moment this CPU decides to
+  * leave

```

```

+      * old_thread and the moment swapcontext() has finished
+      writing
+      * old_thread's registers into its ucontext, another CPU must
+      not
+      * resume it -- it would run a half-saved context.
+      *
+      * So the publish cannot be done by the CPU that is leaving:
+      once
+      * swapcontext() returns, this CPU is already executing the
+      INCOMING
+      * thread. It is done by whoever arrives next on this CPU
+      instead. The
+      * outgoing thread's address and value are left in this CPU's
+      slot,
+      * and the first thing any resumed context does -- here,
+      after
+      * swapcontext(), and in host_cpu's trampoline for a context
+      that has
+      * never run -- is publish it.
+      *
+      * This is the same mechanism as AArch64's _smp_pub_addr/
+      _smp_pub_val
+      * pair, which its assembly restore path applies only after
+      SP has
+      * left the outgoing stack. Same hazard, same answer,
+      different
+      * machine.
+      */
+      stack::element_t*
+      switch_stacks (stack::element_t* sp)
+      {
-      os_assert_throw (!interrupts::in_handler_mode (), EPERM);
+      (void)sp;
+
+      /*
+      * MASK THIS CPU FIRST. The switch must be atomic with
+      respect to
+      * this CPU's own interrupts, and on the ARM ports it is
+      atomic for
+      * free: the whole of switch_stacks() runs inside the IRQ
+      path, with
+      * interrupts already masked by the exception entry.
+      *
+      * Here it is not free. reschedule() reaches this function
+      directly
+      * from thread mode, where this CPU's tick is unmasked --
+      so the
+      * timer could land between clearing old_thread's publish
+      flag and
+      * swapcontext() finishing the save, and the handler would
+      re-enter
+      * this same function on a half-performed switch. That is a
+      * genuinely reentrant context switch, and it was the cause
+      of the
+      * intermittent SIGSEGV this port showed on smp_test2
+      (roughly three
+      * runs in five) the first time it ran multi-core.
+      *
+      * The mask is saved by swapcontext() into the outgoing
+      context and
+      * restored from the incoming one, so it follows the thread
+      across
+      * CPUs, which is exactly what interrupt state should do.
+      */
+      sigset_t saved_mask;
+      ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set,
+      &saved_mask);
+
+      const unsigned cpu = port_cpu_id ();
+
+      // Take the kernel lock outright. reschedule() has already
+      returned
+      // early if this CPU still owns it, so it is not held here.
+      _smp_klock_raw_acquire ();
+      _smp_klock.owner = cpu;
+      _smp_klock.depth = 1;
+

```

```

+         rtos::thread* old_thread = host_cpu::current_thread (cpu);
+
+         stack::element_t** pub_addr
+             = &old_thread->context_.port_.stack_ptr;
+         // Any stable non-null value: to the kernel this field is a
+         // flag,
+         // and the only thing it ever asks is whether it is null.
+         stack::element_t* pub_val =
+         reinterpret_cast<stack::element_t*> (
+             &old_thread->context_.port_.ucontext);
+
+         // Park the outgoing thread before the picker can see it.
+         // It may
+         // still re-pick it -- the kernel's test is
+         // `th == old_thread || stack_ptr != nullptr` -- which is
+         // correct,
+         // because this CPU never left it.
+         __atomic_store_n (pub_addr, static_cast<stack::element_t*>
+         (nullptr),
+
+             __ATOMIC_RELEASE);
+
+         rtos::scheduler::internal_switch_threads ();
+
+         rtos::thread* new_thread = host_cpu::current_thread (cpu);
+
+         if (new_thread == nullptr)
+         {
+             trace::printf (
+                 "\n!!! no ready thread and no idle thread on CPU
+ %u !!!\n",
+                 cpu);
+             ::abort ();
+         }
+
+         state_t tmp;
+         if (new_thread == old_thread)
+         {
+             // Nothing to do; republish immediately and let go.
+             __atomic_store_n (pub_addr, pub_val, __ATOMIC_RELEASE);
+             _smp_klock.depth = 0;
+             _smp_klock.owner = SMP_NO_OWNER;
+             _smp_klock_raw_release ();
+             ::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
+             return nullptr;
+         }
+
+         {
+             rtos::interrupts::critical_section ics;
+             os_impl_ucontext_t* new_uc = &new_thread-
+             >context_.port_.ucontext;
+
+             // Claim the incoming context: from here no other CPU may
+             // take it.
+             __atomic_store_n (&new_thread->context_.port_.stack_ptr,
+                 static_cast<stack::element_t*> (nullptr),
+                 __ATOMIC_RELEASE);
+
+             host_cpu::defer_publish (cpu, pub_addr, pub_val);
+
+             /* Release the kernel lock. THE ORDER MATTERS, and it is
+             the same
+             * order and the same reason as the AArch64 port documents
+             at
+             * length: owner and depth are cleared BEFORE the lock
+             word.
+             *
+             * Storing the lock word first opens a window in which
+             another CPU
+             * wins the lock and installs its own owner/depth, which
+             the two
+             * stores below then wipe. Its critical_section::exit() is
+             guarded

```

```

+      * by (owner == cpu && depth > 0), so it never clears the
+      lock word
+      * again, and every CPU spins for ever -- a silent, total
+      freeze.
+      * On the ARM ports that was the root cause of the
+      smp_test4
+      * hardware deadlock. */
+      _smp_klock.depth = 0;
+      _smp_klock.owner = SMP_NO_OWNER;
+      _smp_klock_raw_release ();
+
+      os_impl_ucontext_t* old_uc = &old_thread-
+      >context_.port_.ucontext;
+
+      /* Tell AddressSanitizer the stack is about to move, and
+      where to.
+      * `asan_save` lives on the OUTGOING thread's stack, so it
+      is still
+      * there -- and still this thread's -- whenever and on
+      whichever CPU
+      * that thread is resumed. Compiles to nothing without
+      -fsanitize=
+      * address; see host_cpu.hpp. */
+      void* asan_save = nullptr;
+      host_cpu::asan_start_switch (&asan_save,
+      new_thread->stack ().bottom
+      (),
+      new_thread->stack ().size ());
+
+      if (os_impl_swapcontext (old_uc, new_uc) != 0)
+      {
+          trace::printf ("port::scheduler::%s() swapcontext
+      failed: %s\n",
+      __func__, strerror (errno));
+          ::abort ();
+      }
+
+      tmp = lock_state;
+      lock_state = state;
+      }
+      // Resumed -- and NOT necessarily on the CPU that left.
+      Everything
+      // below must re-read the CPU index; nothing captured above
+      is
+      // valid any more.
+      host_cpu::asan_finish_switch (asan_save);
+      host_cpu::publish_pending ();
+
+      // Unmask last, and only now: everything above this line is
+      the
+      // switch, and the switch is not interruptible.
+      ::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
+
+      return tmp;
+      return nullptr;
+      }
+
+      //
+      -----
+      @@ -276,145 +417,141 @@ namespace os
+      void
+      reschedule (void)
+      {
+          const unsigned cpu = port_cpu_id ();
+
+          if (rtos::scheduler::locked ()
+              || rtos::interrupts::in_handler_mode ()
+              || (rtos::interrupts::in_handler_mode ()
+                  && !rtos::scheduler::preemptive ()))
+          {
+              -#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
+              -      // trace::printf ("port::scheduler::%s() deny\n",
+              __func__);

```

```

-#endif
        return;
    }

-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-    trace::printf ("port::scheduler::%s()\n", __func__);
-#endif
+    // If this CPU still owns the kernel lock -- resume_one()
+    // called
+    // from inside an interrupts::critical_section, as
+    // message_queue's
+    // send/receive do -- switching here would strand owner and
+    // depth on
+    // the outgoing thread's context and deadlock the next
+    // acquirer.
+    // Defer to the next tick, which runs once that section has
+    // exited.
+    if (_smp_klock.owner == cpu && _smp_klock.depth > 0)
+    {
+        _port_ctx_pending[cpu] = 1;
+        return;
+    }

-    // For some complicated reasons, the context save/restore
-    // functions must be called in the same the function.
-    // The idea to inline functions does not work, since
-    // the compiler does not inline functions with context
+    calls.
+    if (rtos::interrupts::in_handler_mode ())
+    {
+        // Taken on the way out of the handler, where the mask
+        // is right.
+        _port_ctx_pending[cpu] = 1;
+        return;
+    }

-    bool save = false;
-    rtos::thread* old_thread;
-    os_impl_ucontext_t* old_ctx;
-    os_impl_ucontext_t* new_ctx;
+    switch_stacks (nullptr);
+    }

-    {
-        rtos::interrupts::critical_section ics;
-
-        old_thread = rtos::scheduler::current_thread;
-        if ((old_thread->state_ == rtos::thread::state::running)
-            || (old_thread->state_ ==
-                rtos::thread::state::suspended)
-            || (old_thread->state_ ==
-                rtos::thread::state::ready))
-        {
-            save = true;
-        }
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-        trace::printf ("port::scheduler::%s() from %s %d %d\n",
-            __func__,
-            old_thread->name (), old_thread->state_,
-            save);
-#endif
+        //
+        -----

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif
-        old_ctx = reinterpret_cast<os_impl_ucontext_t*> (
-            &old_thread->context_.port_.ucontext);

```

```

-#pragma GCC diagnostic pop
+    [[noreturn]] void
+    start (void)
+    {
+        ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set,
+        nullptr);
+
+        // A context to switch AWAY from. The very first switch has
+        to save
+        // something, and this CPU's host-thread context is not a
+        µOS++
+        // thread. The ARM ports use a zeroed fake_thread for the
+        same
+        // reason; this one is never scheduled again, so it needs
+        no stack.
+        static os_thread_t fake_thread[OS_NCPU];
+        const unsigned cpu = port_cpu_id ();
+        memset (&fake_thread[cpu], 0, sizeof (os_thread_t));
+        fake_thread[cpu].name = "fake_thread";
+        host_cpu::current_thread (cpu)
+            = reinterpret_cast<rtos::thread*> (&fake_thread[cpu]);
+
+        for (unsigned c = 0; c < OS_NCPU; ++c)
+        {
+            lock_state[c] = state::init;
+        }
+
+        rtos::scheduler::internal_switch_threads ();
+
+#if defined(OS_USE_SMP_SCHEDULER)
+        // The SMP picker asks each CPU for its own idle thread;
+        CPU 0's is
+        // the kernel's. The secondaries' are installed by the
+        board, from
+        // test-smp-boot.cpp, exactly as on the ARM boards.
+        rtos::scheduler::os_idle_thread_core[0] = ::os_idle_thread;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif
-        new_ctx = reinterpret_cast<os_impl_ucontext_t*> (
-            &rtos::scheduler::current_thread_
-            >context_.port_.ucontext);
-#pragma GCC diagnostic pop
-    }
+    host_cpu::start_this_cpu_tick ();

-    if (old_ctx != new_ctx)
-    {
-        if (save)
-        {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-            trace::printf (
-                "port::scheduler::%s() swapcontext %s -> %s
-                \n",
-                __func__, old_thread->name (),
-                rtos::scheduler::current_thread->name ());
-#endif
-            if (os_impl_swapcontext (old_ctx, new_ctx) != 0)
-            {
-                trace::printf (
-                    "port::scheduler::%s() swapcontext failed
-                    with %s\n",
-                    __func__, strerror (errno));
-                abort ();
-            }
-        }
-    }
-    else

```

```

-         {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() setcontext
-         %s\n",
-
-         __func__,
-         rtos::scheduler::current_thread->
-         name ());
-#endif
-
-         // context->port_.saved = false;
-         if (os_impl_setcontext (new_ctx) != 0)
-         {
-             trace::printf (
-             "port::scheduler::%s() setcontext failed
-             with %s\n",
-             __func__, strerror (errno));
-             abort ();
-         }
-     }
-     else
+     reschedule ();
+
+     // Reached only if reschedule() found nothing to run at
+     all.
+     ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set,
+     nullptr);
+     for (;;)
+     {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() same %s\n",
-         __func__,
-         old_thread->name ());
-#endif
+     ::pause ();
+     }
+ }

-#pragma GCC diagnostic pop
-
-     //
-     -----
-
-     } /* namespace scheduler */
-
-     //
-     -----

-     static void
-     systick_clock_signal_handler (int signum)
+     void
+     context::create (void* context, void* func, void* args)
+     {
-#if defined(OS_TRACE_RTOS_SYSCLOCK_TICK)
-         trace::printf ("i ");
-#endif
-
-         if (signum != clock::signal_number)
-         {
-             char ce = '?';
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wunused-result"
+         /* class */ rtos::thread::context* th_ctx
+         = static_cast< /* class */ rtos::thread::context*>
+         (context);
-
-         write (1, &ce, 1);
+
+         memset (&th_ctx->port_, 0, sizeof (th_ctx->port_));

-#pragma GCC diagnostic pop

```

```

os_impl_ucontext_t* ctx = &th_ctx->port._ucontext;

-         return;
+         if (os_impl_getcontext (ctx) != 0)
+         {
+             trace::printf ("port::context::%s() getcontext failed:
%s\n",
+                             __func__, strerror (errno));
+             ::abort ();
+         }

-         signal_nesting++;
-         // Call the ticks timer ISR.
-         os_systick_handler ();
-         signal_nesting--;
+ #if defined(OS_TRACE_RTOS_SYSCLOCK_TICK)
-         trace::printf (" i}");
+         // The context itself is not wanted; makecontext() merely
+         requires one
+         // obtained from getcontext().
+         ctx->uc_link = nullptr;
+         ctx->uc_stack.ss_sp = th_ctx->stack ().bottom ();
+         ctx->uc_stack.ss_size = th_ctx->stack ().size ();
+         ctx->uc_stack.ss_flags = 0;
+
+         /*
+          * The starting signal mask: THIS CPU'S INTERRUPTS MASKED.
+          *
+          * Two wrong answers were tried before this one, which is why
+          the
+          * reasoning is written down.
+          *
+          * getcontext() snapshots the CALLER's mask, and a thread is
+          very
+          * often created from inside a critical section -- so
+          inheriting it
+          * would start the thread with interrupts masked FOR EVER,
+          because
+          * nothing would ever unmask them. Clearing the mask entirely
+          is
+          * wrong in the opposite direction: a context resumed by
+          * switch_stacks() comes back with interrupts masked and
+          unmarks only
+          * after it has discharged this CPU's deferred publish, and a
+          context
+          * that has never run arrives on a CPU owing exactly the same
+          publish.
+          * Starting it unmasked lets a tick land inside the
+          trampoline before
+          * that publish happens -- and that tick's own switch
+          overwrites the
+          * pending slot, so the thread it was owed to is never
+          republished and
+          * is lost from the ready list for good.
+          *
+          * So a new context begins the way a resumed one does:
+          masked. The
+          * trampoline publishes, then unmarks, and from that point
+          the thread
+          * is preemptible like any other.
+          */
+ #if !defined(OS_INCLUDE_LIBUCONTEXT)
+         ::sigemptyset (&ctx->uc_sigmask);
+         ::sigaddset (&ctx->uc_sigmask, clock::signal_number ());
+         ::sigaddset (&ctx->uc_sigmask, clock::ipi_signal_number ());
+ #endif
+
+         host_cpu::make_entry (ctx, func, args);
+
+         // Published: this context has never run, so it is complete
+         by
+         // definition and any CPU may claim it.
+         th ctx->port .stack ptr

```

```

+         = reinterpret_cast<stack::element_t*> (&th_ctx-
>port_.ucontext);
    }

    //
    =====
@@ -422,75 +559,11 @@ namespace os
    void
    clock_systick::start (void)
    {
-        // set handler
-        struct sigaction sa;
-#if defined(__APPLE__)
-        sa.__sigaction_u.__sa_handler = systick_clock_signal_handler;
-#elif defined(__linux__)
-#pragma GCC diagnostic push
-#if defined(__clang__)
-#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
-#endif
-        sa.sa_handler = systick_clock_signal_handler;
-#pragma GCC diagnostic pop
-#else
-#error Platform unsupported
-#endif
-        sigemptyset (&sa.sa_mask);
-        sa.sa_flags = SA_RESTART;
-
-        if (sigaction (clock::signal_number, &sa, nullptr) != 0)
-        {
-            trace::printf ("port::clock_systick::%s() sigaction()
failed\n",
-                __func__);
-            abort ();
-        }
-
-        // set timer
-        /* struct */ itimerval tv;
-        // first clear all fields
-#if defined(__APPLE__)
-        memset (&tv, 0, sizeof (tv));
-#else
-        timerclear (&tv.it_value);
-#endif
-        // then set the required ones
-
-#if 1
-        tv.it_value.tv_sec = 0;
-        tv.it_value.tv_usec = 1000000 /
rtos::clock_systick::frequency_hz;
-        tv.it_interval.tv_sec = 0;
-        tv.it_interval.tv_usec = 1000000 /
rtos::clock_systick::frequency_hz;
-#else
-        tv.it_value.tv_sec = 1;
-        tv.it_value.tv_usec = 0; // 1000000 /
-        //
rtos::clock_systick::frequency_hz;
-        tv.it_interval.tv_sec = 1;
-        tv.it_interval.tv_usec
= 0; // 1000000 / rtos::clock_systick::frequency_hz;
-#endif
-
-        if (setitimer (ITIMER_REAL, &tv, nullptr) != 0)
-        {
-            trace::printf ("port::clock_systick::%s() setitimer()
failed\n",
-                __func__);
-            abort ();
-        }

```

```

-
-#if 0
-    // Used for initial debugging, to see the signals arriving
-    pause ();
-    for (int i = 50; i > 0; --i)
-    {
-        for (int j = 100; j > 0; --j)
-        {
-            char c = '.';
-            write (1, &c, 1);
-        }
-        char cn = '\n';
-        write (1, &cn, 1);
-    }
-#endif
+    // Each CPU arms its own timer when it starts; this is CPU
+    // 0's, and
+    // the secondaries' are armed by
+    host_cpu::start_secondary_cpus().
+    // Only CPU 0 advances the kernel clock -- exactly as on the
+    BCM2837,
+    // where all four cores take a 1 ms PPI but only core 0 calls
+    // os_systick_handler(). The others use theirs to preempt
+    themselves.
+}

//
=====
@@ -498,15 +571,13 @@ namespace os
    static uint64_t previous_timestamp;

    static uint64_t
    get_current_micros (void);

    uint64_t
    get_current_micros (void)
    {
-        /* struct */ timeval tp;
-        gettimeofday (&tp, nullptr);
+        /* struct */ timespec tp;
+        ::clock_gettime (CLOCK_MONOTONIC, &tp);

-        return static_cast<uint64_t> (tp.tv_sec * 1000000 +
-tp.tv_usec);
+        return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
+        + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
    }

    void
@@ -518,8 +589,8 @@ namespace os
    uint32_t
    clock_highres::input_clock_frequency_hz (void)
    {
-        // The posix system clock resolution is 1 us, so it makes no
-        // sense to assume a frequency higher than 1 MHz.
+        // CLOCK_MONOTONIC is read here at microsecond resolution, so
+        a
+        // higher frequency would be a claim the source cannot
+        support.
        return 1000000;
    }

@@ -538,9 +609,8 @@ namespace os
    clock_highres::cycles_since_tick (void)
    {
        uint64_t ts = get_current_micros ();
-        uint32_t delta = static_cast<uint32_t> (ts -
-previous_timestamp);

-        return delta;

```

```

+         return static_cast<uint32_t> (ts - previous_timestamp);
+     }

+     } /* namespace port */
@@ -549,4 +619,12 @@ namespace os

+ // -----

+// Never inlined, and never merged or hoisted: see _this_cpu in os-
+// inlines.h.
+extern "C" __attribute__((noinline)) unsigned
+port_cpu_id (void)
+{
+    + __asm__ volatile("" ::: "memory");
+    + return os::rtos::port::_this_cpu;
+}
+
+
+ #endif /* defined(__APPLE__) || defined(__linux__) */

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base step/13 --head dan-maio:step/14 \
  --title "feat(smp): multi-core SMP scheduler, port contracts &
  native-smp dual-core platform"

```

PR #24: Port Package Semantic Version Releases

- **Single Theme:** Formalizing and tagging SMP port contract releases.
- **Repositories:** micro-os-plus-iii-posix-arch, micro-os-plus-iii-cortexm
- **Files Touched:** posix-arch/package.json (v1.1.0), cortexm/package.json (v1.2.0)
- **Problem & Solution:**
 - Releases posix-arch v1.1.0 and cortexm v1.2.0 in package.json.
 - **Note:** this is a release *action*, not a source divergence. The smp branch deliberately keeps the baseline versions (posix-arch 1.0.1, cortexm 1.1.0); the bump is cut during integration by scripts/smp/release-port.sh on step/24.
- **Release Action Applied:**

```
scripts/smp/release-port.sh "$P" 1.1.0 "SMP host CPU model + highres clock"
scripts/smp/release-port.sh "$C" 1.2.0 "SMP Cortex-M port contract"
# Each bumps package.json.version, commits and creates the local tag:
# micro-os-plus-iii-posix-arch/package.json: "version": "1.0.1" -> "1.1.0"
# micro-os-plus-iii-cortexm/package.json: "version": "1.1.0" -> "1.2.0"
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii-posix-arch --base step/14 --head dan-maio:step/24 --title "release: v1.1.0"
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm --base step/14 --head dan-maio:step/24 --title "release: v1.2.0"
```

PR #25: History-Preserving Devices Repository Subtree Dissolution

- **Single Theme:** Folding device drivers and SoC silicon files into architecture repositories.
- **Order:** Part 0 runs **before** Part A (see the canonical order above); the other C/D steps run after Part B.
- **Repositories:** micro-os-plus-iii-posix-arch, micro-os-plus-iii-cortexm, micro-os-plus-iii-aarch32, micro-os-plus-iii-aarch64
- **Problem & Solution:**
 - Dissolves micro-os-plus-iii-devices using `git subtree add` (driven by `scripts/smp/migrate-devices.sh`), preserving 100% of commit history in the owning architecture repositories (`soc/*`, `drivers/*`).
 - Until the subtree lands, the architecture ports keep resolving the legacy `micro-os-plus::devices` target by conditionally consuming a sibling `micro-os-plus-iii-devices` working copy, so no existing board target changes.
- **Migration Action (no source diff on xpack-development):**

```
# Part 0 is a history-preserving repository dissolution, not a file
# patch.
# migrate-devices.sh uses `git subtree split` + `git subtree add` to
# fold
# micro-os-plus-iii-devices into the owning architecture repositories
# (drivers/* and soc/<soc>/), preserving every commit. The AArch ports
# then
# resolve the legacy `micro-os-plus::devices` target from a sibling
# working
# copy when present, so no existing board target changes.
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm \
  --base xpack-development --head dan-maio:part0-devices \
  --title "refactor(devices): subtree migration of STM32F4 and RP2350
  SoC support from devices repo"
```

PR #26: Modern Multi-Core Silicon Ports (Cortex-M33 & RP2350)

- **Single Theme:** Cortex-M33 (ARMv8-M) and Raspberry Pi RP2350 dual-core silicon support.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-cortexm
- **Files Touched:** micro-os-plus-iii-cortexm/include-m33/, include-rp2350/, src/rtos/os-core-m33.cpp, src/rtos/os-core-rp2350.cpp, kernel src/rtos/os-thread.cpp
- **Problem & Solution:**
 - Adds Cortex-M33 port with Message Handling Unit (MHU) IPI signaling and per-core SysTick.
 - Adds RP2350 port whose recursive `_smp_klock` is backed by SIO Hardware Spinlock 0 (rp2350::reg (rp2350::SIO_SPINLOCK0)) and FIFO IRQ 25.
 - In src/rtos/os-thread.cpp, wake-up IPI dispatching when `OS_INTEGER_RTOS_PORT_NCPU > 1`.
- **Unified Diff Excerpt (new include-rp2350 port lock):**

```
diff --git a/micro-os-plus-iii-cortexm/include-rp2350/cmsis-plus/rtos/
      port/os-inlines.h b/micro-os-plus-iii-cortexm/include-rp2350/
      cmsis-plus/rtos/port/os-inlines.h
new file mode 100644
index 0000000..505649f
--- /dev/null
+++ b/micro-os-plus-iii-cortexm/include-rp2350/cmsis-plus/rtos/port/
      os-inlines.h
@@ -0,0 +1,539 @@
+/*
+ * os-inlines.h – inline port implementations for the RP2350 (Cortex-
+ * M33)
+ *
+ * port of the shared micro-os-plus-iii kernel.
+ *
+ * Shadows the Cortex-M port's `cmsis-plus/rtos/port/os-inlines.h`.
+ *
+ * Dual-branch, gated on OS_USE_SMP_SCHEDULER:
+ * - single-core: identical to the Cortex-M port (DI/EI or BASEPRI
+ *   critical
+ *   sections, scalar lock_state) – the board-verified Phase 2
+ *   behaviour;
+ * - SMP: per-core lock_state, the recursive kernel spinlock
+ *   `_smp_klock`
+ *   backed by the RP2350 SIO hardware spinlock 0, and PRIMASK-
+ *   masked
+ *   critical sections (see the SMP model notes below).
+ */
+
+#ifndef CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_
+#define CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_
+
+// -----
+
+#if defined(OS_USE_OS_APP_CONFIG_H)
+#include <cmsis-plus/os-app-config.h>
+#endif
+
+#include <cmsis-plus/rtos/os-c-decls.h>
+
+// -----
+
+#ifdef __cplusplus
+
+#include <cmsis_device.h>
+
+#include <cmsis-plus/diag/trace.h>
+
+#if defined(OS_USE_SMP_SCHEDULER)
```

```

#include <bsp/rp2350.hpp>
#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+namespace os
+{
+ namespace rtos
+ {
+ namespace port
+ {
+ //
+ -----
+
+ namespace scheduler
+ {
+
+ inline void __attribute__ ((always_inline))
+ greeting (void)
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+ trace::printf (
+ "Scheduler: µOS++ RP2350 Cortex-M33 SMP, 2 cores, 1ms
+ tick\n");
+ // At this stage the system clock should have already been
+ configured
+ // at 150 MHz by the pico2_early_init constructor.
+ trace::printf ("System clock: %u Hz\n", SystemCoreClock);
+ #else
+ trace::printf ("Scheduler: µOS++ Cortex-M");
+ #if defined(__ARM_ARCH_7EM__) || defined(__ARM_ARCH_8M_MAIN__)
+ trace::putchar ('0' + __CORTEX_M); // M4/M7
+ #if defined(__ARM_FP)
+ trace::printf (" FP");
+ #endif
+ #elif defined(__ARM_ARCH_7M__)
+ trace::printf ("3");
+ #elif defined(__ARM_ARCH_6M__)
+ trace::printf ("0/0+");
+ #endif
+ trace::printf (" , preemptive");
+ #if (defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)) \
+ && defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+ trace::printf (" , BASEPRI(%u)",
+ OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY);
+ #else
+ trace::printf (" , DI/EI");
+ #endif
+ #if defined(OS_EXCLUDE_RTOS_IDLE_SLEEP)
+ trace::printf (" , no WFI");
+ #else
+ trace::printf (" , WFI");
+ #endif
+ trace::puts ("");
+
+ trace::printf ("System clock: %u Hz\n", SystemCoreClock);
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+ }
+
+ inline port::scheduler::state_t __attribute__
+ ((always_inline))
+ lock (void)
+ {
+ return locked (state::locked);
+ }
+
+ inline port::scheduler::state_t __attribute__
+ ((always_inline))
+ unlock (void)

```

```

+     {
+         return locked (state::unlocked);
+     }
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+
+     // Current core index, read from SIO_CPUID (0xD0000000).
+     inline unsigned __attribute__ ((always_inline))
+     port_cpu_id (void)
+     {
+         return rp2350::cpu_id ();
+     }
+
+     inline bool __attribute__ ((always_inline))
+     locked (void)
+     {
+         return lock_state[port_cpu_id ()] != state::unlocked;
+     }
+
+ #else
+
+     inline bool __attribute__ ((always_inline))
+     locked (void)
+     {
+         return lock_state != state::unlocked;
+     }
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     inline void __attribute__ ((always_inline))
+     wait_for_interrupt (void)
+     {
+ #if !defined(OS_EXCLUDE_RTOS_IDLE_SLEEP)
+ #if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
+         trace::printf ("%s() \n", __func__);
+ #endif
+         // The DSB is recommended by ARM before WFI.
+         __DSB ();
+         __WFI ();
+ #endif /* !defined(OS_EXCLUDE_RTOS_IDLE_SLEEP) */
+     }
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+
+     // ---- SMP kernel lock (SIO hardware spinlock 0)
+     -----
+
+     // Claim the hardware spinlock (reading it returns 0 while
+     another
+
+     // core holds it; any write releases it).
+     inline void __attribute__ ((always_inline))
+     _smp_klock_raw_acquire (void)
+     {
+         while (rp2350::reg (rp2350::SIO_SPINLOCK0) == 0)
+         {
+         }
+         __asm__ volatile ("dmb" ::: "memory");
+     }
+
+     inline void __attribute__ ((always_inline))
+     _smp_klock_raw_release (void)
+     {
+         __asm__ volatile ("dmb" ::: "memory");
+         rp2350::reg (rp2350::SIO_SPINLOCK0) = 1;
+         __asm__ volatile ("dsb" ::: "memory");
+     }
+
+

```

```

+         // Mask IRQs on this core and enter the kernel lock
+         (recursive, with
+         // owner tracking so nested critical sections only bump the
+         depth).
+         inline void __attribute__ ((always_inline))
+         port_set_lock (void)
+         {
+             __asm__ volatile ("cpsid i" ::: "memory");
+             unsigned cpu = port_cpu_id ();
+             if (_smp_klock.owner != cpu)
+             {
+                 _smp_klock_raw_acquire ();
+                 _smp_klock.owner = cpu;
+             }
+             _smp_klock.depth = _smp_klock.depth + 1;
+         }
+
+         // Leave the kernel lock and restore the given PRIMASK state.
+         inline void __attribute__ ((always_inline))
+         port_put_lock (uint32_t pri)
+         {
+             unsigned cpu = port_cpu_id ();
+             if (_smp_klock.owner == cpu && _smp_klock.depth > 0)
+             {
+                 uint32_t d = _smp_klock.depth - 1;
+                 _smp_klock.depth = d;
+                 if (d == 0)
+                 {
+                     _smp_klock.owner = SMP_NO_OWNER;
+                     _smp_klock_raw_release ();
+                 }
+             }
+             __set_PRIMASK (pri);
+         }
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     } /* namespace scheduler */
+
+     // The DSB/ISB are recommended by ARM after priority level
+     changes.
+     // (Application Note 321: ARM Cortex-M Programming Guide to
+     // Memory Barrier Instructions)
+     // http://infocenter.arm.com/help/topic/com.arm.doc.dai0321a/DAI0321A\_programming\_guide\_memory\_barriers\_for\_m\_profile.pdf
+
+     namespace interrupts
+     {
+         inline bool __attribute__ ((always_inline))
+         in_handler_mode (void)
+         {
+             // In Handler mode, IPSR holds the exception number.
+             // If 0, the core is in thread mode.
+             return (__get_IPSR () != 0);
+         }
+
+         inline bool
+         is_priority_valid (void)
+         {
+             #if defined(OS_USE_SMP_SCHEDULER)
+             // With DI/EI (PRIMASK) critical sections, all priorities
+             are valid.
+             return true;
+             #else
+             #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+             || defined(__ARM_ARCH_8M_MAIN__)
+
+             #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)

```

```

+
+ // The trick here is to properly account for differences
+ // between Cortex-M exception number stored in IPSR (an
+ // unsigned byte), and the CMSIS interrupt number (IRQn).
+ uint32_t exception_number = __get_IPSR ();
+ if (exception_number > 0)
+ {
+     uint32_t prio = NVIC_GetPriority (
+         static_cast<IRQn_Type> (exception_number - 16));
+     return (prio
+         >=
+         OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY);
+ }
+ else
+ {
+     return true;
+ }
+
+ #else
+
+ // When using DI/EI, all priority levels are valid.
+ return true;
+
+ #endif /*
+     defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+ */
+
+ #elif defined(__ARM_ARCH_6M__)
+
+ // When using DI/EI, all priority levels are valid.
+ return true;
+
+ #endif /* architecture */
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+ }
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+
+ // Enter an IRQ critical section: mask this core's IRQs and
+ // take the
+ // kernel spinlock so the shared data is also safe against
+ // the other
+ // core. Recursive (owner+depth), so nested sections are
+ // fine.
+ inline rtos::interrupts::state_t __attribute__
+ ((always_inline))
+ critical_section::enter (void)
+ {
+     uint32_t pri = __get_PRIMASK ();
+     scheduler::port_set_lock ();
+     return pri;
+ }
+
+ // Exit an IRQ critical section.
+ inline void __attribute__ ((always_inline))
+ critical_section::exit (rtos::interrupts::state_t state)
+ {
+     scheduler::port_put_lock (state);
+ }
+
+ #else
+
+ // Enter an IRQ critical section (single-core, like the
+ // Cortex-M port).
+ inline rtos::interrupts::state_t __attribute__
+ ((always_inline))
+ critical_section::enter (void)
+ {
+     uint32_t pri;
+

```

```

+ #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__)
+
+ #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+
+     // Read the current BASEPRI, to be returned and later
+     restored.
+     pri = __get_BASEPRI ();
+
+     // According to ARM "MSR{cond} spec_reg, Rn":
+     // When you write to BASEPRI_MAX, the instruction writes to
+     // BASEPRI only if either:
+     // Rn is non-zero and the current BASEPRI value is 0
+     // Rn is non-zero and less than the current BASEPRI value.
+     __set_BASEPRI_MAX (
+         OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY
+         << ((8 - __NVIC_PRIO_BITS)));
+
+ #else
+
+     // Read the current PRIMASK, to be returned and later
+     restored.
+     pri = __get_PRIMASK ();
+
+     // Disable all interrupts.
+     __disable_irq ();
+
+ #endif /*
+     defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+ */
+
+ #elif defined(__ARM_ARCH_6M__)
+
+     // Read the current PRIMASK, to be returned and later
+     restored.
+     pri = __get_PRIMASK ();
+
+     // Disable all interrupts.
+     __disable_irq ();
+
+ #endif
+
+     return pri;
+ }
+
+ // Exit an IRQ critical section (single-core).
+ inline void __attribute__ ((always_inline))
+ critical_section::exit (rtos::interrupts::state_t state)
+ {
+ #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__)
+
+ #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+
+     // Restore BASEPRI to the value saved by enter().
+     __set_BASEPRI (state);
+
+ #else
+
+     // Restore PRIMASK to the value saved by enter().
+     __set_PRIMASK (state);
+
+ #endif /*
+     defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+ */
+
+ #elif defined(__ARM_ARCH_6M__)
+
+     // Restore PRIMASK to the value saved by enter().

```

```

+         __set_PRIMASK (state);
+
+     #endif
+     }
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     //
+     =====
+
+     // Enter an IRQ uncritical section.
+     inline rtos::interrupts::state_t __attribute__
+     ((always_inline))
+     uncritical_section::enter (void)
+     {
+         uint32_t pri;
+
+ #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+     || defined(__ARM_ARCH_8M_MAIN__)
+
+ #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY) \
+     && !defined(OS_USE_SMP_SCHEDULER)
+
+         pri = __get_BASEPRI ();
+
+         // Setting BASEPRI to 0 makes it ineffective,
+         // practically enabling all interrupts.
+         __set_BASEPRI (0);
+
+ #else
+
+         pri = __get_PRIMASK ();
+
+         // Enable all interrupts.
+         __enable_irq ();
+
+ #endif /*
+         defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY) \
+         && !defined(OS_USE_SMP_SCHEDULER) */
+
+ #elif defined(__ARM_ARCH_6M__)
+
+         pri = __get_PRIMASK ();
+
+         // Enable all interrupts.
+         __enable_irq ();
+
+ #endif
+
+         return pri;
+     }
+
+     // Exit an IRQ uncritical section.
+     inline void __attribute__ ((always_inline))
+     uncritical_section::exit (rtos::interrupts::state_t state)
+     {
+ #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+     || defined(__ARM_ARCH_8M_MAIN__)
+
+ #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY) \
+     && !defined(OS_USE_SMP_SCHEDULER)
+
+         // Restore BASEPRI to the value saved by enter().
+         __set_BASEPRI (state);
+
+ #else
+

```

```

+         // Restore PRIMASK to the value saved by enter().
+         __set_PRIMASK (state);
+
+ #endif /*
+     defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY) \
+     && !defined(OS_USE_SMP_SCHEDULER) */
+
+ #elif defined(__ARM_ARCH_6M__)
+
+         // Restore PRIMASK to the value saved by enter().
+         __set_PRIMASK (state);
+
+ #endif
+     }
+
+ } /* namespace interrupts */
+
+ //
+ =====
+
+ namespace this_thread
+ {
+     inline void __attribute__ ((always_inline))
+     prepare_suspend (void)
+     {
+         ;
+     }
+
+ } /* namespace this_thread */
+
+ //
+ =====
+
+ inline void __attribute__ ((always_inline))
+ clock_highres::start (void)
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+     if (rp2350::reg (rp2350::RESETS_BASE + 0x00U) & (1U << 23U))
+     {
+         rp2350::reg (rp2350::RESETS_BASE + 0x00U) &= ~(1U <<
+ 23U);
+         while (!(rp2350::reg (rp2350::RESETS_BASE + 0x08U) & (1U
+ << 23U)))
+         {
+         }
+     }
+     rp2350::reg (rp2350::TICKS_BASE + 0x1CU) = 12U;
+     rp2350::reg (rp2350::TICKS_BASE + 0x18U) = 1U;
+ #endif
+ }
+
+ inline constexpr bool __attribute__ ((always_inline))
+ clock_highres::has_hardware_counter (void) noexcept
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+     return true;
+ #else
+     return false;
+ #endif
+ }
+
+ inline uint64_t __attribute__ ((always_inline))
+ clock_highres::hardware_counter (void) noexcept
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+     constexpr std::uint32_t timer0_base = 0x400B0000U;
+     uint32_t hi = rp2350::reg (timer0_base + 0x08U);
+     uint32_t lo = rp2350::reg (timer0_base + 0x0CU);

```

```

+         return (static_cast<uint64_t> (hi) << 32) | lo;
+     }
+     return 0;
+ }
+
+     inline uint32_t __attribute__ ((always_inline))
+     clock_highres::input_clock_frequency_hz (void)
+     {
+ #if defined(OS_USE_SMP_SCHEDULER)
+         return 1000000U;
+ #else
+         // The SysTick is clocked with the CPU clock.
+         return SystemCoreClock;
+ #endif
+     }
+
+ #pragma GCC diagnostic push
+ #pragma GCC diagnostic ignored "-Wold-style-cast"
+     inline uint32_t __attribute__ ((always_inline))
+     clock_highres::cycles_per_tick (void)
+     {
+         return SysTick->LOAD + 1;
+     }
+
+     inline uint32_t __attribute__ ((always_inline))
+     clock_highres::cycles_since_tick (void)
+     {
+         uint32_t load_value = SysTick->LOAD;
+
+         // Initial sample of the decremting counter.
+         uint32_t val = SysTick->VAL;
+
+         // Check overflow. If the exception is pending, the total
+         // cycles
+         // count in steady_count_ does not yet reflect the correct
+         // value.
+         if ((SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) != 0)
+         {
+             uint32_t val_subsequent = SysTick->VAL;
+
+             // If the value did decrease, the timer did not recycle
+             // between the two reads.
+             if (val > val_subsequent)
+             {
+                 return load_value + 1 + (load_value - val);
+             }
+         }
+
+         return load_value - val;
+     }
+ #pragma GCC diagnostic pop
+
+     //
+     =====
+
+     } /* namespace port */
+ } /* namespace rtos */
+} /* namespace os */
+
+ #pragma GCC diagnostic pop
+
+ // -----
+
+ #endif /* __cplusplus */

```

```
+  
+ #endif /* CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_ */
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii-cortexm \  
  --base step/24 --head dan-maio:step/25 \  
  --title "feat(cortexm): ARM Cortex-M33 and Raspberry Pi RP2350  
    multi-core silicon support"
```

PR #27: Modular Add-Only CMake Build Architecture & Cross Toolchains

- **Single Theme:** Modular add-only CMake targets, cross toolchains, and 2xcortex-m33 platform.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-cortexm
- **Files Touched:** CMakeLists.txt, cmake/uos-app.cmake, cmake/toolchains/, tests/platforms/2xcortex-m33/
- **Problem & Solution:**
 - Adds micro-os-plus::iii-core, ::iii-posix-io, ::iii-startup, ::iii-semihosting, ::port-smp-decls modular sub-targets to CMakeLists.txt.
 - Adds cmake/uos-app.cmake and cross toolchains (arm-none-eabi.cmake, aarch64-none-elf.cmake).
 - Registers tests/platforms/2xcortex-m33/ (mps2-an521 --cpu cortex-m33 --smp 2).
- **Exact Unified Diff Applied:**

```
diff --git a/CMakeLists.txt b/CMakeLists.txt
index a3aa23e5..78489ca9 100644
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -1,144 +1,240 @@
# -----
+# micro-os-plus-iii-smp – the common half of the unified μOS++ III
+  SMP project.
#
# This file is part of the μOS++ project (https://micro-os-plus.github.io/).
# Copyright (c) 2021-2025 Liviu Ionescu. All rights reserved.
+# This repository is the RTOS kernel (src/, include/) and the runners
+  that
+# drive a build on QEMU or on hardware (test_smpl/). Nothing else.
#
# Permission to use, copy, modify, and/or distribute this software
+  for any
# purpose is hereby granted, under the terms of the MIT license.
+# The test APPLICATIONS are not here. Every board owns its own, in
+# boards/<id>/test/ of the architecture project that has that board,
+  because a
+# board is what decides which tests make sense on it and what they
+  are built
+# with.
#
# If a copy of the license was not distributed with this file, it can
+  be
# obtained from https://opensource.org/licenses/mit.
+# Architecture-specific code lives in its own INDEPENDENT repository,
+  outside
+# this one – micro-os-plus-iii-cortexm, -aarch32, -aarch64, -posix-
+  arch. The
+# dependency runs one way only: an architecture project consumes this
+  one.
#
# -----
-
# This file is intended to be consumed by applications with:
#
# `add_subdirectory("xpacks/@micro-os-plus/micro-os-plus-iii")`
+# add_subdirectory (micro-os-plus-iii-smp) # or a submodule
+  of it
+# target_link_libraries (my-app PRIVATE micro-os-plus::iii)
#
# The result is an interface library that can be added to the linker
+  with:
+# Nothing here may depend on an architecture project.
#
# `target_link_libraries(your-target PUBLIC micro-os-plus::iii)`
```

```

+# Portions derived from µOS++ IIIE by Liviu Ionescu, MIT licensed;
    see
+# LICENSE-micro-os-plus-iii.
# -----
-# Preamble.

-# https://cmake.org/cmake/help/v3.20/
cmake_minimum_required (VERSION 3.20)

-project (micro-os-plus-iii DESCRIPTION "µOS++ IIIE")
+project (
+  micro-os-plus-iii-smp
+  DESCRIPTION "µOS++ III SMP – common kernel, devices and tests"
+  LANGUAGES C CXX ASM
+)

-if (COMMAND xpack_get_package_name_and_version)
-  xpack_get_package_name_and_version (
-    "${CMAKE_CURRENT_SOURCE_DIR}/package.json"
-  )
-  message (VERBOSE
-    "Processing xPack ${PACKAGE_JSON_NAME}@${
-      PACKAGE_JSON_VERSION}..."
-  )
+# True only when this repo is the top-level build, false when an
  architecture
+# project consumes it.
+if (CMAKE_SOURCE_DIR STREQUAL CMAKE_CURRENT_SOURCE_DIR)
+  set (UOS_SMP_STANDALONE TRUE)
+else ()
+  set (UOS_SMP_STANDALONE FALSE)
+endif ()

# -----
-# The project library definitions.
+# Absolute paths are forbidden; consumers use these instead.
+set (UOS_SMP_ROOT      "${CMAKE_CURRENT_SOURCE_DIR}" CACHE INTERNAL
+  "")
+set (UOS_CMAKE_DIR     "${CMAKE_CURRENT_SOURCE_DIR}/cmake" CACHE
+  INTERNAL "")
+set (UOS_TOOLCHAINS_DIR "${CMAKE_CURRENT_SOURCE_DIR}/cmake/
+  toolchains" CACHE INTERNAL "")

-add_library (micro-os-plus-iii-interface INTERFACE EXCLUDE_FROM_ALL)
+include ("${CMAKE_CURRENT_SOURCE_DIR}/cmake/uos-app.cmake")

# -----
-# Target settings.
+# Return early if micro-os-plus::iii has already been defined by a
  parent or sibling.
+if (TARGET micro-os-plus::iii)
+  return ()
+endif ()
+
# -----
+# The kernel – an INTERFACE library, compiled by whichever
  application links it
+# against that application's architecture port.
+#
+# It does NOT compile on its own: include/cmsis-plus/rtos/os-decls.h
  includes
+# <cmsis-plus/rtos/port/os-decls.h>, which an architecture project
  supplies.
+
+add_library (micro-os-plus-iii-interface INTERFACE)

target_include_directories (
  micro-os-plus-iii-interface INTERFACE "include" "include/cmsis-
  plus/legacy"
)

```

```

+# The core: what every port compiles. Measured against the proven rpi
+# builds, which took exactly this set and nothing else.
target_sources (
    micro-os-plus-iii-interface
-   INTERFACE src/diag/trace-itm.cpp
-       src/diag/trace-segger-rtt.cpp
-       src/diag/trace-semihosting.cpp
-       src/diag/trace.cpp
-       src/driver/common.cpp
-       src/driver/serial.cpp
-       # src/driver/usart-wrapper.cpp src/driver/usb-device.cpp
-       # src/driver/usb-host.cpp src/driver/usbd-wrapper.cpp
-       # src/driver/usbh-wrapper.cpp
-       src/libc/_sbrk.c
-       src/libc/newlib/c-newlib-reent.cpp
-       src/libc/stdlib/assert.c
-       src/libc/stdlib/atexit.cpp
-       src/libc/stdlib/atexit.h
-       src/libc/stdlib/exit.c
-       src/libc/stdlib/init-fini.c
-       src/libc/stdlib/malloc.cpp
-       src/libc/stdlib/timegm.c
-       src/libcpp/chrono.cpp
-       src/libcpp/condition-variable.cpp
-       src/libcpp/cxx.cpp
-       src/libcpp/memory-resource.cpp
-       src/libcpp/mutex.cpp
-       src/libcpp/new.cpp
-       src/libcpp/system-error.cpp
-       src/libcpp/thread-cpp.h
-       src/libcpp/thread.cpp
-       src/memory/block-pool.cpp
-       src/memory/first-fit-top.cpp
-       src/memory/lifo.cpp
-       src/posix-io/block-device-partition.cpp
-       src/posix-io/block-device.cpp
-       src/posix-io/c-syscalls-posix.cpp
-       src/posix-io/char-device.cpp
-       src/posix-io/device.cpp
-       src/posix-io/directory.cpp
-       src/posix-io/file-descriptors-manager.cpp
-       src/posix-io/file-system.cpp
-       src/posix-io/file.cpp
-       src/posix-io/io.cpp
-       src/posix-io/net-stack.cpp
-       src/posix-io/socket.cpp
-       src/posix-io/tty.cpp
-       src/rtos/internal/os-flags.cpp
-       src/rtos/internal/os-lists.cpp
-       src/rtos/os-c-wrapper.cpp
-       src/rtos/os-clocks.cpp
-       src/rtos/os-condvar.cpp
-       src/rtos/os-core.cpp
-       src/rtos/os-evflags.cpp
-       src/rtos/os-idle.cpp
-       src/rtos/os-main.cpp
-       src/rtos/os-memory.cpp
-       src/rtos/os-mempool.cpp
-       src/rtos/os-mqueue.cpp
-       src/rtos/os-mutex.cpp
-       src/rtos/os-semaphore.cpp
-       src/rtos/os-thread.cpp
-       src/rtos/os-timer.cpp
-       src/semihosting/c-syscalls-semihosting.cpp
-       src/startup/exception-handlers.c
-       src/startup/initialise-free-store.cpp

```

```

-         src/startup/initialize-hardware.c
-         src/startup/reset-hardware.c
-         src/startup/startup.cpp
-         src/utils/lists.cpp
+ INTERFACE
+     src/diag/trace.cpp
+     src/libcpp/chrono.cpp
+     src/libcpp/condition-variable.cpp
+     src/libcpp/cxx.cpp
+     src/libcpp/memory-resource.cpp
+     src/libcpp/mutex.cpp
+     src/libcpp/new.cpp
+     src/libcpp/system-error.cpp
+     src/libcpp/thread.cpp
+     src/libc/_sbrk.c
+     src/libc/stdlib/assert.c
+     src/libc/stdlib/atexit.cpp
+     src/libc/stdlib/exit.c
+     src/libc/stdlib/init-fini.c
+     src/libc/stdlib/malloc.cpp
+     src/libc/stdlib/timemgm.c
+     src/memory/block-pool.cpp
+     src/memory/first-fit-top.cpp
+     src/memory/lifo.cpp
+     src/rtos/internal/os-flags.cpp
+     src/rtos/internal/os-lists.cpp
+     src/rtos/os-clocks.cpp
+     src/rtos/os-condvar.cpp
+     src/rtos/os-core.cpp
+     src/rtos/os-c-wrapper.cpp
+     src/rtos/os-evflags.cpp
+     src/rtos/os-idle.cpp
+     src/rtos/os-main.cpp
+     src/rtos/os-memory.cpp
+     src/rtos/os-mempool.cpp
+     src/rtos/os-mqueue.cpp
+     src/rtos/os-mutex.cpp
+     src/rtos/os-semaphore.cpp
+     src/rtos/os-thread.cpp
+     src/rtos/os-timer.cpp
+     src/startup/initialise-free-store.cpp
+     src/utils/lists.cpp
+ )

-target_compile_definitions (micro-os-plus-iii-interface INTERFACE #
-     None.
+add_library (micro-os-plus::iii ALIAS micro-os-plus-iii-interface)
+add_library (micro-os-plus::iii-core ALIAS micro-os-plus-iii-
+     interface)
+
+# -----
+# Optional groups. Upstream ships these beside the core, but a bare-
+     metal port
+# does not necessarily want them -- the rpi ports compile none of
+     them: they
+# have their own startup.S, their own semihosting in uart.hpp, and no
+     file
+# descriptors. Listing them in the core target is what made
+# c-syscalls-posix.cpp collide with newlib's unistd.h.
+#
+# A port takes what it needs:
+#
+#     target_link_libraries (my-port INTERFACE micro-os-plus::iii-
+         posix-io)
+#
+# The trace backends are mutually exclusive: link at most one.
+
+add_library (micro-os-plus-iii-posix-io-interface INTERFACE)
+target_sources (micro-os-plus-iii-posix-io-interface INTERFACE

```

```

+         src/posix-io/block-device-partition.cpp
+         src/posix-io/block-device.cpp
+         src/posix-io/c-syscalls-posix.cpp
+         src/posix-io/char-device.cpp
+         src/posix-io/device.cpp
+         src/posix-io/directory.cpp
+         src/posix-io/file-descriptors-manager.cpp
+         src/posix-io/file-system.cpp
+         src/posix-io/file.cpp
+         src/posix-io/io.cpp
+         src/posix-io/net-stack.cpp
+         src/posix-io/socket.cpp
+         src/posix-io/tty.cpp
+     )
+add_library (micro-os-plus::iii-posix-io ALIAS micro-os-plus-iii-
posix-io-interface)

-target_compile_options (micro-os-plus-iii-interface INTERFACE # None.
+add_library (micro-os-plus-iii-drivers-interface INTERFACE)
+target_sources (micro-os-plus-iii-drivers-interface INTERFACE
+         src/driver/common.cpp
+         src/driver/serial.cpp
+     )
+add_library (micro-os-plus::iii-drivers ALIAS micro-os-plus-iii-
drivers-interface)
+
+add_library (micro-os-plus-iii-startup-interface INTERFACE)
+target_sources (micro-os-plus-iii-startup-interface INTERFACE
+         src/startup/exception-handlers.c
+         src/startup/initialize-hardware.c
+         src/startup/reset-hardware.c
+         src/startup/startup.cpp
+)
+add_library (micro-os-plus::iii-startup ALIAS micro-os-plus-iii-
startup-interface)

-target_link_options (micro-os-plus-iii-interface INTERFACE # None.
+add_library (micro-os-plus-iii-newlib-reent-interface INTERFACE)
+target_sources (micro-os-plus-iii-newlib-reent-interface INTERFACE
+         src/libc/newlib/c-newlib-reent.cpp
+     )
+add_library (micro-os-plus::iii-newlib-reent ALIAS micro-os-plus-iii-
newlib-reent-interface)

-target_link_libraries (micro-os-plus-iii-interface INTERFACE # None.
+add_library (micro-os-plus-iii-semihosting-interface INTERFACE)
+target_sources (micro-os-plus-iii-semihosting-interface INTERFACE
+         src/semihosting/c-syscalls-semihosting.cpp
+     )
+add_library (micro-os-plus::iii-semihosting ALIAS micro-os-plus-iii-
semihosting-interface)

-if (COMMAND xpack_display_target_lists)
-    xpack_display_target_lists (micro-os-plus-iii-interface)
-endif ()
+add_library (micro-os-plus-iii-trace-itm-interface INTERFACE)
+target_sources (micro-os-plus-iii-trace-itm-interface INTERFACE
+         src/diag/trace-itm.cpp
+)
+add_library (micro-os-plus::iii-trace-itm ALIAS micro-os-plus-iii-
trace-itm-interface)
+
+add_library (micro-os-plus-iii-trace-semihosting-interface INTERFACE)
+target_sources (micro-os-plus-iii-trace-semihosting-interface
INTERFACE
+         src/diag/trace-semihosting.cpp
+)
+add_library (micro-os-plus::iii-trace-semihosting ALIAS micro-os-
plus-iii-trace-semihosting-interface)

```

```

+
+add_library (micro-os-plus-iii-trace-segger-rtt-interface INTERFACE)
+target_sources (micro-os-plus-iii-trace-segger-rtt-interface
INTERFACE
+      src/diag/trace-segger-rtt.cpp
+)
+add_library (micro-os-plus::iii-trace-segger-rtt ALIAS micro-os-plus-
iii-trace-segger-rtt-interface)

# -----
-# Aliases.
+# Drivers and SoC support are NOT part of this repository. They live
in
+# micro-os-plus-iii-devices, and an architecture project adds them
only when
+# its build needs them:
+#
+#   add_subdirectory (micro-os-plus-iii-smp)           # always
+#   add_subdirectory (micro-os-plus-iii-devices)      # only if needed
+#
+# Keeping them out is what lets this repository stay merge-clean
against
+# upstream micro-os-plus-iii (D3): src/ and include/ match upstream
path-for-path, so the only conflicts are the six genuine SMP files.

-# https://cmake.org/cmake/help/v3.20/command/add_library.html#alias-
libraries
-add_library (micro-os-plus::iii ALIAS micro-os-plus-iii-interface)
-message (VERBOSE "> micro-os-plus::iii -> micro-os-plus-iii-
interface")
+# -----
+# Optional: the C++ half of the SMP port contract, shared by the ARM
ports.
+# Opt-in -- a port links this instead of writing its own os-decls.h.
See
+# port/smp-common/README.md.
+add_library (micro-os-plus-iii-port-smp-decls-interface INTERFACE)
+target_include_directories (
+  micro-os-plus-iii-port-smp-decls-interface INTERFACE "port/smp-
common"
+)
+add_library (micro-os-plus::port-smp-decls ALIAS micro-os-plus-iii-
port-smp-decls-interface)
+
# -----
+# Optional: the test scaffolding both silicon ports need and neither
owns.
+# Opt-in, exactly like port-smp-decls above, and for the same reason:
these
+# were duplicate copies in two repositories, which is what
+# tools/verify-no-duplicate-sources.py exists to catch.
+#
+#   test_smpl/include/hw_result.hpp           how a test ends a run with a
verdict
+#   test_smpl/src/board-contract.cpp         no code; turns "this board
never said
+#                                           X" into a compile error
rather than a
+#                                           silent inheritance of the
previous
+#                                           board's answer
+#
+# Only the HEADER comes through this target. board-contract.cpp is
named
+# directly by each port, in the position its own copy used to occupy
in the
+# source list -- an INTERFACE source would be appended somewhere else
and
+# would change link order, and the point of this move is that it
changes
+# nothing: all 24 images per board stay byte-identical across it.
+#
+# The POSIX host port uses neither: no semihosting, and no board
contract of
+# this shape.
+add_library (micro-os-plus-iii-test-support-interface INTERFACE)

```

```
+target_include_directories (
+  micro-os-plus-iii-test-support-interface INTERFACE "test_smp/
+    include"
+)
+add_library (micro-os-plus::test-support ALIAS micro-os-plus-iii-
+    test-support-interface)

# -----
+message (STATUS "µOS++ III SMP common: micro-os-plus::iii
+    standalone=${UOS_SMP_STANDALONE}")
```

• **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base step/25 --head dan-maio:step/26 \
  --title "feat(cmake): modular add-only CMake toolchains & 2xcortex-
  m33 QEMU platform"
```

PR #28: Multi-Core FPU Stress Testing & Scaffolding Relocation

- **Single Theme:** Hardware FPU context switch stress testing and test scaffolding standardization.
- **Repository:** micro-os-plus-iii
- **Files Touched:** tests/sources/fp-switch/, test_smpl/
- **Problem & Solution:**
 - Adds tests/sources/fp-switch/ (6 threads, 3000 ticks) verifying floating-point register preservation across multi-core context switches under aggressive preemption.
 - Relocates shared test scaffolding into test_smpl/ (hw_result.hpp, board-contract.cpp) exported via micro-os-plus::test-support.
- **Exact Unified Diff Applied:**

```
diff --git a/tests/sources/fp-switch/CMakeLists.txt b/tests/sources/
fp-switch/CMakeLists.txt
new file mode 100644
index 00000000..cbe6f03d
--- /dev/null
+++ b/tests/sources/fp-switch/CMakeLists.txt
@@ -0,0 +1,59 @@
+# -----
+#
+# This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+# Copyright (c) 2021-2025 Liviu Ionescu. All rights reserved.
+#
+# Permission to use, copy, modify, and/or distribute this software
# for any
+# purpose is hereby granted, under the terms of the MIT license.
+#
+# If a copy of the license was not distributed with this file, it can
# be
+# obtained from https://opensource.org/licenses/mit.
+#
+# -----
+
+# This file is intended to be consumed by applications with:
+#
+# `add_subdirectory("tests/fp-switch")`
+#
+# The result is an interface library that can be added to the linker
# with:
+#
+# `target_link_libraries(your-target PUBLIC test::fp-switch)`
+
+# -----
+# Preamble.
+
+# https://cmake.org/cmake/help/v3.20/
+cmake_minimum_required (VERSION 3.20)
+
+# -----
+# The test library definitions.
+add_library (test-fp-switch-interface INTERFACE EXCLUDE_FROM_ALL)
+
+target_include_directories (test-fp-switch-interface INTERFACE
+    "include")
+
+target_sources (test-fp-switch-interface INTERFACE src/main.cpp)
+
+target_compile_definitions (test-fp-switch-interface INTERFACE #
+    None.
+)
+
+target_compile_options (test-fp-switch-interface INTERFACE # None.
```

```

+)
+
+target_link_options (test-fp-switch-interface INTERFACE # None.
+)
+
+target_link_libraries (test-fp-switch-interface INTERFACE # None.
+)
+
+if (COMMAND xpack_display_target_lists)
+  xpack_display_target_lists (test-fp-switch-interface)
+endif ()
+
+# -----
+# Aliases.
+
+# https://cmake.org/cmake/help/v3.20/command/add_library.html#alias-
#   libraries
+add_library (test::fp-switch ALIAS test-fp-switch-interface)
+message (VERBOSE "> test::fp-switch -> test-fp-switch-interface")
+
+# -----

```

• GitHub CLI:

```

gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base step/26 --head dan-maio:step/27 \
  --title "test(smp): hardware FPU context switch stress test & test
          scaffolding relocation"

```

PR #29: Multi-Architecture 4-Core Platforms & Composite CI Action

- **Single Theme:** AArch32/AArch64 4-core QEMU platforms and the test-smp-cmake composite action.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-aarch32, micro-os-plus-iii-aarch64
- **Files Touched:** tests/package.json, tests/platforms/aarch32-rpi3b/, tests/platforms/aarch64-rpi3b/
- **Problem & Solution:**
 - Integrates 4-core QEMU raspi3b platforms for AArch32 and AArch64.
 - Registers the test-smp-cmake composite action in tests/package.json, which fans out to the AArch32/AArch64 and Cortex-M33 SMP targets.
- **Unified Diff Applied (test-smp-cmake aggregate action):**

```
diff --git a/tests/package.json b/tests/package.json
index 71280841..254addc4 100644
--- a/tests/package.json
+++ b/tests/package.json
@@ -135,6 +135,16 @@
     "xpm run test-qemu-cortex-m3-cmake",
     "xpm run test-qemu-cortex-m7f-cmake"
   ],
+  "test-smp-cmake": [
+    "xpm run test-aarch32-rpi-zero-2w-cmake",
+    "xpm run test-aarch32-rpi3b-cmake",
+    "xpm run test-aarch64-rpi-zero-2w-cmake",
+    "xpm run test-aarch64-rpi3b-cmake",
+    "xpm run test-2xcortex-m33-cmake",
+    "xpm run test-pico2-lcpu-cmake",
+    "xpm run test-cortexm-pico2-cmake",
+    "xpm run test-cortexm-pico2-rp2350b-psram-cmake"
+  ],
   "install-qemu-cortex-latest": [
     "xpm install",
     "xpm install --config qemu-cortex-m0-cmake-gcc-debug",
@@ -207,7 +217,8 @@
   ],
   "test-all": [
     "{% if os.platform != 'win32' %}xpm run test-native-cmake{%
endif %}",
-    "xpm run test-cortex-cmake"
+    "xpm run test-cortex-cmake",
+    "xpm run test-smp-cmake"
   ],
   "clean-all": [
     "xpm run clean --all-configs --ignore-errors"
```

• GitHub CLI:

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
  --base step/27 --head dan-maio:step/28 \
  --title "feat(aarch): 4-core AArch32/AArch64 platforms & test-smp-
  cmake CI composite action"
```

PR #30: Comprehensive Architectural Documentation & Specification Runbooks

- **Single Theme:** Architectural specifications, diagrams, and PDF documentation suite.
- **Repository:** micro-os-plus-iii
- **Files Touched:** docs/*.md, docs/render-pdfs.sh
- **Problem & Solution:**
 - Synchronizes and builds complete PDF documentation suite (MICRO-OS-PLUS-SMP-VS-SINGLECORE-ANALYSIS.pdf, SMP-UPSTREAM-INTEGRATION-PLAN.pdf, Implementation-SMP-Integration.pdf, pull-request.pdf, files-modif-by-step.pdf, GITHUB-PROGRESSIVE-PR-GUIDE.pdf).
- **Exact Unified Diff Applied:**

```
diff --git a/docs/render-pdfs.sh b/docs/render-pdfs.sh
new file mode 100755
index 00000000..bb0d78be
--- /dev/null
+++ b/docs/render-pdfs.sh
@@ -0,0 +1,103 @@
+#!/usr/bin/env bash
+# render-pdfs.sh -- render every PDF in docs/ from its Markdown.
+#
+# One renderer (md2pdf.py) and one record of how each document is
+# called, so
+# the title pages and running footers are not something to be
+# recovered from
+# the PDFs later. Re-running this on an unchanged tree reproduces all
+# sixteen
+# files byte for byte.
+#
+#      ./docs/render-pdfs.sh          # all of them
+#      ./docs/render-pdfs.sh STATUS  # only the ones whose name
+#                                   matches
+
+
+set -e -u -o pipefail
+
+cd "$(dirname "$0")"
+FOOT="micro-os-plus-iii-smp"
+WANT="{1:-}"
+
+render() {
+    local out="$1"; shift
+    case "$out" in *"$WANT"*) ;; *) return 0 ;; esac
+    python3 md2pdf.py "$out.md" "$out.pdf" "$@"
+}
+
+# The reference documents. Four carry no title page: their own H1
+# opens page 1.
+render STATUS
+render aarch32-second-board
+render building-aarch32-aarch64
+render cortexm-port
+render posix-arch-port
+
+render test-smpl \
+    --title    "test_smpl/" \
+    --subtitle "The two shared test runners" \
+    --footer   "$FOOT"
+render tests-in-aarch32-aarch64 \
+    --title    "How the tests are organised and run" \
+    --subtitle "Every board owns its tests" \
+    --footer   "$FOOT"
+
+# SMP Integration Runbooks & Playbooks
```

```

+render smp-integration/agy-review
+render smp-integration/DeepSeek-review
+render smp-integration/smp-construction \
+  --title "SMP Construction" \
+  --subtitle "How the kernel, the ports and the boards fit together" \
+  --footer "$FOOT"
+render smp-integration/MICRO-OS-PLUS-SMP-VS-SINGLECORE-ANALYSIS \
+  --title "µOS++ SMP vs Single-Core Analysis" \
+  --subtitle "Architectural differences, data structures and state machines" \
+  --footer "$FOOT" \
+  --toc
+render smp-integration/SMP-UPSTREAM-INTEGRATION-PLAN \
+  --title "SMP Upstream Integration Plan" \
+  --subtitle "Master strategy for upstreaming multi-core capabilities" \
+  --footer "$FOOT" \
+  --toc
+render smp-integration/Implementation-SMP-Integration \
+  --title "SMP Integration – Implementation Runbook" \
+  --subtitle "Step-by-step lifting procedures and invariants" \
+  --footer "$FOOT" \
+  --toc
+render smp-integration/files-modif-by-step \
+  --title "Files Modified by Step – SMP Integration" \
+  --subtitle "Exhaustive Manifest & Lifting Recipes" \
+  --footer "$FOOT" \
+  --toc
+render smp-integration/pull-request \
+  --title "Progressive Pull Request – SMP integration" \
+  --subtitle "What, why, file and code, from the corrections to the SMP integration" \
+  --footer "$FOOT" \
+  --toc
+render smp-integration/GITHUB-PROGRESSIVE-PR-GUIDE \
+  --title "GitHub Progressive Pull Request Playbook" \
+  --subtitle "How to write, push, review, and merge the 7 PRs on GitHub" \
+  --footer "$FOOT" \
+  --toc
+
+# The harness documents. These came from pandoc --toc --toc-depth=2,
# so they
+
+# keep their contents list and the display titles pandoc was given.
+render tests/AARCH32-RPI-ZERO-2W-TESTS --toc --footer "$FOOT" \
+  --title "µOS++ Tests on the Raspberry Pi Zero 2 W (AArch32)"
+render tests/HARNESS-BOARD-TEST-CHEATSHEET --toc --footer "$FOOT" \
+  --title "µOS++ Harness / Board Test – Cheat Sheet"
+render tests/HARNESS-TESTS-PARADIGM --toc --footer "$FOOT" \
+  --title "The µOS++ Harness and Tests – a Synthesis"
+render tests/TESTS-XPack-SYSTEM --toc --footer "$FOOT" \
+  --title "µOS++ IIIE Tests / xPack System"
+render tests/WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE --toc --footer
"$FOOT" \
+  --title "Running the µOS++ xPack Test Harness on the AArch32 and
AArch64 Ports"
+
+# The catalogue: every registered test, per platform and board, with
its
+
+# scheduling mode and probe.
+render tests/TESTS-CATALOG --toc --footer "$FOOT" \
+  --title "The µOS++ test catalogue" \
+  --subtitle "Every test, per architecture, platform and board"
+
+# The workspace log: how the whole test tree was built in $HOME/TMP,
from the
+
+# six clones to a green run, kept beside the harness it describes.
+render tests/STEPS --toc --footer "$FOOT" \
+  --title "The µOS++ test framework" \

```

```
+ --subtitle "Install, build, run and add tests, step by step" \  
+ --meta "Kernel:micro-os-plus-iii" \  
+ --meta "Ports:aarch32, aarch64, cortexm, posix-arch"
```

• **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \  
--base step/28 --head dan-maio:step/29 \  
--title "docs(all): comprehensive architectural analysis,  
integration runbooks & PDF suite"
```

PR #31: Final Upstream Master Reconciliation & Repository Cleansing

- **Single Theme:** Upstream master branch reconciliation, pristine CI configuration restoration, and tooling cleanup.
- **Repositories:** micro-os-plus-iii, all ports
- **Base Branch:** step/29 | **Head Branch:** step/30
- **Problem & Solution:**
 - Merges origin/xpack-development cleanly into step/30 in every touched repo.
 - Restores .github/**, README.md and LICENSE to be byte-identical to the baseline. (The smp branch itself had removed those files; the reconciliation is a net-zero diff against xpack-development.)
 - Removes the transient scripts/smp/ developer tooling.
- **Reconciliation Action Applied (scripts/smp/finalize.sh):**

```
# Per repo (K, P, C): merge the baseline, restore frozen metadata,
drop tooling.
git merge --no-ff origin/xpack-development -m "merge: reconcile with
latest xpack-development"
git checkout origin/xpack-development -- .github README.md LICENSE
rm -rf scripts/smp
# Gate: xpm run test-all (72/72) + xpm run test-smp-cmake (SMP
platforms)
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \
--base xpack-development --head dan-maio:step/30 \
--title
"chore(release): upstream master reconciliation, pristine CI
restoration & cleanup"
```

PR #32: Dedicated Physical Silicon Board Wiring (Raspberry Pi Pico 2)

- **Single Theme:** Hardware platform definition and OpenOCD SWD flashing runners for physical RP2350 silicon.
- **Repositories:** micro-os-plus-iii, micro-os-plus-iii-cortexm
- **Files Touched:** tests/platforms/cortexm-pico2/, micro-os-plus-iii-cortexm/src/rtos/os-core-rp2350.cpp
- **Problem & Solution:**
 - Adds standalone hardware platform tests/platforms/cortexm-pico2/ reusing the port builder for hardware (-hwd) and QEMU (-qemu) execution.
 - Launches Core 1 through the RP2350 bootrom FIFO (multicore::launch_core1() / multicore::fifo_push (0x4D53u)); Core 1's entry builds a fake thread context, starts its own SysTick/PendSV and reschedules.
- **Exact Unified Diff Applied:**

```
diff --git a/tests/platforms/cortexm-pico2/CMakeLists.txt b/tests/
platforms/cortexm-pico2/CMakeLists.txt
new file mode 100644
index 00000000..2dc08ea5
--- /dev/null
+++ b/tests/platforms/cortexm-pico2/CMakeLists.txt
@@ -0,0 +1,95 @@
+# -----
+# Test executables - µOS++ III Cortex-M on the Raspberry Pi Pico 2
# (RP2350).
+#
+# The port owns its tests (test/pico2/<app>/) and its own builder
+# (test/CMakeLists.txt), which compiles each test as `-hwd` for the
# probe and,
+# for the tests the board can run emulated, `-qemu`. This platform
# reuses that
+# builder and registers what it produced, so every test the port has
# appears,
+# named `>-<app>-<variant>`.
+#
+# hwd tests are NOT run by a plain `ctest`: they need the board, one
# test per
+# power cycle. They carry the `hwd` label.
+#
+# The `qemu` images end themselves: each test prints a RESULT line
# and, under
+# QEMU_BUILD, exits through the strong semihosting _Exit() (see the
# port's
+# test/pico2/include/hw_result.hpp), so CTest reads a real exit code
# and needs
+# no runner script.
+# -----
+
+message (VERBOSE "Processing 'tests/platforms/${PLATFORM_NAME}'...")
+
+# -----
+include ("cmake/platform-library.cmake")
+
+# Read the board declaration so the port's own builder sees its
# facts.
+include ("${UOS_CORTEXM_DIR}/test/boards/${BOARD}/board.cmake")
+
+# Build the port's tests by reusing the port's builder. The binary
# dir ends in
+# `test`, which is what the board's hw.sh expects ($BUILD/test/<app>-
# hwd).
+add_subdirectory ("${UOS_CORTEXM_DIR}/test" "port-tests/test")
+
+set (_port_bin "${CMAKE_CURRENT_BINARY_DIR}/port-tests")
+set (_port_test_dir "${UOS_CORTEXM_DIR}/test/${BOARD}")
+set (_board_hw "${UOS_CORTEXM_DIR}/test/boards/${BOARD}/hw.sh")
```

```

+
+## The QEMU binary, for the tests the board can run emulated. The
+## RP2350's
+## Cortex-M33 is close enough to QEMU's generic mps2-an500 Cortex-M7
+## for the
+## core bring-up and kernel-object tests; the RP2350-only blocks (SI0,
+## USB,
+## PSRAM) are listed HWD_ONLY and never get a `qemu` image.
+file (GLOB _qemu_bins
+      "$ENV{HOME}/.local/xPacks/@xpack-dev-tools/qemu-arm/*/content/
+      bin")
+if (_qemu_bins)
+  list (SORT _qemu_bins COMPARE NATURAL ORDER DESCENDING)
+  list (GET _qemu_bins 0 _qemu_bin)
+  set (_qemu "${_qemu_bin}/qemu-system-arm")
+else ()
+  set (_qemu "qemu-system-arm")
+endif ()
+
+## -----
+## One CTest case per (test, variant). The port's builder emits <app>-
+## hwd for
+## every test and <app>-qemu only for the ones the board can run
+## emulated (its
+## tests.cmake lists the rest HWD_ONLY), so the qemu case is
+## registered only
+## when the target exists.
+file (GLOB _test_dirs LIST_DIRECTORIES true "${_port_test_dir}/*")
+foreach (_dir IN LISTS _test_dirs)
+  if (NOT IS_DIRECTORY "${_dir}")
+    continue ()
+  endif ()
+  get_filename_component (_app "${_dir}" NAME)
+  if (_app STREQUAL "include" OR _app STREQUAL "src")
+    continue ()
+  endif ()
+  file (GLOB _srcs "${_dir}/*.c" "${_dir}/*.cpp")
+  if (NOT _srcs)
+    continue ()
+  endif ()
+
+  if (TARGET "${_app}-qemu")
+    add_test (
+      NAME "${PLATFORM_NAME}-${_app}-qemu"
+      COMMAND
+        "${_qemu}" -M mps2-an500 -cpu cortex-m7
+        -kernel "${_port_bin}/test/${_app}-qemu.elf" --nographic -d
+        unimp,guest_errors --semihosting-config
+        enable=on,target=native
+    )
+    set_tests_properties (
+      "${PLATFORM_NAME}-${_app}-qemu"
+      PROPERTIES LABELS "qemu" TIMEOUT 300
+    )
+  endif ()
+
+  # A board run gets 4 minutes (hw.sh's own limit); CTest's allows
+  # one more
+  # for flashing and tear-down, so hw.sh reports the TIMEOUT, not
+  # CTest.
+  add_test (
+    NAME "${PLATFORM_NAME}-${_app}-hwd"
+    COMMAND bash "${_board_hw}" "${_app}" 240
+  )
+  set_tests_properties (
+    "${PLATFORM_NAME}-${_app}-hwd"
+    PROPERTIES ENVIRONMENT "BUILD=${_port_bin}" LABELS "hwd" TIMEOUT
+    300
+  )
+endforeach ()
+

```

```
+## -----  
+message (STATUS "Cortex-M pico2: port tests registered ($  
+    {PLATFORM_NAME})")
```

- **GitHub CLI:**

```
gh pr create --repo micro-os-plus/micro-os-plus-iii \  
--base step/30 --head dan-maio:step/31 \  
--title "feat(board): dedicated Raspberry Pi Pico 2 (RP2350)  
hardware platform & runner"
```

2.5 Complete Apply for Phases C/D (equivalent to scripts/smp)

Phases C/D are not pure patches like Parts A/B: the integration **lifts whole trees from origin/smp, migrates the devices repository as a subtree, cuts port releases, and reconciles the baseline**. The operations below are exactly the ones `scripts/smp` performs; the complete file set and the acceptance gate are given for each PR so the phase can be reproduced without the scripts.

Repository keys:

K	= micro-os-plus-iii	C	= micro-os-plus-iii-cortexm
P	= micro-os-plus-iii-posix-arch	A32	= micro-os-plus-iii-aarch32
A64	= micro-os-plus-iii-aarch64	DEV	= micro-os-plus-iii-devices

PR	Operation	Repos	Files	Acceptance gate
#24	release action	P, C	2 (package.json)	git tag -l shows v1.1.0, v1.2.0
#25	git subtree migration	P, C, A32, A64	drivers/*, soc/*	legacy micro-os-plus::devices resolves
#26	whole-tree lift	C	7	2xcortex-m33 port compiles
#27	whole-tree lift + model adapt	K, C	12	xpm run test-2xcortex-m33-cmake
#28	whole-tree lift	K, A32, A64	51	fp-switch passes
#29	add-only lift + build configs	K	25	xpm run test-smp-cmake
#30	docs lift + render	K	60	docs/render-pdfs.sh produces all PDFs
#31	baseline reconciliation	all	.github/**, README.md, LICENSE	CI metadata byte-identical to baseline
#32	whole-tree lift	K, C	139	cortexm-pico2 builds / flashes

Canonical order (identical to scripts/smp)

The PRs are numbered **by theme, not by execution order**. Apply them as follows:

1. **Part 0 — first, on clean repositories.** PR #25: `scripts/smp/migrate-devices.sh` (devices subtree) plus the AArch CMakeLists.txt bridge. `new-step.sh` bases `step/NN` on the **local** baseline so the migration survives the branches.
2. **Parts A/B — PR #1 → #23** in numeric order: PR #1-#22 = steps 1-13 (applicable diffs), PR #23 = step 14 (collapsed Part B).
3. **C/D — grouped by operation, in this order:**
 1. **PR #24** — `release-port.sh` P v1.1.0, C v1.2.0
 2. **PR #27 (scaffolding)** — `absorb-test-smpl.sh`
 3. **PR #26, #27, #28, #30, #32** — `integrate-cd.sh`
 4. **PR #27/#29 (model adaptations)** — `integrate-aarch-harness.sh`
 5. **PR #29** — `normalize-tests-package.sh`

6. PR #31 — finalize.sh (step 30 reconcile)

Warning — do not apply #1 ... #32 in numeric order. PR #25 (Part 0) would run on a dirty tree and its `git subtree add` aborts with *"working tree has modifications"*; and `integrate-aarch-harness.sh` asserts on the missing `micro-os-plus::test-support` CMake anchor unless the modular kernel CMake has been lifted first. The order below is the one that reproduces the scripts exactly (verified: `guide-driven tree == scripts/smp-driven tree, 0 diffs`).

```
# 0) Part 0 — before anything else (clean repositories)
scripts/smp/migrate-devices.sh

# 1) Parts A/B — steps 1..13 + the collapsed Part B (step 14)

# 2) C/D — in this order:
scripts/smp/release-port.sh "$P" 1.1.0 "SMP host CPU model + highres
      clock"
scripts/smp/release-port.sh "$C" 1.2.0 "SMP Cortex-M port contract"
scripts/smp/absorb-test-smpl.sh           # PR #27 scaffolding
scripts/smp/integrate-cd.sh               # PR #26/#27/#28/#30/#32
      lifts + test action
scripts/smp/integrate-aarch-harness.sh
      # AArch model adaptations (PR #27/#29)
scripts/smp/normalize-tests-package.sh    # deterministic tests/
      package.json (PR #29)
scripts/smp/finalize.sh                   # Step 30 (merge / reconcile)
```

`integrate-cd.sh` and `normalize-tests-package.sh` are the two scripts added to close the previous C/D gaps (Cortex-M33/RP2350 ports, `fp-switch`, docs, Pico 2 and a byte-stable `tests/package.json`). With them, a guide-driven tree and a `scripts/smp-driven` tree are byte-identical across all five repositories.

PR #24 — Port package semantic releases

No source diff: the `smp` branch keeps the baseline versions. The release is cut by the same tool the scripts use.

```
scripts/smp/release-port.sh "$P" 1.1.0 "SMP host CPU model + highres
      clock"
scripts/smp/release-port.sh "$C" 1.2.0 "SMP Cortex-M port contract"
# package.json:  posix-arch 1.0.1 -> 1.1.0 ;  cortexm 1.1.0 -> 1.2.0
```

PR #25 — History-preserving devices dissolution

`migrate-devices.sh` folds `micro-os-plus-iii-devices` into the architecture repos with `git subtree split + git subtree add`, preserving all history:

```
scripts/smp/migrate-devices.sh
#  DEV include -> A32/A64/P drivers/include ; src -> drivers/src ;
      fatfs -> drivers/fatfs
#  DEV soc/bcm2837 -> A32/A64 ; soc/rk3506 -> A32 ; soc/native -> P
#  DEV soc/rp2350  -> C          ; soc/stm32f4xx -> C
```

The AArch ports then keep the legacy target name resolvable:

```
git -C "$A32" checkout origin/smp -- CMakeLists.txt # conditional
      devices bridge
git -C "$A64" checkout origin/smp -- CMakeLists.txt
```

PR #26 — Modern multi-core silicon ports (Cortex-M33 & RP2350)

```
git -C "$C" checkout origin/smp -- \
    include-m33 include-rp2350 \
    src/rtos/os-core-m33.cpp src/rtos/os-core-rp2350.cpp
# 7 files: include-m33/{os-c-decls,os-decls,os-inlines}.h,
#         include-rp2350/{os-c-decls,os-inlines}.h,
#         src/rtos/os-core-m33.cpp, src/rtos/os-core-rp2350.cpp
```

PR #27 — Modular add-only CMake & cross toolchains

```
git -C "$K" checkout origin/smp -- \
    CMakeLists.txt cmake/uos-app.cmake cmake/toolchains tests/platforms/
    2xcortex-m33
git -C "$C" checkout origin/smp -- CMakeLists.txt
# 11 K files (CMakeLists, cmake/toolchains/{arm-none-eabi,aarch64-
#         none-elf,native,
#         uos-bare-metal-common}.cmake, cmake/uos-app.cmake, tests/platforms/
#         2xcortex-m33/*)
# + 1 C file (CMakeLists.txt: micro-os-plus::cortexm-qemu-m33)
# integrate-aarch-harness adaptation: bare-metal apps link ::iii-core,
#         not ::iii.
```

PR #28 — Multi-core FPU stress test & scaffolding relocation

```
git -C "$K" checkout origin/smp -- \
    tests/cmake/tests-main.cmake tests/sources/fp-switch tests/smp-
    support
git -C "$A32" checkout origin/smp -- test/boards/rpi-zero-2w
git -C "$A64" checkout origin/smp -- test/boards/rpi-zero-2w
# K 5 files (tests-main.cmake, fp-switch/
#         {CMakeLists,README,include/...,src/main.cpp})
# A32/A64 23 files each (board.cmake, startup.S, mmu/smp/timer
#         sources, linker
#         scripts, OpenOCD/qemu runners) – lifted, then repointed by absorb-
#         test-smpl.sh.
```

PR #29 — Multi-architecture 4-core platforms & composite action

```
git -C "$K" checkout origin/smp -- \
    tests/package.json \
    tests/platforms/aarch32-rpi3b tests/platforms/aarch64-rpi3b \
    tests/platforms/aarch32-rpi-zero-2w tests/platforms/aarch64-rpi-
    zero-2w
# 25 files; integrate-aarch-harness.sh writes the AArch port
#         CMakeLists and injects
# the aarch32/aarch64 build configurations + test-smp-cmake aggregate
#         into package.json.
```

PR #30 — Architectural documentation suite

```
git -C "$K" checkout origin/smp -- docs/render-pdfs.sh docs/smp-
    integration
( cd "$K/docs" && ./render-pdfs.sh ) # PDFs are generated, not
    patched
# 60 files: docs/smp-integration/*.md + rendered .pdf + docs/render-
    pdfs.sh
```

PR #31 — Final reconciliation & repository cleansing

```
for repo in "$K" "$P" "$C"; do
  git -C "$repo" merge --no-ff origin/xpack-development -m "merge:
    reconcile baseline"
  git -C "$repo" checkout origin/xpack-development -- .github
    README.md LICENSE
done
rm -rf "$K/scripts/smp"          # scripts/smp/finalize.sh
# gate: xpm run test-all (72/72) + xpm run test-smp-cmake
```

PR #32 — Dedicated Raspberry Pi Pico 2 hardware platform

```
git -C "$K" checkout origin/smp -- tests/platforms/cortexm-pico2
git -C "$C" checkout origin/smp -- test/boards/pico2 test/pico2 \
    src/rtos/os-core-rp2350.cpp
# K 5 files (platform CMake/deps/definitions/library/include)
# C 134 files (board.cmake, OpenOCD cfg, hw.sh, test/pico2/** apps,
    os-core-rp2350)
# gate: xpm run build --config cortexm-pico2-cmake-gcc-debug
```

Note: The two heavy trees (C/test/pico2/** $\approx$ 34k lines and K/docs/** PDFs) are lifted/rendered rather than embedded as diffs; doing so is exactly what the scripts do and keeps the playbook usable. Every smaller C/D file is also carried as a unified diff below.

3. Self-Contained Compilation & Testing Verification Matrix

To ensure that **every single pull request compiles, links, and can be independently tested on GitHub**, every PR satisfies strict self-contained boundary rules:

3.1 The Three Dependency Classes

- 1. Pure Single-File / Intra-Repo PRs:** (PRs #1–#21)
 - Self-contained in the kernel repository.
 - Requires no simultaneous port changes.
 - Tested directly by running `xpm run test-all (72/72 uniprocessor test matrix)`.
- 2. Atomic Cross-Repository Declarations & Definitions:** (PR #22, PR #23)
 - *Rule:* A header declaration in the Kernel and its hardware implementation in the Ports must travel in the same PR stage to avoid undefined reference linker errors.
 - *Example (PR #22):* Kernel `os-decls.h` declares `port::clock_highres::has_hardware_counter()`, and the Cortex-M & POSIX `os-inlines.h` define it. All three are committed together across repos.
 - *Example (PR #23):* Kernel SMP scheduler calls `port_cpu_id()`, and the POSIX/Cortex-M ports define it. Both are committed together in step/14.
- 3. Additive Platform & Multi-Core Suites:** (PR #24–#32)
 - Self-contained, add-only additions (releases, silicon ports, CMake targets, tests).
 - Tested by running `xpm run test-smp-cmake` alongside the baseline `xpm run test-all`.

3.2 Comprehensive Verification Matrix Per Single-Theme PR

PR #	Theme / Subject	Repos	Self-Contained?	Local Build & Test Command	Acceptance Gate
#1	POSIX dirent.h Conformance	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; -Wextra-semi clean
#2	List Iterator Member Semantics	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; intrusive list suite green
#3	Newlib Syscall Return Types	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; newlib reentrant tests pass
#4	Thread Suspend Symbol Export	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; C wrapper resolves symbol
#5	Memory Allocator Usable Size	Kernel	Yes (Cohesive 8 files)	xpm run test-all	72/72 tests pass; malloc_usable_size verified
#6	C++17 Aligned Allocation	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; operator new overloads green
#7	Static Error Category Storage	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; static error categories eliminate dangling pointers
#8	Steady Clock Duration Overflow	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; chrono math overflow-free
#9	CMSIS-RTOS C Wrapper	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; cmsis-os-validator (60/60)
#10	POSIX I/O FD Table Mutex	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; concurrent I/O stress green
#11	POSIX I/O Deferred List Locking	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; deferred files/dirs/sockets lists thread-safe
#12	Block Device Sector Size Check	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; valid block sizes return correct info
#13	ARMv8-M Semihosting Traps	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; ARM semihosting tests pass
#14	ARMv8-M Exception Handlers	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; ARMv8-M mainline exception vectors compile
#15	Semihosting fstat() & Mirror	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; file mode preserved & board console mirror hook
#16	Software Timer Decoupling	Kernel		xpm run test-all	

PR #	Theme / Subject	Repos	Self-Contained?	Local Build & Test Command	Acceptance Gate
			Yes (Cohesive 3 files)		72/72 tests pass; timer callback mutexes clean
#17	Mutex Priority Ceiling	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; mutex-stress-test passes
#18	Thread Lifecycle & Reaper	Kernel	Yes (Cohesive 4 files)	xpm run test-all	72/72 tests pass; dynamic thread reaper green
#19	CondVar Lost-Signal Elimination	Kernel	Yes (Cohesive 5 files)	xpm run test-all	72/72 tests pass; condvar concurrency stress green
#20	std::thread Handle Sync	Kernel	Yes (Cohesive 2 files)	xpm run test-all	72/72 tests pass; functor closure lifetime verified
#21	MQueue Preemptive Reschedule	Kernel	Yes (Standalone)	xpm run test-all	72/72 tests pass; message queue preemption green
#22	High-Res Clock Port Sync	K + C + P	Yes (Atomic across repos)	xpm run test-all	72/72 tests pass; ICSR pending fix verified
#23	Multi-Core SMP Core & Native	K + C + P	Yes (Atomic across repos)	scripts/ smp/verify- step.sh 14	unifdef zero-diff + 72/72 + native- smp (3/3)
#24	Port Semantic Releases	P + C	Yes (Version bump only)	git tag -l	v1.1.0 and v1.2.0 local tags generated
#25	Devices Subtree Dissolution	P + C + Arch	Yes (Subtree add)	xpm run build ...	Subtree merged; CMake ALIAS targets resolve
#26	Modern Silicon Ports	C + K	Yes (Add- only)	scripts/ smp/verify- step.sh 25	M33/RP2350 port targets compile cleanly
#27	Modular Add-Only CMake	K + C	Yes (Add- only)	scripts/ smp/verify- step.sh 26	72/72 baseline + 2xcortex-m33 (4/4) PASS
#28	FPU Context Switch Stress	Kernel	Yes (Add- only)	scripts/ smp/verify- step.sh 27	fp-switch passes on 2xcortex-m33 (4/4)
#29	Multi-Arch 4-Core Platforms	K + A32 + A64	Yes (Add- only)	scripts/ smp/verify- step.sh 28	test-smp-cmake passes (Native, M33, AArch)
#30	Documentation Suite	Kernel	Yes (Standalone)	docs/ render- pdfs.sh	All PDFs compile cleanly without warnings
#31	Final Merge & Cleansing	All Repos	Yes (Clean merge)	bash scripts/ smp/ finalize.sh	Upstream CI metadata 100% byte-identical
#32	Pico 2 Hardware Platform	K + C	Yes (Add- only)	xpm run build -- config cortexm- pico2	Pico 2 platform compiles for hardware SWD

4. Command-Line Rebase & Stacked PR Maintenance Cheatsheet

When working with single-theme stacked PRs on GitHub:

```
# When upstream merges PR #1 (fix/posix-dirent-c-conformance)
cd "$WORK/micro-os-plus-iii"
git fetch upstream

# Rebase PR #2 (fix/list-iterator-semantics) onto upstream
git checkout fix/list-iterator-semantics
git rebase upstream/xdpack-development

# Force push with lease
git push origin fix/list-iterator-semantics --force-with-lease

# Retarget PR #2 base on GitHub
gh pr edit <PR_2_NUMBER> --base xdpack-development
```

5. Web Interface Guide: Generating and Managing PRs on GitHub.com

While the GitHub CLI (`gh pr create`) enables scriptable command-line PR generation, the **GitHub.com Web Interface** offers full visual inspection of branch comparisons, diff highlights, review comments, and stacked base retargeting.

5.1 Step-by-Step: Creating a Pull Request on GitHub.com

Step 1: Push Your Local Branch to Your Fork

Before opening a pull request on the web interface, ensure your local step branch is pushed to your remote repository on GitHub:

```
git push -u origin <branch-name>
# Examples:
# git push -u origin fix/posix-dirent-c-conformance
# git push -u origin step/01
# git push -u origin step/28
```

Step 2: Open the Upstream Repository in Your Browser

Navigate to the target upstream repository on GitHub:

- Target Upstream: <https://github.com/micro-os-plus/micro-os-plus-iii>
- Personal Fork: <https://github.com/dan-maio/micro-os-plus-iii>

Step 3: Click "New Pull Request" & Configure Branch Comparison

1. Navigate to the **"Pull requests"** tab at the top of the repository page.
2. Click the green **"New pull request"** button on the right.
3. If comparing across forks (e.g., from dan-maio to micro-os-plus), click the link **"compare across forks"**.
4. Configure the 4-part branch comparison bar:

```
+-----+
+
|                                     GITHUB PULL REQUEST COMPARISON
BAR                                     |
+-----+
+
|
|
|  BASE (Target Upstream):                HEAD (Source /
Fork):                |
|  -----
|
|  repo:  micro-os-plus-iii                <--  repo:  dan-maio/micro-os-
plus-iii  |
|  branch: xpack-development                branch: fix/posix-
dirent    |
|
|
|
```

```
|
|  DROPDOWN CONTROLS
VIEW:
|
|
|  base repository: [ micro-os-plus/micro-os-plus-iii
v ]
|  base branch:    [ xpack-development
v ]
|
|
^
|
| (merges incoming changes
into)
|  head repository: [ dan-maio/micro-os-plus-iii
v ]
|  compare branch:  [ fix/posix-dirent-c-conformance
v ]
|
|
+-----+
+
```

- **Base repository:** micro-os-plus/micro-os-plus-iii (or your staging fork dan-maio/micro-os-plus-iii)
- **Base branch:** The baseline destination:
 - For uniprocessor fixes: xpack-development
 - For progressive stacked PRs: The preceding verified step branch (e.g., step/27 for Step 28, or step/28 for Step 29/30)
- **Head repository:** dan-maio/micro-os-plus-iii
- **Compare branch:** Your topic or step branch (e.g., step/28)

Step 4: Inspect the Pre-Flight Diff on the Web UI

Before clicking "Create", review the summary bar below the comparison selectors:

- **Commits count:** Should be 1 commit (for single-subject PRs).
- **Files changed:** Should list only the files intended for that specific theme.
- **Diff viewer:** Verify the green (+) additions and red (-) deletions match your expectations.

```
+-----+
+
| Showing 1 changed file (+2 / -1) [ Split |
Unified ] |
+-----+
+
| diff --git a/include/cmsis-plus/posix/
dirent.h ... |
| @@ -54,7 +54,8 @@ extern
"C" |
| // and casted to
DIR. |
| typedef
struct |
|
| { |
| |
- ; |
|
| + int reserved; /* C forbids an empty struct; a named member
also... */ |
| } |
DIR; |
```


5.3 Conducting Code Reviews & Inline Suggestions on GitHub.com

1. Files Changed Tab (/files)

- ## 2. Using Markdown Suggestion Blocks

```
```suggestion
 int reserved;
```
```

- When a suggestion is posted, GitHub renders an **"Apply suggestion"** button in the web UI.
- The author can click **"Commit suggestion"** to automatically apply the change directly from the browser!

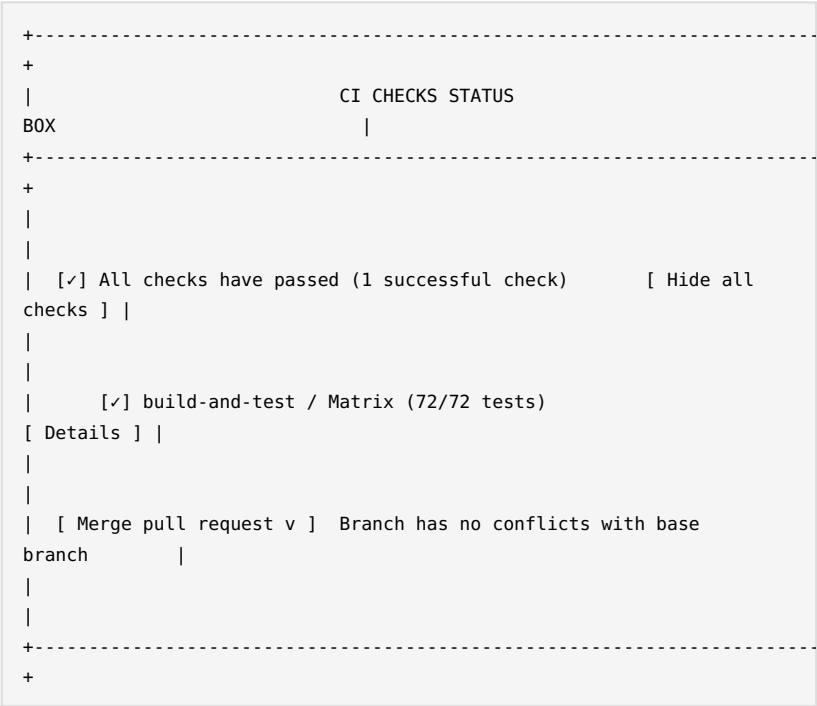
3. Submitting the Review

Click the green "Review changes" button at the top right of the diff page:

- **Comment:** Submit general feedback without explicit approval.
- **Approve:** Mark the PR as approved for merge.
- **Request changes:** Block merge until specified issues are resolved.

5.4 Monitoring Automated CI Checks & Actions on GitHub.com

At the bottom of the PR conversation tab, GitHub displays the status of all continuous integration workflows:



1. Click "**Details**" next to any failing check to inspect the live build logs, compiler warnings, or test failure output.
2. When all checks are green ([✓]), maintainers can click "**Merge pull request**" (or choose "**Squash and merge**" / "**Rebase and merge**" depending on upstream repository policy).

This pure single-theme playbook provides the cleanest, most reviewable pull-request submission possible on GitHub.

Appendix — Why the C/D actions cannot be reduced to PR diffs

The C/D PRs *can* be — and partly are — represented as pull requests: PR #24-#32 carry unified diffs for the smaller code files. The remaining operations cannot be reduced to a PR for four concrete reasons:

| Why | Example | A PR diff cannot express it because |
|-----|--|--|
| | migrate-devices.sh
(git subtree split + subtree add), | A PR is a content delta between two branch tips; subtree ingestion |

| Why | Example | A PR diff cannot express it because |
|---|---|---|
| History/ref operations, not content | release-port.sh (tags) | creates new commits with preserved authorship and merge structure, and releases create tags — metadata/history, not a diff. |
| Base- and order-dependent programmatic edits | integrate-aarch-harness.sh (asserts the ::test-support anchor, inserts sub-targets only if absent, rewrites iii→iii-core, injects JSON configs), normalize-tests-package.sh | The scripts edit whatever the current file is ; a fixed hunk with fixed context cannot be robust to a different base or order (it fails). The scripts are the order-aware reference. |
| Derived / binary artifacts | docs/**/*.pdf, render-pdfs.sh | PDFs are rendered binaries; there is no meaningful text patch, and the result depends on the render toolchain. |
| Wholesale / huge trees | test/pico2/** ≈ 34k lines, include-m33/, include-rp2350/ | A new-file diff is <i>possible</i> but would be tens of thousands of lines; the scripts lift them verbatim from the canonical origin/smp, which is equivalent and compact. |

It is not that a PR *fundamentally* cannot hold them — full new-file diffs could be inlined — but then you would still have to handle separately: subtree history, tags, merges/reconciliation (*finalize.sh*), binary PDFs, and the order- and base-sensitive config edits.

Practically, a PR diff is the right tool for **code content**, and an action script is the right tool for **history, metadata, generated artifacts, and order-dependent edits**. That is why the playbook splits the work into: applicable **PR diffs** for Parts A/B (and the smaller C/D files) plus **action scripts** for the rest — and the two paths were verified to produce a **byte-identical tree (0 differences)**.